

Advanced Features

Power users only!

The primary reason we hide these features by default is because new users stumble upon them without realizing that they can have unintended consequences. These features give you enough power to break your station or installation, so they should be used with caution.

If you modify an advanced feature and it breaks something, your best recourse is to revert the changes you made. If you make custom configuration to Liquidsoap or Icecast and those changes don't work, you should use the Log Viewer utility to view the logs of those respective tools; the issue may be with either your code or a bug in those pieces of software.

Enabling Advanced Features

In new installations of AzuraCast, we turn off some of the more "dangerous" features which can have unintended consequences if misused by users who are unfamiliar with them.

Advanced features hidden by default include:

- Station custom port assignments (which are not cleared through the firewall by default in new Docker installations),
- Providing a custom media base directory (because only certain paths are mounted into the Docker filesystem),
- Customizing Icecast or Liquidsoap configuration (as errors caused by custom configuration are not caused by AzuraCast itself), and
- Advanced playlist settings (because they bypass AzuraCast's AutoDJ).



On this page >

advanced features of AzuraCast.

Broadcasting

Custom Configuration

A common use case of the station-wide Icecast config is to publish your station's mount points to [YellowPage directories](#).

Additionally, you can overwrite some values such as `<location>` and the `<admin>` email address.

 Edit page

← Previous
Streaming Software

Next →
AzuraRelay

AzuraCast APIs

Once installed and running, AzuraCast exposes an API that allows you to monitor and interact with your stations. You can perform the following functions and more from the JSON REST API:

- View now-playing data and recent song history for all stations
- View general station information
- Submit song requests (if allowed by the station)
- Start, stop and restart stations individually

Per-Install API Documentation

Each AzuraCast installation includes documentation for the API at the exact version it's currently using. In our public documentation, we include the API as of the latest Rolling Release version.

- [View API Documentation](#)

API Authentication

If you're accessing sensitive information or modifying the server, you will be required to authenticate your API requests with an authorization key.

You can create an API key from the AzuraCast web interface, by clicking the user menu in the top right and clicking "My API Keys". Any API keys you create will share the same permissions that you have as a user.

The preferred method of authenticating is to send the following header along with your API request:



On this page >

Now Playing Data APIs

The most important and frequently accessed pieces of information that AzuraCast stores are all served as part of a single group of data, which we refer to as the “Now Playing” data.

Because of how valuable this information is, we serve it in a number of ways depending on whether performance or flexibility is your main concern.

Check out our [Now Playing Data APIs](#) page for more detailed information on these APIs.

SDKs & API clients

PHP

- Packagist: [azuracast/php-api-client](#)
- GitHub: <https://github.com/AzuraCast/php-api-client>

Edit page

← Previous
Getting Started

Next →
Now Playing Data APIs

Plugins

AzuraCast contains a plugin system that can be used to extend AzuraCast without having to modifying AzuraCasts code. When you need to modify AzuraCasts behavior you should use the plugin system so that you can still easily update your AzuraCast installation without losing any modifications.

Example Plugin

We have built an example plugin that you should use as the base for your own plugin. You can find the git repository at:

<https://github.com/AzuraCast/example-plugin>

Including Plugins

Plugins are automatically discovered if they're located in the `/plugins` directory relative to the main AzuraCast installation. Plugins are ignored by the parent AzuraCast instance, so you can update your instance any time you like without worrying about your plugins being removed.

You can clone the plugin directory anywhere you want on the host machine, then create a `docker-compose.override.yml` in your `/var/azuracast` directory to mount the plugin as a volume in the correct location, like so:

```
services:
  web:
    environment:
      AZURACAST_PLUGIN_MODE: true
    volumes:
```



On this page >

Additional Information

Details about how to build plugins, what events are available, etc. are included in the documentation of the [Example Plugin](#).

 Edit page

← Previous
Liquidsoap

Install AzuraCast with Docker

AzuraCast is powered by Docker and uses pre-built images that contain every component of the software. Don't worry if you aren't very familiar with Docker; our easy installer tools will handle installing Docker and Docker Compose for you, and updates are very simple.

System Requirements

Please check the page with the [AzuraCast system requirements](#) before installing to make sure your server meets the minimum requirements.

Installing

Connect to the server or computer you want to install AzuraCast on via an SSH terminal. You should be an administrator user with either root access or the ability to use the `sudo` command. If you are not the root user, you may need to run `sudo su -` before executing the commands below.

Pick a base directory on your host computer that AzuraCast can use. If you're on Linux, you can follow the steps below to use the recommended directory:

```
mkdir -p /var/azuracast  
cd /var/azuracast
```

Use these commands to download our Docker Utility Script, set it as executable and then run the Docker installation process:



On this page >

On-screen prompts will show you how the installation is progressing.

You're almost ready...

Once installation has completed, be sure to open the URL to your AzuraCast installation in your browser to create your super administrator account and finish the initial setup.

Troubleshooting

Some Virtual Private Server (VPS) providers sometimes ship a old version of Docker or Docker-Compose with their systems and the version isn't fully compatible with our installation script. This can be resolved by running these commands then attempting the installation process again:

```
cd /var/azuracast
./docker.sh install-docker
./docker.sh install-docker-compose
./docker.sh install
```

Unattended Installation

To automate the installation process for AzuraCast, you can run the following commands or place them into a `cloud-init` script:

```
mkdir -p /var/azuracast
cd /var/azuracast
curl -fsSL https://raw.githubusercontent.com/AzuraCast/AzuraCast/main/docker.sh > docker.sh
chmod a+x docker.sh
```



On this page >

 Edit page

Users

Required Permissions

To manage users, users must be in a role that has the “Administer Users” permission globally.

From the “Users” administration panel, you can modify any existing users or create new ones. You can modify the roles that any given user are a part of, and force a password change in the event the user has forgotten their password.

Resetting an Account Password

If you have lost the password to log into an account, but still have access to the SSH terminal for the server, you can execute the following command to generate a new random password for an account in the system.

Replace `YOUREMAILADDRESS` with the e-mail address whose password you intend to reset.

```
cd /var/azuracast
./docker.sh cli azuracast:account:reset-password YOUREMAILADDRESS
```

 Edit page



On this page >

Reporting a Bug



Tip

You must follow [troubleshooting steps](#) prior to filing an issue on GitHub.

In our issue template, we ask for [logs](#) and other details that are part of the troubleshooting steps, and supplying these greatly helps us resolve issues.

Support for AzuraCast is only provided for the current [Stable](#) and up to date [Rolling-Release](#) versions. Make sure to update your installation before reporting an issue. We regularly publish updates that resolve issues and add new features.

If your issue isn't in our known issues or common solutions, check out our [GitHub Issues](#) board. We use GitHub to track the status of errors, bug reports and general questions.

If there's already an open GitHub issue for the specific issue you're having, you can follow that issue for updates. If not, you can create a new issue from the same page.

When creating a new issue, you'll be presented with a template of questions to answer to help our team provide quick and accurate support.

Please make sure to answer these questions and provide as much detail as possible!

Please include the following:

- Any steps that you have already attempted
- Any errors you may have encountered
- A full step-by-step guide on how to reproduce the issue
- Screenshots if applicable
- Relevant [logs \(click here for instructions on how to get them\)](#).



On this page >

← Previous
Optimizing

Next →
Troubleshooting

Troubleshooting

Our core maintainers, along with our vast community of station operators, can answer many questions you may have about our software. We also have knowledge base articles set up for common issues encountered by our users.

First Steps

Your first steps should be to check these docs for any tips or pointers. Many common issues can be solved by following the instructions in our guide pages.

If you aren't sure of what is causing your issue, [checking the logs](#) can help. Not only do the system logs often explain your error in more detail, but they're essential for our developers to be able to easily diagnose and resolve your issue.

Flushing the System Cache

Many parts of the AzuraCast system depend on caches to speed up site performance. Sometimes, these caches can get out of date, and they may cause errors. You can always flush all site-wide caches using one command-line script:

For Docker Installations:



```
./docker.sh cli cache:clear
```

For Ansible Installations:

[On this page >](#)

Common Docker Errors

“Invalid interpolation format” related errors

This issue is related to an out of date docker-compose version, since we use a more up to date docker-compose version it'll return these types of errors.

- `ERROR: Invalid interpolation format for "installer" option in service "services":
"ghcr.io/azuracast/web:${AZURACAST_VERSION:-latest}"`
- `ERROR: Invalid interpolation format for "nginx_proxy" option in service "services":
"ghcr.io/azuracast/nginx_proxy:${AZURACAST_VERSION:-latest}"`

The solution is simple, run these commands. You may need to run it with `sudo` permissions:

```
./docker.sh install-docker-compose  
./docker.sh install
```

“bind: address already in use”

If you get this error when starting the Docker AzuraCast instance, it means that something else (some other process or server software) is already running on the same port(s) that AzuraCast is trying to reserve for itself.

There are generally two ways to resolve this:

- Find the process that's currently listening on the port (commands like `netstat -tulpn` can help you with this) and either disable the program that's currently listening, edit its configuration to change the port it listens on, or uninstall it from the server completely.
- Switch the port AzuraCast uses for its own traffic to an unused one. [See instructions](#)



On this page >

Common services that listen on ports 80 and 443 include web servers like Apache and nginx. One common port to use if you still need these services to remain online alongside AzuraCast is port 8080, which we deliberately leave open due to its common usage with other software.

“WARNING: The LETSENCRYPT_X variable is not set. Defaulting to a blank string.”

This message doesn't indicate anything is wrong with your installation; it is simply Docker Compose letting you know that it defaults to an empty string if a certain environment variable isn't present. You will see these messages if you don't have LetsEncrypt configured on your server, and you can safely disregard them.

Other General Errors

InnoDB: Upgrade after a crash is not supported

The DB crashed while using MariaDB 10.4 and was then not restarted with the same MariaDB version so that it can recover from the redo log files but was updated to 10.5 which now is not able to handle the old redo log format (in 10.5 they changed that format).

Renaming the `ib_logfile0` (and if there are more `ib_logfile*` files those too) and then starting the db again should work.

You can find those files here: `/var/lib/docker/volumes/azuracast_db_data/_data`

Stop your AzuraCast via `docker-compose down`, then try renaming them via `mv ib_logfile0 ib_logfile0.old` and after that run the update again.

Failed Database Migrations

In some situations, especially if the automated update process is interrupted for any reason, you can wind up in a state where a database migration has only halfway completed. During subsequent



On this page >

What Causes This

We use MariaDB as our database layer, and while MariaDB is a very powerful and capable database solution, one of its limitations is that it cannot do transactional table alterations. In other words, you can't wrap "ALTER TABLE" statements in a transaction that can then be rolled back in the event of errors. Some other databases (like PostgreSQL) support transactional table alterations, but switching databases would be difficult, if not impossible, for our installed user base.

Easy Solution: Restore from a Backup

This situation is one of the primary reasons why we strongly encourage users to keep regular backups of their installations. Restoring from a backup is a very simple way to resolve the issue, as it will bring you back to a known working state, where you can then run the update process again (and it will likely continue without issues).

Advanced Solution: Modify the Database

Caution: This is only advised for very advanced users who are comfortable modifying the underlying database powering AzuraCast.

We don't provide specific instructions for this process as it can cause irreparable damage to your installation, but basically, the process would be:

- Identify which migration is failing.
- Find the corresponding migration (in the `src/Entity/Migrations` folder).
- Find which step is failing in the `up` function, and find the corresponding SQL commands to undo those specific migrations in the `down` command in the same file.
- Manually run those commands via `./docker.sh cli dbal:run-sql "SQL COMMAND HERE"`
- Re-run the update process.



On this page >

 Edit page

Previous
← Reporting a Bug

Media Management →
Next

Logs

Before submitting any GitHub issues, you should take a look at the terminal logs that AzuraCast outputs. They can often provide additional information about the error, or include very useful information that should be included in any GitHub issue you create.

Users with the appropriate permissions can also view many logs directly through AzuraCast itself. The Log Viewer feature is available under “Logs” in each station’s management page.

System Logs

If you are administrating your own AzuraCast installation you can check most logs through the `Administration` -> `System Logs` .

In addition to the `Icecast` , `SHOUTcast` and `Liquidsoap` logs you can find the `AzuraCast Application Log` here which contains useful information about what the AzuraCast application itself is doing. Email message logs are also sent here.

Log Viewer

The `Logs` link in your station sidebar menu gives you access to the Logs from your station’s `Icecast` , `SHOUTcast` and `Liquidsoap` processes.

In addition to the logs, you can also view the Icecast, SHOUTcast and Liquidsoap configuration scripts that AzuraCast automatically generates for you.

Log Types



On this page >

file.

Liquidsoap Log

This log file contains the log output of the Liquidsoap instance for your station. Liquidsoap is responsible for the audio transcoding in AzuraCast and interfaces with the AzuraCast application to provide the AutoDJ for your station.

In this log you can find information on how the streams are generated, what information the AutoDJ sent to Liquidsoap, errors, warnings and more.

If you encounter any problems with your stream generation this log should contain useful information in diagnosing the issue.

Icecast Access Log

This log file contains the raw access log data of the Icecast server.

For most issues this log does not contain any useful information but can be used for analytical purposes.

Icecast Error Log

This log file contains the error output of the Icecast server.

If you encounter any problems with accessing the streams or your mount points this is where you should start looking into.

Docker Container Logs

Some system logs can only be accessed from a shell session on the host computer. To view the logs for your AzuraCast installation, run:

[On this page](#) >

For Ansible Installations

Since the Ansible installation interacts directly with your host server, its logs are in various locations across the system.

- AzuraCast: `/var/azuracast/www_tmp/app.log`
- Nginx Access: `/var/azuracast/www_tmp/access.log`
- Nginx Errors: `/var/azuracast/www_tmp/error.log`
- PHP: `/var/azuracast/www_tmp/php_errors.log`
- Supervisord: `/var/azuracast/www_tmp/supervisord.log`
- Redis: `/var/log/redis/redis-server.log`
- MariaDB: `/var/log/mysql`

For each station, logs for radio software will be inside

`/var/azuracast/stations/{station_short_name}/config` , with the following filenames:

- Liquidsoap: `liquidsoap.log`
- Icecast: `icecast.log`
- SHOUTcast: `sc_serv.log`

[✎ Edit page](#)

← Previous
Update AzuraCast

Next
Optimizing →

Optimizing

If you're running AzuraCast on limited system resources, you may find that you need to tweak or remove some optional features to improve your overall performance. This page includes some of the simplest changes you can take to reduce your overall resource (CPU, RAM, etc.) usage.

Inside AzuraCast

Disable Replaygain or Pre-Calculate Replaygain for Audio Files

If Replaygain is enabled in your AutoDJ, then Liquidsoap will examine each file to see if it already has Replaygain metadata associated with it; if not, it will try to automatically calculate it on-the-fly, which can be a very CPU-intensive process that can significantly slow your installation.

In order to resolve this, you can take one of two approaches:

1. Disable Replaygain from the "AutoDJ" tab of your station's profile; or
2. Ensure all of your uploaded media is pre-tagged with Replaygain metadata using a tool like [loudgain](#)

Disable Audio Post-Processing

If your installation is limited in available CPU, you should disable any "Audio Post-Processing" settings on any stations hosted on your server.

The total amount of extra resources per station consumed by post-processing is roughly as so:

- Stereo Tool (Most CPU)
- `master_me` (Med-High CPU)
- Basic Compression/Normalization (Low-Med CPU)



On this page >

AutoCue is a new feature built by the community and offered as part of AzuraCast. It analyzes your music and automatically calculates cue points, fade points, and volume levels for a consistent listening experience.

To compute these values, it has to analyze every track that plays on your station. This can take time and extra CPU resources, especially when a station first starts up (because it has to analyze every starting track in every playlist, just in case it might have to transition to that track immediately).

If you have shorter, simpler tracks or have a few large playlists, this will only result in a slight CPU bump, after which the station will stabilize. If you have longer tracks or a large number of individual playlists, the initial CPU load can be so high that it disrupts your station operations entirely.

If you are in the latter group, it is recommended to either disable AutoCue from the station profile or to precompute the AutoCue values for your files before starting your station. See the [AutoCue project documentation](#) for the current recommendations on how to achieve this.

Disable Unnecessary Mount Points

If you have configured multiple mount points for your station, consider reducing the total number of mount points to a smaller subset if possible. Every mount point that the AutoDJ has to broadcast to uses a small, but steady, amount of CPU at all times.

For example, if you offer a mobile MP3 and AAC/Ogg stream, consider limiting the mobile stream strictly to the (higher-performance per bit) AAC/Ogg stream, if your clients support it.

Keep Less Playback History

In "System Settings", you will notice a setting called "Days of Playback History to Keep". This setting not only controls how far back you can view song history, but also how far back you can view detailed listener statistics.

Setting it to a large value or "Indefinitely" will quickly fill up your available hard drive space. If you change this setting, within an hour or so AzuraCast will automatically "trim" any extra song history



On this page >

If your VPS or cloud hosting environment is limited in disk space, you can always configure storage locations that use external storage instead.

The most popular method of remote storage in AzuraCast is S3-compatible bucket storage. There are very many providers of S3-compatible storage out there, from the namesake Amazon's Simple Storage Service (S3) to Cloudflare's R2, Backblaze B1, or Wasabi. Pricing for storage and bandwidth can vary greatly between providers, so consider which option is best for your setup.

If configuring a remote storage location for station media, be sure to select one that has little or no fees for so-called "egress" bandwidth to your server, as there is a lot of back-and-forth traffic between AzuraCast and media storage locations. This is less of a concern if you are using S3 buckets for backups or recordings.

Docker

Disable Userland Proxy

By default, Docker ships with a "Userland Proxy" feature, which allows you to connect to your Docker containers from inside the same Linux OS that's running Docker itself. For most VPS or cloud hosting scenarios, this is unnecessary, and the userland proxy can often take up *huge* amounts of resources because of the large number of ports forwarded by AzuraCast.

Unfortunately, we can't automatically disable this functionality, but you can configure Docker on your host OS to disable it.

On your host OS, use an editor to open `/etc/docker/daemon.json`. Add or edit the following line into the JSON:

```
{  
  "userland-proxy": false,  
}
```

Save your changes, then restart the host OS.



On this page >

Previous
← Logs

Next
Reporting a Bug →

Roles & Permissions

Required Permissions

To manage roles and permissions, users must be in a role that has the “Administer Permissions” permission globally.

AzuraCast has a highly granular Access Control List (ACL) system that allows you to control the specific permissions of each group of users.

In summary, you can think of the access control list as working this way:

- One or more **Users** can be in a **Role**.
- Each **Role** has one or more **Permissions**, which gives everyone in that Role that level of access. Permissions can either be specific to a single station, or global for the entire installation.

The Roles & Permissions page has protections in place to prevent users from accidentally denying themselves permissions to the Permissions page itself.

Special Permissions

There are special permissions that will grant access to other sections of the system, even when you don't explicitly grant those lower-level permissions to each user:

- The **All Permissions** global permission creates a super-user with access to all of the AzuraCast installation, and the **All Permissions** station-specific permission creates a super-user with full access to that specific station only.
- The **Manage Stations** global permission also allows the user to administer any stations that exist in the AzuraCast installation.



On this page >

Previous

← Multi-Site Docker
Installation

Next

SSL & Let's Encrypt →

Docker

Docker installations come with four files by default:

- `docker.sh`, the Docker Utility Script;
- `.env`, which contains environment variables used by Docker Compose itself;
- `azuracast.env`, which contains customizable environment variables sent to AzuraCast and related services; and
- `docker-compose.yml`, a large file that defines all of the services used by AzuraCast and how they interact.

Docker Utility Script

If you're using the Docker installation method to run AzuraCast, we have created a helpful utility script to perform common functions without having to type long command names.

Download the Utility Script

If you've recently followed the [Docker installation instructions](#), you already have the Docker Utility Script installed. The file is named `docker.sh`.

If you have an older installation, you can use the Docker Utility Script by running these commands inside your AzuraCast directory on your host computer:

```
curl -L https://raw.githubusercontent.com/AzuraCast/AzuraCast/main/docker.sh > docker.sh
chmod a+x docker.sh
```




On this page >

```
./docker.sh ci1 [command_name]
```

Runs any command exposed by the command line interface tools.

Available Commands

Install AzuraCast

```
./docker.sh install
```

Pulls the latest version of all Docker images and sets up the AzuraCast database. When complete, your AzuraCast instance should be up and running.

Update AzuraCast

```
./docker.sh update-self  
./docker.sh update
```

Note: You should always run update-self first to update the updater script.

Automatically pulls down any updated Docker images and applies any database and configuration changes since your last AzuraCast update.

Uninstall AzuraCast

```
./docker.sh uninstall
```



On this page >

This command will fully remove any station media, statistics and metrics, and the entire database associated with your AzuraCast instance.

Back Up Files and Settings

```
./docker.sh backup [/path/to/backup.tar.gz]
```

Creates a .tar.gz backup copy of the media, statistics and metrics of every station, along with a copy of the full AzuraCast database. You can later restore from this same file in the event of data loss or corruption.

Restore Files and Settings

```
./docker.sh restore /path/to/backup.tar.gz
```

Extracts a .tar.gz file previously created by this same script's backup command, copying media, statistics and metrics for each station into AzuraCast and importing the version of the database contained in the backup.

⚠ This command replaces your current installation!

Restoring from a backup will remove any existing AzuraCast database or media that exists inside the Docker volumes.

Customizing Docker Installations



On this page >

- When updating (using the `docker.sh` utility script), it is recommended to run `./docker.sh update-self` before running `./docker.sh update`, to ensure the Docker Utility Script itself is up to date before it updates your Docker installation.
- Environment variables set in `.env` are only used by Docker Compose itself, and aren't passed directly into the AzuraCast containers. You should only modify this file to change the HTTP and HTTPS port mappings used by Nginx (see the "Use Non-Standard Ports" section above).
- The `azuracast.env` file is specific to your environment and can be customized however you like. It will not be replaced during any updates. Once your database has been created, however, changing the password listed in this file will cause the system to fail. If you want to destructively wipe your existing database and other files and set up a new one with the updated password, add the `-v` flag to the end of `docker-compose down` to remove all existing volumes, including your database.
- If possible, you should not directly modify `docker-compose.yml`, as some updates may modify how it is defined to resolve bugs or add new features. When updating, you will always be asked if you want to update this file; if you have not modified it, you should always do so.
- Instead of modifying `docker-compose.yml`, you can create a file named `docker-compose.override.yml` with your customizations. The structure of this file is the same as the main `docker-compose.yml` file, and is automatically parsed by Docker Compose to override any definitions in the main file. Updates will not replace this file.

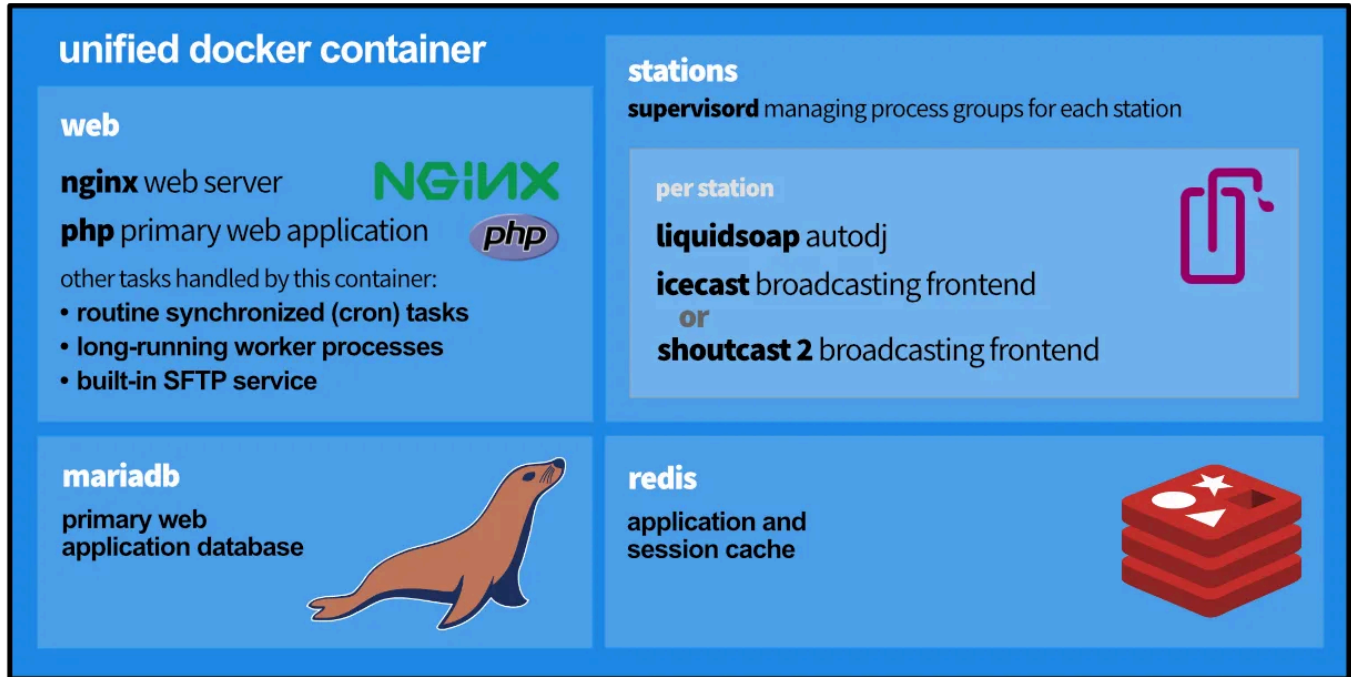
Modifying Docker

The AzuraCast Docker installation is built to serve the needs of the vast majority of installations out of the box, but there may be times when you need to customize how Docker serves your installation, while still retaining the benefits of compatibility and easy updating that Docker offers.

The Overall Infrastructure



On this page >



Docker Compose

In order to make our Docker installation simple to maintain, we rely on a secondary piece of software called [Docker Compose](#). Rather than using lengthy command-line commands, Docker Compose allows you to write simple YAML files to configure the way our Docker containers launch, which ports they forward to the host, mounted disk volumes, etc.

If you follow our installation instructions (or use one of our prebuilt images), your Docker installation is located on your host computer at `/var/azuracast`. Within this folder, you'll see four files by default:

- `docker-compose.yml`, the primary Docker Compose configuration file. AzuraCast's updater automatically prompts you to keep this file updated, which is strongly recommended;
- `docker.sh`, the Docker Utility Script that provides easy aliases for common Docker tasks, such as updating, executing commands inside containers, and more;



On this page >

thus used for higher-level configuration changes like port mappings.

Overriding Docker Compose

Rather than modifying `docker-compose.yml` directly, it is strongly recommended to instead create a new file called `docker-compose.override.yml` in the same folder. Docker Compose will automatically look for and include this file if it exists, and apply its configuration on top of the base file's instructions.

There are some considerations when creating your own `docker-compose.override.yml` :

- The file must be valid YAML using the standard [Docker Compose format](#).
- When modifying ports or other lists, you can only add new items, not remove existing ones.

Once you've modified your Docker Compose configuration, you should apply these changes by running:

```
docker-compose down
docker-compose up -d
```

Note that this will temporarily shut down your AzuraCast installation and will briefly disconnect your listeners.

Using Non-standard Ports



Tip

In newer versions, you no longer need to manually do this; you can instead use the installer by answering "Yes" when prompted to customize settings.



On this page >

```
cd /var/azuracast
./docker.sh update-self
./docker.sh change-ports
```

To override more complex functionality in your Docker installation, see the “Customizing Docker” section below.

Expanding the Station Port Range



Tip

Installations running version 0.16.0 or higher can now use our installation script to customize ports via the terminal which will automatically write the changes into the `docker-compose.yml` file for you.

For performance reasons, by default Docker installations only open radio ports from port 8000 to 8500. This allows for 50 unique stations to operate.

Depending on your hardware, it may be possible to run more than 50 stations on one AzuraCast instance, but if you want to directly access the additional radio ports, you can follow this simple process.

In the same folder where your Docker installation is (if using recommended instructions, this is `/var/azuracast`), create a new file named `docker-compose.override.yml`.

In this file, paste the following contents:

```
services:
  web:
    ports:
```



On this page >

When creating a new station, AzuraCast will attempt to automatically assign it an available port from the available port range. If you change this port range, you should let AzuraCast know by adding or updating the following values in your `azuracast.env` file:

```
AUTO_ASSIGN_PORT_MIN=8500
AUTO_ASSIGN_PORT_MAX=8999
```

You will need to restart your Docker containers using `docker-compose down`, then `docker-compose up -d` to apply any changes made to these files.

Mounting a Directory Into a Station

You may want to add music to a station from a directory on your host machine without copying the data into AzuraCast. You can mount the directory into your stations and web container to make them available to AzuraCast by creating a `docker-compose.override.yml`.

In the same folder where your Docker installation is (if using recommended instructions, this is `/var/azuracast`), create a new file named `docker-compose.override.yml`.

In this file, paste the following contents:

```
services:
  web:
    volumes:
      - /path/on/host/computer:/var/azuracast/stations/<STATION_NAME>/media/
```

Replace the `<STATION_NAME>` with the name of the station directory found under the "Administration" section of the station's profile settings and modify the `/path/on/host/computer` with the path to the directory that you want to mount.

You will need to restart your Docker containers using `docker-compose down`, then `docker-compose up -d` to apply any changes made to this file.



On this page >

In the same folder where your Docker installation is (if using recommended instructions, this is `/var/azuracast`), create a new file named `docker-compose.override.yml`.

In this file, paste the following contents:

```
services:
  web:
    volumes:
      - /path/on/host/computer:/var/azuracast/stations
```

Modify the `/path/on/host/computer` with the path to the directory that you want to mount.

You will need to restart your Docker containers using `docker-compose down`, then `docker-compose up -d` to apply any changes made to this file.

Custom Default Track



Tip

In newer versions of AzuraCast, you no longer need to upload a custom default track this way; you can upload a unique "Custom Fallback Track" per-station using the web interface.

When nothing is playing on your station you'll hear the default `error.mp3` file of AzuraCast playing. You can replace this file by mounting your own `.mp3` file via a `docker-compose.override.yml`.

In the same folder where your Docker installation is (if using recommended instructions, this is `/var/azuracast`), create a new file named `docker-compose.override.yml`.

You can place your `.mp3` file anywhere on your host machine. You just have to specify the path to it by replacing this part: `/path/to/your/file.mp3`



On this page >

In this file, paste the following contents:

```
services:
  web:
    volumes:
      - /path/to/your/file.mp3:/usr/local/share/icecast/web/error.mp3
```

You will need to restart your Docker containers using `docker-compose down`, then `docker-compose up -d` to apply any changes made to this file.

Stream Intro File



Tip

In newer versions of AzuraCast, you can now directly upload an intro track on the Mount Point edit page.

You can add a music file to play when someone initially connects to your stream. Remember when creating intro files that they **must match the exact same format, bitrate and sample rate as your mount point to work properly.**

First, tell the Docker filesystem where to find your intro file. Inside the AzuraCast directory on your host (by default, `/var/azuracast`), create a file named `docker-compose.override.yml` with the following contents:

```
services:
  stations:
    volumes:
      - /path/to/your/file.mp3:/usr/local/share/icecast/web/intro.mp3
```

Now restart AzuraCast via `docker-compose down && docker-compose up -d`.



On this page >

```
<intro>/intro.mp3</intro>
```

For more information, see the [Icecast documentation](#).

Custom Favicon



Tip

In newer versions of AzuraCast, you can manage this via the web UI.

To override the favicon and other browser icons used by AzuraCast, you should follow these steps:

1. Upload your icon to favicon-generator.org. Be sure to leave "Generate icons for Web, Android, Microsoft, and iOS (iPhone and iPad) Apps" checked.
2. Once your icon set is downloaded, it will be in a .zip file format; extract that file and copy it to the host server where AzuraCast is hosted.
3. Inside the AzuraCast directory on your host (by default, /var/azuracast), create a file named docker-compose.override.yml with the following contents:

```
services:
  web:
    volumes:
      - /path/to/your/favicon/folder:/var/azuracast/www/static/icons/production
```

4. Restart AzuraCast via `docker-compose down && docker-compose up -d`.

Increase PHP Allowed Memory

When you encounter an error like the following one you will need to increase the amount of memory that PHP is allowed to use:



On this page >

There you can find the following entry:

```
# PHP's maximum memory limit.  
# PHP_MEMORY_LIMIT=128M
```

Change this to look like this:

```
# PHP's maximum memory limit.  
PHP_MEMORY_LIMIT=512M
```

You should increase the value until the error dissappears.

The `M` after the value stands for `Megabytes`.

Refer to [PHP's documentation](#) for more information about the allowed values for this setting.

After you have changed this setting you need to restart AzuraCast via `docker-compose down && docker-compose up -d`.

Adding custom NGINX configurations

If you need to add some custom configurations to NGINX you can do this by your `custom.conf` file into the `web` container via the `docker-compose.override.yml` like this:

```
services:  
  web:  
    volumes:  
      - ./custom.conf:/etc/nginx/azuracast.conf.d/custom.conf
```

All config files placed in the `/etc/nginx/azuracast.conf.d/` are automatically included in the main AzuraCast NGINX configuration.



On this page >

 Edit page

Previous
← Using Custom JavaScript

Next
Multi-Site Docker →
Installation

About AzuraCast

AzuraCast is currently in beta!

Many web radio stations already run AzuraCast, but keeping your server up-to-date with the latest code from the GitHub repository is strongly recommended for security, bug fixes and new feature releases. It's unlikely, but updates may result in unexpected issues or data loss, so always make sure to keep your station's media files backed up in a second location.

To install AzuraCast, you should have a basic understanding of the Linux shell terminal. Once installed, every aspect of your radio station can be managed via AzuraCast's web interface.

LATEST STABLE VERSION

v0.21.0

AzuraCast is a self-hosted, all-in-one web radio management suite. Using its easy installer and powerful but intuitive web interface, you can start up a fully working web radio station in a few quick minutes.

AzuraCast works for web radio stations of all types and sizes, and is built to run on even the most affordable VPS web hosts.

Core Features

For Radio Stations

- **Rich Media Management:** Upload songs, edit metadata, preview songs and organize music into folders from your browser.
- **Playlists:** Add music to standard-rotation playlists (in sequential or shuffled playback order) or schedule a playlist to play at a scheduled time, or once per x songs/minutes/etc.



On this page > Overview

AzuraCast's built-in web DJ tool.

- **Public Pages:** AzuraCast includes embeddable public pages that you can integrate into your existing web page or use as the basis for your own customized player.
- **Listener Requests:** Let your listeners request specific songs from your playlists, both via an API and a simple public-facing listener page.
- **Remote Relays:** Broadcast your radio signal (including live DJs) to any remote server running Icecast or SHOUTcast.
- **Web Hooks:** Integrate your station with Slack, Discord, TuneIn, Twitter and more by setting up web hooks that connect to third-party services.
- **Detailed Analytics and Reports:** Keep track of every aspect of your station's listeners over time. View reports of each song's impact on your listener count. You can also generate a report that's compatible with SoundExchange for US web radio royalties.

For Server Administrators

- **Role-based User Management:** Assign global and per-station permissions to a role, then add users to those roles to control access.
- **Custom Branding:** Modify every aspect of both the internal and public-facing AzuraCast pages by supplying your own custom CSS and JavaScript.
- **Authenticated RESTful API:** Individual users in the system can create API keys which have the same permissions they have in the system. The AzuraCast API is a powerful and [well-documented](#) tool for interacting with installations.
- **Web Log Viewing:** Quickly diagnose problems affecting any part of the AzuraCast system through the system-wide web log viewer.
- **Automatic Radio Proxies:** Many users can't connect directly to radio station ports (i.e. 8000) by default, so AzuraCast includes an automatic nginx proxy that lets listeners connect via the http (80) and https (443) ports. These proxies are also compatible with services like CloudFlare.
- **Storage Location Management:** Station media, live recordings and backups can be stored locally or on an S3 compatible storage provider.



On this page



Overview

Radio Software

- [Liquidsoap](#) as the always-playing “AutoDJ”
- [Icecast-KH](#) as a radio broadcasting frontend

For x86/x64 installations, [SHOUTcast 2 DNAS](#) can also be used as a broadcasting frontend.

SHOUTcast is non-free software and does not come bundled with AzuraCast, but can be installed via the administration panel after AzuraCast has been installed.

Supporting Software

- [NGINX](#) for serving web pages and the radio proxy
- [MariaDB](#) as the primary database
- [PHP](#) powering the web application
- [Redis](#) for sessions, message queue storage, database and general caching
- [Centrifugo](#) for high-performance “Now Playing” data feeds

AzuraCast API

Once installed and running, AzuraCast exposes a REST API that allows you to monitor and interact with your stations. Documentation about this API and its endpoints are available on the [AzuraCast API Documentation](#).

All of AzuraCast’s frontend actions are performed through the API, so through API calls, you can manage your station(s) entirely via the API.

License

AzuraCast is licensed under the [Affero GNU General Public License \(GPL\), version 3.0](#). This project is free and open-source software, and pull requests are always welcome



On this page



Overview

- [DigitalOcean](#) For generously providing the server resources we use for our demonstration instance, our staging and testing environments, and more
- [JetBrains](#) For making our development faster, easier and more productive with tools like PhpStorm
- [CrowdIn](#) For giving us a simple and powerful tool to help translate our application for users around the world
- [Netlify](#) For supporting open-source software like ours and for serving as the host of our web site!
- [BrowserStack](#) For providing us a tool that allows us to test across multiple browsers in one place
- [Depot](#) for powering our Docker image builds.

...and the creators and maintainers of the many free and open-source tools that AzuraCast is built on, who have done so much to help move FOSS forward!

A small, light blue pencil icon.[Edit page](#)

← Previous
Home

Next
Live Demo →

Streamers & DJs

Required Permissions

To manage streamers and DJs, users must be in a role that has the “Manage Station Streamers” permission for the given station.

If your station has “Enable Streamers” turned on, you can create multiple DJ accounts that can connect and broadcast live audio to your listeners.

In order to allow a seamless transition between live DJs and the AutoDJ, each streamer connects directly to the AutoDJ software (Liquidsoap). Liquidsoap accepts connections from broadcast software that are compatible with either Icecast or SHOUTcast servers.

AzuraCast will keep track of the broadcast history for each DJ that broadcasts live on your station.

We have guides for how to configure many kinds of popular streaming software. See the [Streaming Software](#) page for more information.

Recording Broadcasts

You can opt to record live broadcasts by editing the station’s profile. Broadcasts will automatically be recorded in the format and bitrate that you specify, and can be downloaded from the “Broadcasts” popup menu next to each streamer.



On this page >

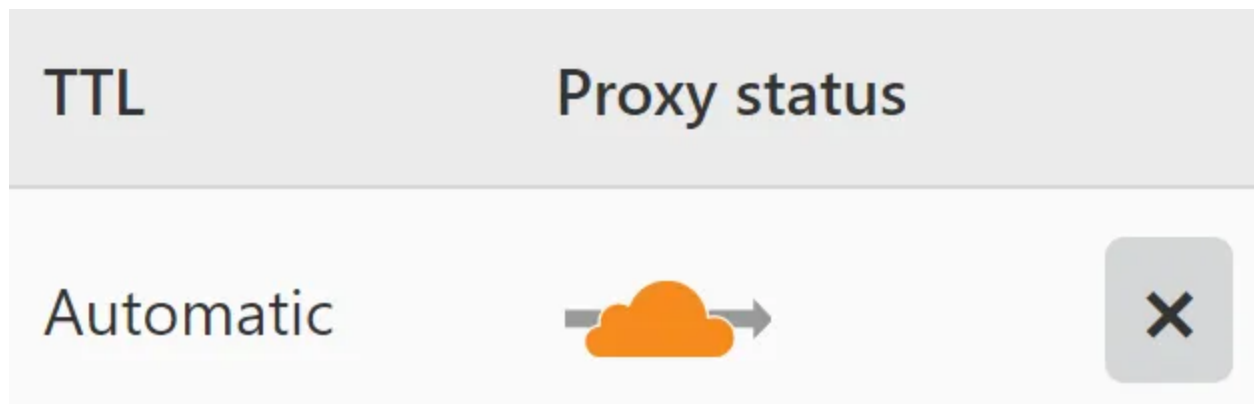
Cloudflare

[Cloudflare](#) is a leading provider of reverse proxying and CDN services for the web. Their free tier offers huge benefits in performance and protection that you can take advantage of while using AzuraCast, with a few important caveats. This document details how to use Cloudflare with AzuraCast.

Enabling Cloudflare with AzuraCast

AzuraCast has full support for using Cloudflare's protection in front of your radio server.

Enabling Cloudflare support from the Cloudflare control panel is as simple as ensuring the little "cloud" next to your radio server's domain (or subdomain) is orange, indicating protection is enabled.



The default settings for Cloudflare will suffice, though you can also switch from "Flexible" to "Full" SSL mode if you like (since AzuraCast itself supports HTTPS). "Strict" SSL mode isn't recommended, as this requires that you maintain an up-to-date SSL certificate on AzuraCast itself (which is possible, but not necessary).



On this page >

Enabling the Web Proxy for Radio Broadcasts

One major limitation imposed by Cloudflare is that they do not forward incoming connections to your server that don't come from the traditional web ports (that is, 80 and 443). By default, AzuraCast serves each radio station on its own distinct port in a range from 8000 to 9000. This means your listeners wouldn't normally be able to connect.

Use Web Proxy for Radio

☒ Use Web Proxy for Radio

By default, radio stations broadcast on their own ports (i.e. 8000). If you're using a service like CloudFlare or accessing your radio station by SSL, you should enable this feature, which routes all radio through the web ports (80 and 443).

Fortunately, we've already built a solution to this problem! In AzuraCast's system administration, on the "System Settings" page, we have a checkbox labeled "Use Web Proxy for Radio". Enable this checkbox and all of the station playback URLs across the system will be updated to automatically use the web port proxy links, which are fully accessible even when Cloudflare protection is enabled.

Important Notes

Do Not Cache Radio Content

Cloudflare does not allow you to serve your web radio content via their cached CDN, as the content is a streaming audio file that isn't cacheable via their traditional services. The easiest way to instruct Cloudflare not to cache your radio broadcasts is via their Page Rules feature.

To create a new Page Rule:

1. Visit the Cloudflare control panel, then "Rules", then "Page Rules".
2. Click "Create Page Rule".



On this page >

5. Click **Save and Deploy Page Rule** at the bottom of the page.

Repeat these steps for all of the following patterns:

- `/radio/*`
- `/listen/*`
- `/hls/*`

Always Disable Rocket Loader

AzuraCast is **not** compatible with the Rocket Loader optimization offered by Cloudflare. If Rocket Loader is enabled, you will encounter unexpected errors with AzuraCast.

In order to resolve this error, you must go to your Cloudflare Dashboard and disable the Rocket Launcher settings.

About Incoming DJ Connections

Because Cloudflare blocks any incoming connections that aren't on the standard web ports, it also blocks the incoming connections that your streamers/DJs would use to broadcast to your station. Unlike the experience we offer listeners through our radio proxy, we can't proxy the incoming broadcast in the same way for technical reasons.

We recommend instructing your streamers/DJs to connect to your server using its IP address rather than its Cloudflare-protected domain name. This will allow them to connect without any issue, and without exposing your origin IP to your regular listener base.

About AzuraRelay Instances

If you use AzuraRelay instances that should relay a Cloudflare-protected installation, you should use the IP address of the installation as the base URL for the relay (in a format like <http://127.0.0.1>), rather than the public-facing Cloudflare-protected address. Otherwise, no changes are needed.



On this page >

Previous
← Backup & Restore

Next
Command-Line Interface →

Suggest a New Feature

Our community votes on what features we should prioritize via a special web site called FeatureUpvote. Visit the AzuraCast FeatureUpvote Site to submit new feature ideas or vote on existing ones.

[Suggest Features on FeatureUpvote](#) [Edit page](#)[← Previous
Donate](#)[Next
Translation →](#)

Now Playing Data APIs

The most important and frequently accessed pieces of information that AzuraCast stores are all served as part of a single group of data, which we refer to as the “Now Playing” data.

This data includes:

- Basic information about the station, its mount points and remote relays;
- Current listener counts, including total and unique listeners;
- The currently playing song, including a timestamp of when it was started, and whether it came from a playlist, request, or live source;
- Whether a live DJ is currently connected, and if so, what their name is; and
- A truncated history of the most recent ~5 songs (this number is customizable) played on the station.

Here’s a [real-world example of the Now Playing API’s return data](#) from the AzuraCast demo page.

AzuraCast always serves this data in the exact same format, regardless of whether you’re broadcasting locally or remotely, or whether you’re using Icecast or SHOUTcast.

Because of how valuable this information is, we serve it in a number of ways depending on whether performance or flexibility is your main concern.

Standard Now Playing API

This is the “Now Playing” API endpoint listed as part of the [API documentation](#). On any installation, an array of now-playing data for all public stations is available at:

```
http://your-azuracast-site.example.com/api/nowplaying
```


[On this page >](#)

...replacing `station_shortcode` with either the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio") or the numeric ID of the station (visible in the URL when managing the station in AzuraCast).

Advantages

- This method is available on all installations and will always be available.
- If you have the "Prefer Browser URL" setting turned on in your installation, any URLs included in the API response (i.e. URLs for album art or listening to the station) will be updated to reflect the browser URL that was used to make the API request.
- The "elapsed" and "remaining" values on `current_song` are automatically updated to reflect their current values (in seconds) at the moment you make the API request.

Disadvantages

- Every request sent to this endpoint calls the entire Nginx/PHP-FPM stack, resulting in a very brief (under 100ms) but possibly significant performance penalty. If you have several hundred users checking this API every few seconds, the combined load can quickly overwhelm a server.

Example Implementation

Using the [jQuery](#) JavaScript library, an example implementation might look like:

```
var nowPlayingTimeout;
var nowPlaying;

function loadNowPlaying() {
    $.ajax({
        cache: false,
        dataType: "json",
        url: 'http://your-azuracast-site.example.com/api/nowplaying/station_shortcode',
        success: function(np) {
```



On this page >

```

    }
  }).fail(function() {
    nowPlayingTimeout = setTimeout(loadNowPlaying, 30000);
  });
}

$(function() {
  loadNowPlaying();
});

```

Using the [Axios](#) HTTP client library, an example implementation might look like:

```

var nowPlaying;
var nowPlayingTimeout;

function loadNowPlaying() {
  axios.get('http://your-azuracast-site.example.com/api/nowplaying/station_shortcode').then(
    // Do something with the Now Playing data.
    nowPlaying = response.data;
  ).catch((error) => {
    console.error(error);
  }).then(() => {
    clearTimeout(nowPlayingTimeout);
    nowPlayingTimeout = setTimeout(checkNowPlaying, 15000);
  });
}

loadNowPlaying();

```

If your application is written in PHP, you can use the Composer package manager to install our [PHP API Client](#), which has full support for the Now Playing API endpoints.

Static Now Playing JSON File



On this page >

```
http://your-azuracast-site.example.com/api/nowplaying_static/station_shortcode.json
```

...replacing `station_shortcode` with the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio").

Advantages

- Because the file is static and is only changed every few seconds, Nginx, your browser, and any other reverse proxies (i.e. CloudFlare) can cache the output for a few seconds, resulting in significantly higher performance.
- Since this file's format is the same as the standard API, no code modifications or third-party libraries are needed to switch to this format.

Disadvantages

- Any URLs in the API responses will always use the "Base URL" of your AzuraCast installation.
- The "elapsed" and "remaining" durations will only be accurate as of when the file was written, not when it was downloaded by your client. You should instead compare the user's current UNIX timestamp against the `played_at` timestamp.

Example Implementation

Implementations of this method look exactly the same as for the Standard Now Playing API (above), except with the URL updated to the static URL for the station.

Simple Text File

AzuraCast also generates a simple text file containing `Artist - Title` for each station. This can be useful if you need to fetch the current playing track for display or in automation tools.

[On this page >](#)

...replacing `station_shortcode` with the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio").

High-Performance Updates

We deliver a high-performance (low-latency and low server CPU burden) Now Playing feed thanks to a realtime messaging library called [Centrifugo](#). Using this connection method, each listener gets immediate track updates while only maintaining a single lightweight HTTP connection.

Now Playing updates are delivered as unidirectional messages via Websocket, Server-Sent Events (SSE) or Long Polling web streams.

Websockets

The URL for websocket connections is:

```
wss://your-azuracast-url/api/live/nowplaying/websocket
```

Upon connection, you should send a connection string in the form of a JSON request:

```
{ "subs": { "station:your_station_name": {} } }
```

Replacing `your_station_name` with your station's URL stub or short name.

You will start to receive a feed of empty pings (`{}`) and Now Playing updates. You can identify Now Playing updates as they will have, in their parsed JSON, `pub.data.np` .

An example JavaScript implementation is below:

```
let socket = new WebSocket("wss://your-azuracast-url/api/live/nowplaying/websocket");

socket.onopen = function(e) {
```



On this page >

```
    });  
};  
  
let nowplaying = {};  
let currentTime = 0;  
  
// Handle a now-playing event from a station. Update your now-playing data accordingly.  
function handleSseData(ssePayload, useTime = true) {  
    const jsonData = ssePayload.data;  
  
    if (useTime && 'current_time' in jsonData) {  
        currentTime = jsonData.current_time;  
    }  
  
    nowplaying = jsonData.np;  
}  
  
socket.onmessage = function(e) {  
    const jsonData = JSON.parse(e.data);  
  
    if ('connect' in jsonData) {  
        const connectData = jsonData.connect;  
  
        if ('data' in connectData) {  
            // Legacy SSE data  
            connectData.data.forEach(  
                (initialRow) => handleSseData(initialRow)  
            );  
        } else {  
            // New Centrifugo time format  
            if ('time' in connectData) {  
                currentTime = Math.floor(connectData.time / 1000);  
            }  
  
            // New Centrifugo cached NowPlaying initial push.  
            for (const subName in connectData.subs) {
```



On this page >

```
    }  
  }  
  } else if ('pub' in jsonData) {  
    handleSseData(jsonData.pub);  
  }  
};
```

Server-Sent Events (SSE/EventSource)

The URL for SSE connections is:

```
https://your-azuracast-url/api/live/nowplaying/sse?cf_connect=TOKEN
```

Where the `cf_connect` URL parameter is the same connection token as the first message in the Websocket example.

An example JavaScript implementation is below:

```
const sseBaseUri = "https://your-azuracast-url/api/live/nowplaying/sse";  
const sseUriParams = new URLSearchParams({  
  "cf_connect": JSON.stringify({  
    "subs": {  
      "station:azuratest_radio": {"recover": true}  
    }  
  })  
});  
const sseUri = sseBaseUri+"?" + sseUriParams.toString();  
const sse = new EventSource(sseUri);  
  
let nowplaying = {};  
let currentTime = 0;  
  
// This is a now-playing event from a station. Update your now-playing data accordingly.
```



On this page >

```
    currentTime = jsonData.current_time;
  }

  nowplaying = jsonData.np;
}

sse.onmessage = (e) => {
  const jsonData = JSON.parse(e.data);

  if ('connect' in jsonData) {
    const connectData = jsonData.connect;

    if ('data' in connectData) {
      // Legacy SSE data
      connectData.data.forEach(
        (initialRow) => handleSseData(initialRow)
      );
    } else {
      // New Centrifugo time format
      if ('time' in connectData) {
        currentTime = Math.floor(connectData.time / 1000);
      }

      // New Centrifugo cached NowPlaying initial push.
      for (const subName in connectData.subs) {
        const sub = connectData.subs[subName];
        if ('publications' in sub && sub.publications.length > 0) {
          sub.publications.forEach((initialRow) => handleSseData(initialRow, false));
        }
      }
    }
  } else if ('pub' in jsonData) {
    handleSseData(jsonData.pub);
  }
};
```



On this page >

On this page

← Previous
AzuraCast APIs

Next →
Liquidsoap

Now Playing Data APIs

The most important and frequently accessed pieces of information that AzuraCast stores are all served as part of a single group of data, which we refer to as the “Now Playing” data.

This data includes:

- Basic information about the station, its mount points and remote relays;
- Current listener counts, including total and unique listeners;
- The currently playing song, including a timestamp of when it was started, and whether it came from a playlist, request, or live source;
- Whether a live DJ is currently connected, and if so, what their name is; and
- A truncated history of the most recent ~5 songs (this number is customizable) played on the station.

Here’s a [real-world example of the Now Playing API’s return data](#) from the AzuraCast demo page.

AzuraCast always serves this data in the exact same format, regardless of whether you’re broadcasting locally or remotely, or whether you’re using Icecast or SHOUTcast.

Because of how valuable this information is, we serve it in a number of ways depending on whether performance or flexibility is your main concern.

Standard Now Playing API

This is the “Now Playing” API endpoint listed as part of the [API documentation](#). On any installation, an array of now-playing data for all public stations is available at:

```
http://your-azuracast-site.example.com/api/nowplaying
```

[On this page >](#)

...replacing `station_shortcode` with either the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio") or the numeric ID of the station (visible in the URL when managing the station in AzuraCast).

Advantages

- This method is available on all installations and will always be available.
- If you have the "Prefer Browser URL" setting turned on in your installation, any URLs included in the API response (i.e. URLs for album art or listening to the station) will be updated to reflect the browser URL that was used to make the API request.
- The "elapsed" and "remaining" values on `current_song` are automatically updated to reflect their current values (in seconds) at the moment you make the API request.

Disadvantages

- Every request sent to this endpoint calls the entire Nginx/PHP-FPM stack, resulting in a very brief (under 100ms) but possibly significant performance penalty. If you have several hundred users checking this API every few seconds, the combined load can quickly overwhelm a server.

Example Implementation

Using the [jQuery](#) JavaScript library, an example implementation might look like:

```
var nowPlayingTimeout;
var nowPlaying;

function loadNowPlaying() {
    $.ajax({
        cache: false,
        dataType: "json",
        url: 'http://your-azuracast-site.example.com/api/nowplaying/station_shortcode',
        success: function(np) {
```



On this page >

```

    }
  }).fail(function() {
    nowPlayingTimeout = setTimeout(loadNowPlaying, 30000);
  });
}

$(function() {
  loadNowPlaying();
});

```

Using the [Axios](#) HTTP client library, an example implementation might look like:

```

var nowPlaying;
var nowPlayingTimeout;

function loadNowPlaying() {
  axios.get('http://your-azuracast-site.example.com/api/nowplaying/station_shortcode').then(
    // Do something with the Now Playing data.
    nowPlaying = response.data;
  ).catch((error) => {
    console.error(error);
  }).then(() => {
    clearTimeout(nowPlayingTimeout);
    nowPlayingTimeout = setTimeout(checkNowPlaying, 15000);
  });
}

loadNowPlaying();

```

If your application is written in PHP, you can use the Composer package manager to install our [PHP API Client](#), which has full support for the Now Playing API endpoints.

Static Now Playing JSON File



On this page >

```
http://your-azuracast-site.example.com/api/nowplaying_static/station_shortcode.json
```

...replacing `station_shortcode` with the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio").

Advantages

- Because the file is static and is only changed every few seconds, Nginx, your browser, and any other reverse proxies (i.e. CloudFlare) can cache the output for a few seconds, resulting in significantly higher performance.
- Since this file's format is the same as the standard API, no code modifications or third-party libraries are needed to switch to this format.

Disadvantages

- Any URLs in the API responses will always use the "Base URL" of your AzuraCast installation.
- The "elapsed" and "remaining" durations will only be accurate as of when the file was written, not when it was downloaded by your client. You should instead compare the user's current UNIX timestamp against the `played_at` timestamp.

Example Implementation

Implementations of this method look exactly the same as for the Standard Now Playing API (above), except with the URL updated to the static URL for the station.

Simple Text File

AzuraCast also generates a simple text file containing `Artist - Title` for each station. This can be useful if you need to fetch the current playing track for display or in automation tools.

[On this page >](#)

...replacing `station_shortcode` with the station's abbreviated name (i.e. `azuratest_radio` for "AzuraTest Radio").

High-Performance Updates

We deliver a high-performance (low-latency and low server CPU burden) Now Playing feed thanks to a realtime messaging library called [Centrifugo](#). Using this connection method, each listener gets immediate track updates while only maintaining a single lightweight HTTP connection.

Now Playing updates are delivered as unidirectional messages via Websocket, Server-Sent Events (SSE) or Long Polling web streams.

Websockets

The URL for websocket connections is:

```
wss://your-azuracast-url/api/live/nowplaying/websocket
```

Upon connection, you should send a connection string in the form of a JSON request:

```
{ "subs": { "station:your_station_name": {} } }
```

Replacing `your_station_name` with your station's URL stub or short name.

You will start to receive a feed of empty pings (`{}`) and Now Playing updates. You can identify Now Playing updates as they will have, in their parsed JSON, `pub.data.np` .

An example JavaScript implementation is below:

```
let socket = new WebSocket("wss://your-azuracast-url/api/live/nowplaying/websocket");

socket.onopen = function(e) {
```



On this page >

```
    });  
};  
  
let nowplaying = {};  
let currentTime = 0;  
  
// Handle a now-playing event from a station. Update your now-playing data accordingly.  
function handleSseData(ssePayload, useTime = true) {  
    const jsonData = ssePayload.data;  
  
    if (useTime && 'current_time' in jsonData) {  
        currentTime = jsonData.current_time;  
    }  
  
    nowplaying = jsonData.np;  
}  
  
socket.onmessage = function(e) {  
    const jsonData = JSON.parse(e.data);  
  
    if ('connect' in jsonData) {  
        const connectData = jsonData.connect;  
  
        if ('data' in connectData) {  
            // Legacy SSE data  
            connectData.data.forEach(  
                (initialRow) => handleSseData(initialRow)  
            );  
        } else {  
            // New Centrifugo time format  
            if ('time' in connectData) {  
                currentTime = Math.floor(connectData.time / 1000);  
            }  
  
            // New Centrifugo cached NowPlaying initial push.  
            for (const subName in connectData.subs) {
```



On this page >

```
    }  
  }  
  } else if ('pub' in jsonData) {  
    handleSseData(jsonData.pub);  
  }  
};
```

Server-Sent Events (SSE/EventSource)

The URL for SSE connections is:

```
https://your-azuracast-url/api/live/nowplaying/sse?cf_connect=TOKEN
```

Where the `cf_connect` URL parameter is the same connection token as the first message in the WebSocket example.

An example JavaScript implementation is below:

```
const sseBaseUri = "https://your-azuracast-url/api/live/nowplaying/sse";  
const sseUriParams = new URLSearchParams({  
  "cf_connect": JSON.stringify({  
    "subs": {  
      "station:azuratest_radio": {"recover": true}  
    }  
  })  
});  
const sseUri = sseBaseUri+"?" + sseUriParams.toString();  
const sse = new EventSource(sseUri);  
  
let nowplaying = {};  
let currentTime = 0;  
  
// This is a now-playing event from a station. Update your now-playing data accordingly.
```



On this page >

```
    currentTime = jsonData.current_time;
  }

  nowplaying = jsonData.np;
}

sse.onmessage = (e) => {
  const jsonData = JSON.parse(e.data);

  if ('connect' in jsonData) {
    const connectData = jsonData.connect;

    if ('data' in connectData) {
      // Legacy SSE data
      connectData.data.forEach(
        (initialRow) => handleSseData(initialRow)
      );
    } else {
      // New Centrifugo time format
      if ('time' in connectData) {
        currentTime = Math.floor(connectData.time / 1000);
      }

      // New Centrifugo cached NowPlaying initial push.
      for (const subName in connectData.subs) {
        const sub = connectData.subs[subName];
        if ('publications' in sub && sub.publications.length > 0) {
          sub.publications.forEach((initialRow) => handleSseData(initialRow, false));
        }
      }
    }
  } else if ('pub' in jsonData) {
    handleSseData(jsonData.pub);
  }
};
```




On this page >

On this page

← Previous
AzuraCast APIs

Next →
Liquidsoap

Troubleshooting

Our core maintainers, along with our vast community of station operators, can answer many questions you may have about our software. We also have knowledge base articles set up for common issues encountered by our users.

First Steps

Your first steps should be to check these docs for any tips or pointers. Many common issues can be solved by following the instructions in our guide pages.

If you aren't sure of what is causing your issue, [checking the logs](#) can help. Not only do the system logs often explain your error in more detail, but they're essential for our developers to be able to easily diagnose and resolve your issue.

Flushing the System Cache

Many parts of the AzuraCast system depend on caches to speed up site performance. Sometimes, these caches can get out of date, and they may cause errors. You can always flush all site-wide caches using one command-line script:

For Docker Installations:



```
./docker.sh cli cache:clear
```

For Ansible Installations:

[On this page >](#)

Common Docker Errors

“Invalid interpolation format” related errors

This issue is related to an out of date docker-compose version, since we use a more up to date docker-compose version it'll return these types of errors.

- `ERROR: Invalid interpolation format for "installer" option in service "services": "ghcr.io/azuracast/web:${AZURACAST_VERSION:-latest}"`
- `ERROR: Invalid interpolation format for "nginx_proxy" option in service "services": "ghcr.io/azuracast/nginx_proxy:${AZURACAST_VERSION:-latest}"`

The solution is simple, run these commands. You may need to run it with `sudo` permissions:

```
./docker.sh install-docker-compose
./docker.sh install
```

“bind: address already in use”

If you get this error when starting the Docker AzuraCast instance, it means that something else (some other process or server software) is already running on the same port(s) that AzuraCast is trying to reserve for itself.

There are generally two ways to resolve this:

- Find the process that's currently listening on the port (commands like `netstat -tulpn` can help you with this) and either disable the program that's currently listening, edit its configuration to change the port it listens on, or uninstall it from the server completely.
- Switch the port AzuraCast uses for its own traffic to an unused one. [See instructions](#)



On this page >

Common services that listen on ports 80 and 443 include web servers like Apache and nginx. One common port to use if you still need these services to remain online alongside AzuraCast is port 8080, which we deliberately leave open due to its common usage with other software.

“WARNING: The LETSENCRYPT_X variable is not set. Defaulting to a blank string.”

This message doesn't indicate anything is wrong with your installation; it is simply Docker Compose letting you know that it defaults to an empty string if a certain environment variable isn't present. You will see these messages if you don't have LetsEncrypt configured on your server, and you can safely disregard them.

Other General Errors

InnoDB: Upgrade after a crash is not supported

The DB crashed while using MariaDB 10.4 and was then not restarted with the same MariaDB version so that it can recover from the redo log files but was updated to 10.5 which now is not able to handle the old redo log format (in 10.5 they changed that format).

Renaming the `ib_logfile0` (and if there are more `ib_logfile*` files those too) and then starting the db again should work.

You can find those files here: `/var/lib/docker/volumes/azuracast_db_data/_data`

Stop your AzuraCast via `docker-compose down`, then try renaming them via `mv ib_logfile0 ib_logfile0.old` and after that run the update again.

Failed Database Migrations

In some situations, especially if the automated update process is interrupted for any reason, you can wind up in a state where a database migration has only halfway completed. During subsequent



On this page >

What Causes This

We use MariaDB as our database layer, and while MariaDB is a very powerful and capable database solution, one of its limitations is that it cannot do transactional table alterations. In other words, you can't wrap "ALTER TABLE" statements in a transaction that can then be rolled back in the event of errors. Some other databases (like PostgreSQL) support transactional table alterations, but switching databases would be difficult, if not impossible, for our installed user base.

Easy Solution: Restore from a Backup

This situation is one of the primary reasons why we strongly encourage users to keep regular backups of their installations. Restoring from a backup is a very simple way to resolve the issue, as it will bring you back to a known working state, where you can then run the update process again (and it will likely continue without issues).

Advanced Solution: Modify the Database

Caution: This is only advised for very advanced users who are comfortable modifying the underlying database powering AzuraCast.

We don't provide specific instructions for this process as it can cause irreparable damage to your installation, but basically, the process would be:

- Identify which migration is failing.
- Find the corresponding migration (in the `src/Entity/Migrations` folder).
- Find which step is failing in the `up` function, and find the corresponding SQL commands to undo those specific migrations in the `down` command in the same file.
- Manually run those commands via `./docker.sh cli dbal:run-sql "SQL COMMAND HERE"`
- Re-run the update process.



On this page >

 Edit page

Previous
← Reporting a Bug

Media Management →
Next

Multi-Site Docker Installation

If you use the recommended Docker installation method for AzuraCast, you can configure your installation to sit behind a nginx proxy that can also handle other web sites on the same host. This allows you to host, for example, your station's web site on the same server as your AzuraCast installation.

Follow the steps below to configure your installation for multisite support.

Nginx Proxy Manager

Most users prefer to use a proxy manager that includes a GUI for easier management of the proxy. There are a lot of different tools for this, one of which is the [Nginx Proxy Manager](#), which uses NGINX and includes an easy to use GUI and automatic Let's Encrypt SSL which should be sufficient for most users.

Enter AzuraCast Base Directory

Connect to your server via SSH and change to the base directory where your AzuraCast installation is located. If you followed the recommended installation instructions (or used a prebuilt image) you can run:

```
cd /var/azuracast
```

Turn Off Existing Services



On this page >

[docker-compose down](#)

This will disconnect your listeners and stop your station from broadcasting. You will not lose any data or statistics during the upgrade.

Edit AzuraCast Web Serving Ports

The first step is to prevent AzuraCast's own `web` container from trying to listen on ports 80 and 443, as a proxy will now be doing that job instead.

There is a file in the AzuraCast installation directory named `.env` which has two values set by default:

```
AZURACAST_HTTP_PORT=80
AZURACAST_HTTPS_PORT=443
```

Note the filename!

This file is actually named `.env`, not `azuracast.env`, which is a separate file. Most Linux hosts will hide the `.env` file by default, but you can still see it by running `ls -la` or edit it with a command like `vim .env`.

Edit the `.env` file in your editor of choice to change those values to an unused public-facing port. A good option would be the examples below:

```
AZURACAST_HTTP_PORT=10080
AZURACAST_HTTPS_PORT=10443
```

Save your changes and return to the shell.

Start Docker Services



On this page >

[docker compose up](#)

Setup Nginx Proxy Manager

Create a new directory for the Nginx Proxy Manager and enter it via the following commands:

```
mkdir /var/proxy_manager
cd /var/proxy_manager
```

Then create a file with the name `docker-compose.yml` in this directory (for example with `nano`) like this:

```
nano docker-compose.yml
```

Now follow the [Quick Setup](#) for the Nginx Proxy Manager starting with step [2](#) .

Create Proxy Host for AzuraCast

When you have successfully setup the Nginx Proxy Manager and have access to the GUI you can start creating a proxy for AzuraCast via the `Proxy Hosts` page.

Click on the `Create Proxy Host` button, enter a domain that points to the server in the `Domain Names` field.

In the "Scheme" field, select `https` . In "Forward Hostname/IP", enter the public-facing IP of your server (not the domain name). In "Forward Port", enter the same port you specified above for AzuraCast (in our example, 10443).

Enable the `Websockets Support` option.

Switch to the `SSL` tab and select `Request a new SSL Certificate` in the `SSL Certificate` dropdown.



On this page >

 Edit page

← Previous
Docker

Next →
Roles & Permissions

Multi-Site Docker Installation

If you use the recommended Docker installation method for AzuraCast, you can configure your installation to sit behind a nginx proxy that can also handle other web sites on the same host. This allows you to host, for example, your station's web site on the same server as your AzuraCast installation.

Follow the steps below to configure your installation for multisite support.

Nginx Proxy Manager

Most users prefer to use a proxy manager that includes a GUI for easier management of the proxy. There are a lot of different tools for this, one of which is the [Nginx Proxy Manager](#), which uses NGINX and includes an easy to use GUI and automatic Let's Encrypt SSL which should be sufficient for most users.

Enter AzuraCast Base Directory

Connect to your server via SSH and change to the base directory where your AzuraCast installation is located. If you followed the recommended installation instructions (or used a prebuilt image) you can run:

```
cd /var/azuracast
```

Turn Off Existing Services



On this page >

[docker-compose down](#)

This will disconnect your listeners and stop your station from broadcasting. You will not lose any data or statistics during the upgrade.

Edit AzuraCast Web Serving Ports

The first step is to prevent AzuraCast's own `web` container from trying to listen on ports 80 and 443, as a proxy will now be doing that job instead.

There is a file in the AzuraCast installation directory named `.env` which has two values set by default:

```
AZURACAST_HTTP_PORT=80
AZURACAST_HTTPS_PORT=443
```

Note the filename!

This file is actually named `.env`, not `azuracast.env`, which is a separate file. Most Linux hosts will hide the `.env` file by default, but you can still see it by running `ls -la` or edit it with a command like `vim .env`.

Edit the `.env` file in your editor of choice to change those values to an unused public-facing port. A good option would be the examples below:

```
AZURACAST_HTTP_PORT=10080
AZURACAST_HTTPS_PORT=10443
```

Save your changes and return to the shell.

Start Docker Services



On this page >

[docker-compose up](#)

Setup Nginx Proxy Manager

Create a new directory for the Nginx Proxy Manager and enter it via the following commands:

```
mkdir /var/proxy_manager
cd /var/proxy_manager
```

Then create a file with the name `docker-compose.yml` in this directory (for example with `nano`) like this:

```
nano docker-compose.yml
```

Now follow the [Quick Setup](#) for the Nginx Proxy Manager starting with step [2](#).

Create Proxy Host for AzuraCast

When you have successfully setup the Nginx Proxy Manager and have access to the GUI you can start creating a proxy for AzuraCast via the `Proxy Hosts` page.

Click on the `Create Proxy Host` button, enter a domain that points to the server in the `Domain Names` field.

In the "Scheme" field, select `https`. In "Forward Hostname/IP", enter the public-facing IP of your server (not the domain name). In "Forward Port", enter the same port you specified above for AzuraCast (in our example, 10443).

Enable the `Websockets Support` option.

Switch to the `SSL` tab and select `Request a new SSL Certificate` in the `SSL Certificate` dropdown.



On this page >

 Edit page

← Previous
Docker

Next →
Roles & Permissions

Liquidsoap

[Liquidsoap](#) is a powerful and flexible language for describing audio and video streams. It offers a rich collection of operators that you can combine at will for creating or transforming streams.

Liquidsoap in AzuraCast

AzuraCast uses Liquidsoap as the the AutoDJ to generate and manipulate audio streams. It transforms the input streams to the correct format for each mount point, handles live DJ connections, filtering, compression, and more.

Liquidsoap Annotation Reference

This information is intended for plugin developers who are looking to supply additional annotation metadata to Liquidsoap. It is a compilation of information that wasn't available elsewhere online, and is intended only for advanced users.

Appending Tracks

liquidsoap/src/operators/append.ml

Append an extra track to every track.

liq_append

Set the metadata 'liq_append' to 'false' to inhibit appending on one track.

Crossing Tracks



On this page >

the combination.

A buffer is needed to store the end of a track before combining it with the next track. We could always have a full buffer, but this would involve copying all the time. Instead, we try to fill the buffer only when getting close to the end of track. The problem then is to cope with tracks which are longer than expected, i.e. which end doesn't really fit in the buffer.

This operator works with any type of stream. All three parameters are durations in ticks.

liq_start_next

Metadata field which, if present and containing a float, overrides the 'duration' parameter for current track.

Cue Points

liquidsoap/src/operators/cuepoint.ml

The [cue_cut] class is able to skip over the beginning and end of a track according to cue points. This involves quite a bit of trickery involving clocks, #seek as well as reverting frame contents. Even more trickery would be needed to implement a [cue_split] operator that splits tracks according to cue points: in particular, the frame manipulation would get nasty, involving storing chunks that have been fetched too early, replaying them later, glued with new content.

We use ticks for precision, but store them as Int64 to allow long durations. This should eventually be generalized to all of liquidsoap, removing limitations such as the duration passed to #seek or returned by #remaining. We introduce a few notations to make this comfortable.

Start track after a cue in point and stop it at cue out point. The cue points are given as metadata, in seconds from the beginning of tracks.

liq_cue_in

Metadata for cue in points.



On this page >

Fader

liquidsoap/src/operators/fade.ml

Fade durations (in, initial, out, final) are indicated in total seconds.

liq_fade_type

Metadata field which, if present and correct, overrides the 'type' parameter for current track.

Options: lin|sin|log|exp (linear, sinusoidal, logarithmic or exponential)

Default: lin

liq_fade_in

Fade the beginning of **tracks**.

liq_fade_initial

Fade the beginning of **a stream**.

liq_fade_out

Fade the end of **tracks**.

liq_fade_final

Fade **a stream** to silence.

Offset

liquidsoap/src/operators/on_offset.ml



On this page >

Metadata field which, if present and containing a float, overrides the 'offset' parameter.

Prepending Tracks

liquidsoap/src/operators/prepend.ml

Prepend an extra track before every track.

liq_prepend

Set the metadata 'liq_prepend' to 'false' to inhibit appending on one track.

Set Volume

liquidsoap/src/operators/setvol.ml

Multiply the amplitude of the signal.

liq_amplify

Specify the name of a metadata field that, when present and well-formed, overrides the amplification factor for the current track. Well-formed values are floats in decimal notation (e.g. '0.7') which are taken as normal/linear multiplicative factors; values can be passed in decibels with the suffix 'dB' (e.g. '-8.2 dB', but the spaces do not matter).

Video Fade

liquidsoap/src/operators/video_fade.ml

liq_video_fade_in

Fade the beginning of tracks. Metadata 'liq_video_fade_in' can be used to set the duration for a specific track (float in seconds).

[On this page >](#)

LADSPA Plugins

[LADSPA](#) is a standard that allows software audio processors and effects to be plugged into a wide range of audio synthesis and recording packages.

Our latest rolling-release Docker image and Ansible scripts add support for LADSPA to Liquidsoap. We include several popular plugins, allowing you to perform powerful equalization in your configuration.

Use the following command to view all the available LADSPA plugins that come pre-installed with AzuraCast:

```
docker-compose exec --user=azuracast web liquidsoap --list-plugins
```

To get detailed information about the usage of a specific LADSPA plugin use the following command and replace the `plugin` part of the `ladspa.plugin` with the name of the plugin you want to look at:

```
docker-compose exec --user=azuracast web liquidsoap -h ladspa.plugin
```

We have modified the [MK Pascal script](#) to work with AzuraCast.

You can check out the [full script here](#) that you can include in your Liquidsoap configuration via the [Utilities](#) -> [Edit Liquidsoap Configuration](#) page of your station after enabling the [Advanced Features](#).

Scrobbling to Last.fm

[On this page >](#)

```
settings.audioscrobbler.api_key := "API_KEY"
settings.audioscrobbler.api_secret := "API_SECRET"

radio = audioscrobbler.submit(
  username="USERNAME",
  password="PASSWORD",
  force=true,
  metadata_preprocessor=(fun(m) -> if m["jingle_mode"] == "true" then [] else m end),
  radio
)
```

You must change `API_KEY`, `API_SECRET`, `USERNAME`, and `PASSWORD` to reflect the credentials of the account you wish to scrobble to.

While a track is being played on your station it is marked as “Scrobbling now” on the account page of the Last.fm account specified. When the track finishes, it becomes scrobbled. Jingles are not scrobbled. Tracks with no `Artist` or no `Title` are not scrobbled.

[✎ Edit page](#)

← Previous
Now Playing Data APIs

Next →
Plugins

Installing AzuraCast on a Synology NAS

Configure Environment

1. Install Docker

Install Docker from the Synology package center.

2. Enable SSH

To enable SSH on Synology, open control panel and go to `Terminal & SNMP`. Check `Enable SSH service`. The default port is 22. You may change the port to something else if desired.

After enabling SSH, connect to the NAS from a remote machine with a supported SSH terminal.

3. OPTIONAL: Enable Home Folder for User

As a convenience factor, you can set up home folders for users on the Synology NAS

To enable user home folder on Synology DSM, open control panel and go to `User` -> `Advanced`. Scroll down and enable user home service.

(Note: If you enable both SSH and home folder service, you can drop your SSH key into your home folder at `~/.ssh/authorized_keys`. Just make sure that folder and its children are `chmod` ded to 755, i.e. `chmod 755 /var/services/homes/my-nas-user`)

4. OPTIONAL: Add User to Docker Group

Note: This will effectively give the user account "root" level access (via Docker volumes), so only do this if you are comfortable with that arrangement.



On this page >

If you decide not to do this, just add `sudo` to the beginning of any commands that appear after this.

Install AzuraCast

1. Create AzuraCast Folder

By default, Docker prefers that its relevant files be located in `/volume1/docker`. This doesn't necessarily mean that your media files have to be located on this directory, but the core AzuraCast installation should live within that folder. In this example, we will install AzuraCast into

`/volume1/docker/azuracast`.

```
mkdir -p /volume1/docker/azuracast
cd /volume1/docker/azuracast

curl -fsSL https://raw.githubusercontent.com/AzuraCast/AzuraCast/main/docker.sh > docker.sh
chmod a+x docker.sh
```

Before running the initial installation, you'll want to configure the ports that AzuraCast uses, as the Synology NAS by default will occupy ports 80 and 443. If you want AzuraCast to occupy those ports, you'll have to modify the configuration of the existing software to accomplish this.

For this example, we'll run AzuraCast on port 2080 and 2443.

Run the script below to configure the relevant environment variables:

```
sudo ./docker.sh setup-ports
```

Enter the custom ports when prompted. You can also specify a custom port for SFTP connections, but the default (2022) should be available.

[On this page >](#)

AzuraCast should then be immediately available on the specified ports.

2. OPTIONAL: Specify Media directory for Station

Given the setup of a Synology NAS, it's likely that you won't necessarily want to host your station's media inside the internal volume managed by Docker but rather elsewhere on your system.

For this example, I have created a station named "AzuraTest Radio" inside AzuraCast and created a separate shared folder, on Volume 2, labeled "azuracast_media".

In the Docker directory, `/volume1/docker/azuracast`, create a new file named `docker-compose.override.yml` with contents like those below:

```
services:
  web:
    volumes:
      - /volume2/azuracast_media:/var/azuracast/stations/azuratest_radio/media

  stations:
    volumes:
      - /volume2/azuracast_media:/var/azuracast/stations/azuratest_radio/media
```

By default, the user inside the Docker container runs as UID/GID 1000, the default UID for the first created user in an Ubuntu installation. It is very likely, however, that your Synology user account is not running with this UID/GID.

You should also update the UID/GID of the account running inside Docker to match yours.

First, find your UID by running:

```
id -u
id -g
```



On this page >

```
AZURACAST_PGID=your_user_gid
```

With both the `docker-compose.override.yml` file and the UID/GID set, you can now restart your Docker containers by running:

```
sudo docker-compose down
sudo docker-compose up -d
```

 [Edit page](#)

Donate

AzuraCast is a fully free and open-source project built as a labor of love over the course of many years. AzuraCast has always been, and will remain, both “free as in freedom” (libre) and “free as in beer” (gratis) for the thousands of stations that depend on it for their day-to-day operations.

Although the AzuraCast community has contributed in many ways to the project, much of the day-to-day work of maintaining the application, updating its dependencies and fixing bugs falls upon its lead developer, [Buster Neece](#). The AzuraCast project is akin to a full-time job for him, and your support goes to directly helping him pay for essentials like rent and groceries.

Not only are donations to AzuraCast going to good use, but they help us devote the time we need to keep the application new and updated. Almost every day for years now, we’ve updated the application to fix a bug, add a new feature or improve the user experience. We never stop working to serve our audience, and your donations help make that possible.

Ways to Donate

GitHub Sponsors

If you’re a GitHub user, supporting through GitHub Sponsors is one of the best ways to show your support for AzuraCast’s development. You can manage your pledge at any time through the Sponsors dashboard.

If you donate through GitHub Sponsors, you will be listed as a sponsor on your GitHub profile and with any issues or discussions you create.

Sponsor AzuraCast on GitHub Sponsors



[On this page >](#)

it's a convenient way to support this project along with other creative professionals and projects.

[Become a Patron of AzuraCast](#)

OpenCollective

AzuraCast has an OpenCollective account that is fiscally hosted by the Open Source Collective. Contributions made to, and expenses withdrawn from, this account are fully open and transparent.

[Support AzuraCast on OpenCollective](#)[✎ Edit page](#)[← Previous
Code of Conduct](#)[Next →
Suggest a New Feature](#)

Donate

AzuraCast is a fully free and open-source project built as a labor of love over the course of many years. AzuraCast has always been, and will remain, both “free as in freedom” (libre) and “free as in beer” (gratis) for the thousands of stations that depend on it for their day-to-day operations.

Although the AzuraCast community has contributed in many ways to the project, much of the day-to-day work of maintaining the application, updating its dependencies and fixing bugs falls upon its lead developer, [Buster Neece](#). The AzuraCast project is akin to a full-time job for him, and your support goes to directly helping him pay for essentials like rent and groceries.

Not only are donations to AzuraCast going to good use, but they help us devote the time we need to keep the application new and updated. Almost every day for years now, we’ve updated the application to fix a bug, add a new feature or improve the user experience. We never stop working to serve our audience, and your donations help make that possible.

Ways to Donate

GitHub Sponsors

If you’re a GitHub user, supporting through GitHub Sponsors is one of the best ways to show your support for AzuraCast’s development. You can manage your pledge at any time through the Sponsors dashboard.

If you donate through GitHub Sponsors, you will be listed as a sponsor on your GitHub profile and with any issues or discussions you create.

Sponsor AzuraCast on GitHub Sponsors



[On this page >](#)

it's a convenient way to support this project along with other creative professionals and projects.

[Become a Patron of AzuraCast](#)

OpenCollective

AzuraCast has an OpenCollective account that is fiscally hosted by the Open Source Collective. Contributions made to, and expenses withdrawn from, this account are fully open and transparent.

[Support AzuraCast on OpenCollective](#)[✎ Edit page](#)[← Previous
Code of Conduct](#)[Next →
Suggest a New Feature](#)

Mount Points

Required Permissions

To manage mount points, users must be in a role that has the “Manage Station Mount Points” permission for the given station.

Mount Points are how listeners connect and listen to your broadcasts.

Each Station in AzuraCast represents one single broadcast (with one currently playing title, artist, etc.), but with multiple Mount Points you can output that broadcast in multiple formats (i.e. MP3, Ogg Vorbis) or bitrates (i.e. 128kbps or 64kbps).

Note that not all formats are compatible with all platforms. For instance, iOS users won’t be able to listen to the Ogg format as it is not supported by Safari on iOS at the moment. [Checking formats compatibility](#) can help you decide which one will be better for your station.

Furthermore, it should be noted that OGG Opus and FLAC (OGG FLAC) do NOT send “Current Song”, “Artist”, or “Title” (and subsequently, “yp_currently_playing”) information to Icecast.

Advanced

Mount Point URL

You can set a custom URL for streams that AzuraCast will use when referring to it in both the web UI and in [API responses](#) (specifically, `station.listen_url` and `mounts.*.url`). Leaving the field empty uses the default value, generated in the format:

```
{HTTP/HTTPS}://{BASE_URL}.TLD/listen/{STATION_URL_STUB}/{MOUNT_POINT_URL}.
```



On this page >

 Edit page

← Previous
Media Management

Next
Playlists →

Code of Conduct

Our Pledge

We as members, contributors, and leaders pledge to make participation in our community a harassment-free experience for everyone, regardless of age, body size, visible or invisible disability, ethnicity, sex characteristics, gender identity and expression, level of experience, education, socio-economic status, nationality, personal appearance, race, religion, or sexual identity and orientation.

We pledge to act and interact in ways that contribute to an open, welcoming, diverse, inclusive, and healthy community.

Our Standards

Examples of behavior that contributes to a positive environment for our community include:

- Demonstrating empathy and kindness toward other people
- Being respectful of differing opinions, viewpoints, and experiences
- Giving and gracefully accepting constructive feedback
- Accepting responsibility and apologizing to those affected by our mistakes, and learning from the experience
- Focusing on what is best not just for us as individuals, but for the overall community

Examples of unacceptable behavior include:

- The use of sexualized language or imagery, and sexual attention or advances of any kind
- Trolling, insulting or derogatory comments, and personal or political attacks
- Public or private harassment



On this page >

Enforcement Responsibilities

Community leaders are responsible for clarifying and enforcing our standards of acceptable behavior and will take appropriate and fair corrective action in response to any behavior that they deem inappropriate, threatening, offensive, or harmful.

Community leaders have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that are not aligned to this Code of Conduct, and will communicate reasons for moderation decisions when appropriate.

Scope

This Code of Conduct applies within all community spaces, and also applies when an individual is officially representing the community in public spaces. Examples of representing our community include using an official e-mail address, posting via an official social media account, or acting as an appointed representative at an online or offline event.

Enforcement

Instances of abusive, harassing, or otherwise unacceptable behavior may be reported to the community leaders responsible for enforcement at help@azuracast.com. All complaints will be reviewed and investigated promptly and fairly.

All community leaders are obligated to respect the privacy and security of the reporter of any incident.

Enforcement Guidelines

Community leaders will follow these Community Impact Guidelines in determining the consequences for any action they deem in violation of this Code of Conduct:



On this page >

Consequence: A private, written warning from community leaders, providing clarity around the nature of the violation and an explanation of why the behavior was inappropriate. A public apology may be requested.

2. Warning

Community Impact: A violation through a single incident or series of actions.

Consequence: A warning with consequences for continued behavior. No interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, for a specified period of time. This includes avoiding interactions in community spaces as well as external channels like social media. Violating these terms may lead to a temporary or permanent ban.

3. Temporary Ban

Community Impact: A serious violation of community standards, including sustained inappropriate behavior.

Consequence: A temporary ban from any sort of interaction or public communication with the community for a specified period of time. No public or private interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, is allowed during this period. Violating these terms may lead to a permanent ban.

4. Permanent Ban

Community Impact: Demonstrating a pattern of violation of community standards, including sustained inappropriate behavior, harassment of an individual, or aggression toward or disparagement of classes of individuals.

Consequence: A permanent ban from any sort of public interaction within the community.



On this page >

Community Impact Guidelines were inspired by [Mozilla's code of conduct enforcement ladder](#).

For answers to common questions about this code of conduct, see the FAQ at <https://www.contributor-covenant.org/faq>. Translations are available at <https://www.contributor-covenant.org/translations>.

 Edit page

← Previous
Users

Next
Donate →

Storage Locations

For each station there are `Storage Locations` defined in AzuraCast for `Station Media`, `Station Recordings` and `Backups`.

A `Storage Location` can be one of the following types:

- Local Filesystem
- S3-Compatible
- Dropbox

S3-Compatible Storage Location



Media processing performance varies depending on the provider you're using (Backblaze, Digital Ocean, Linode, S3, etc) so don't panic if it's taking some time.

If the processing seems slow to you, measure the download speeds of your S3 bucket to determine if this is an issue with the S3 provider's resources before contacting us through our support methods.

This guide is aimed for smaller installations that have small storage disk sizes. Our guides will guide you through collecting the Access / Secret Keys. You'll require the following to use the S3 compatible service: `Access Key ID`, `Secret Key`, `Endpoint`, `Region`, `Bucket Name` and `API version` (API version should always be `latest`).

To configure a station to use an S3 compatible service, you'll need to go to

`yourdomain.com/admin/storage_locations`, click on `edit` and then tick the `Remote: S3 Compatible`



On this page



Overview

Add Storage Location



Storage Adapter

- ☐ Local Filesystem
- ☒ Remote: S3 Compatible

Path/Suffix

/bucket_media

For local filesystems, this is the base path of the directory. For remote filesystems, this is the folder prefix.

Storage Quota

Set a maximum disk space that this storage location can use. Specify the size with unit, i.e. "8 GB". Units are measured in 1024 bytes. Leave blank to default to the available space on the disk.

Remote: S3 Compatible

Access Key ID

Secret Key

Endpoint

Bucket Name

Region

API Version

latest

CLOSE

SAVE CHANGES

Amazon Simple Storage Service (S3)

[Amazon Simple Storage Service](#), otherwise known as (S3) is an object storage service. [S3's pricing](#) is more complex than the others, instead of offering a 'base' fee, it charges for what you use. You can use our small guide listed below or use Amazon's official documentation You can use our small guide to create a bucket and collect key's or use [Amazon's documentation](#)

- With Amazon's S3 system, you'll need to sign into the [S3 console](#) and click Create Bucket



On this page > Overview

- You can find your access and security key within the `My Security Credentials`

Backblaze

[Backblaze B2 Cloud Storage](#) is an S3-compatible service. It has a similar price plan to Amazon's S3 service but it has a [pricing calculator](#). You can use our small guide or use [Backblaze's extensive documentation](#) on B2 Cloud Storage.

- With Backblaze you'll need to sign in and go to your Buckets page.
- From there you can collect the S3 endpoint (Important Note: Buckets created prior to the 4th of May 2020 are not S3 compatible, if a bucket isn't compatible, create a new one)
- You can create the Application Key ID and Application Key in the App Keys page. (Important Note: Application Key ID = Access Key | Application Key = Secret Key)

Digital Ocean Spaces

[Digital Ocean Spaces](#) is an S3-compatible object storage service. The [basic plan](#) costs \$5 for 250GB with 1 TB outbound transfer. You can use our guide listed below or use [Digital Ocean's documentation on Spaces](#)

- To create the Spaces storage service you'll need to create an account through Digital Ocean, if you have one, you can skip this step.
- You can create a `Spaces` from the `Create menu and it'll take you to the page. You'll need to pick a datacenter region and enable or disable the CDN (optional).
- Finally you can choose a unique name for the Space, it has to be unique as it'll serve as your `Endpoint`.
- You'll need to create a Access/Secret Key by following [this guide](#) from Digital Ocean.

Linode Object Storage (S3)

[Linode Object Storage](#) is a storage service offered by Linode and the [basic plan](#) starts at \$5 for 250 GB with 1 TB outbound transfer. You can use our small guide listed below or use [Linode's](#)



On this page > Overview

- If this is the first bucket you've created with Linode, you'll need to confirm the Object Storage. (Skip this part if you have a bucket)
- After that you'll be able to generate a Access and Secret key.

Dropbox Storage Location

In order to create a Dropbox Storage Location in AzuraCast you will first need to create an OAuth 2 access token for your Dropbox account.

Go to the [Dropbox App Console](#) and click on the `Create app` button.

Select the following options:

- `1. Choose an API`
 - `Scoped access`
- `2. Choose the type of access you need`
 - If you are using your Dropbox other things apart from AzuraCast use `App folder` to automatically create a separate folder in your Dropbox account for this app
 - Otherwise select `Full Dropbox`
- `3. Name your app`
 - Give your app a name, it is recommended to choose a name specific to your radio

Then click on `Create app` to finish this step.

On the now generated page for you app switch to the `Permissions` tab at the top of the page and select the following permissions:

- `files.metadata.write`
- `files.metadata.read`
- `files.content.write`
- `files.content.read`

[On this page](#)[Overview](#)

Change the `Access token expiration` from `Short-lived` to `No expiration` and under `generated access token` click on `Generate`.

Copy the token that is now shown to you and paste it into the `Dropbox Auth Token` text box when creating a new Storage Location of type `Remote: Dropbox` in AzuraCast.

A small, light blue pencil icon.[Edit page](#)

← Previous
SSL & Let's Encrypt

Next →
Synchronized Tasks

AzuraRelay

AzuraRelay is “relay in a box” containing a lightweight web application and Icecast servers that can connect to and relay an AzuraRelay parent instance.

If you want to offer your streams to users around the world or accept a large volume of incoming traffic, AzuraRelay is the perfect companion to AzuraCast.

Parent Installation Requirements

Before installing AzuraRelay, make sure your “parent” AzuraCast installation:

- is updated to the latest version,
- has the direct radio ports (i.e. 8000, 8010) exposed (i.e. isn't behind a CloudFlare proxy)

System Requirements

- A 64-bit x86 (x86_64) CPU (ARM devices, like the Raspberry Pi 3/4, are not supported.)
- 512MB or greater of RAM
- 20GB or greater of hard drive space

For Linux hosts, the `sudo`, `curl` and `git` packages should be installed before installing . Most Linux distributions include these packages already.

Installing

AzuraRelay is powered by Docker and uses pre-built images that contain every component of the software. Don't worry if you aren't very familiar with Docker; our easy installer tools will handle

[On this page >](#)

Pick a base directory on your host computer that AzuraRelay can use. If you're on Linux, you can follow the steps below to use the recommended directory:



```
mkdir -p /var/azurarelay  
cd /var/azurarelay
```

Use these commands to download our Docker Utility Script, set it as executable and then run the Docker installation process:



```
curl -L https://raw.githubusercontent.com/AzuraCast/AzuraRelay/main/docker.sh > docker.sh  
chmod a+x docker.sh  
./docker.sh install
```

On-screen prompts will show you how the installation is progressing.

Updating

Using the included Docker utility script, updating is as simple as running:



```
./docker.sh update-self  
./docker.sh update
```

[Edit page](#)



On this page >

SSL & Let's Encrypt

AzuraCast now includes built-in support for creating and managing SSL (HTTPS) certificates via LetsEncrypt from the System Settings panel.

LetsEncrypt is a free and simple way to allow safe and secure connections to your AzuraCast installation. With a valid SSL certificate, you can:

- Secure your connection to AzuraCast when administering your stations,
- Enforce security for all AzuraCast administrators via HTTP Strict Transport Security (HSTS), and
- Provide a secure listening endpoint to listeners, avoiding “Mixed Content” warnings when your radio signal is played from a secure web page.

Important Considerations

Before setting up LetsEncrypt, you should make sure the following conditions are met:

- **AzuraCast must be on its own domain or subdomain.** You can't set up LetsEncrypt using only an IP address; you must have a domain (i.e. example.com) or a subdomain (radio.example.com) set up to point to your AzuraCast installation.
- **AzuraCast's web server must be served on the default ports, 80 for HTTP and 443 for HTTPS.** By default, AzuraCast is already set up this way, but if you've modified the ports to serve the site on a secondary port, you must switch the ports back to the defaults when setting up LetsEncrypt and when performing renewals.

Enabling LetsEncrypt



On this page >

- Click "System Settings"
- Select the "Services" tab
- Complete the LetsEncrypt section fields
- Click "Save Changes" at the bottom

The HTTPS certificate will automatically be generated in the next few minutes, but you can do it manually by clicking the "Create/Renew Certificate" button under the LetsEncrypt fields.



Tip

If users are still having issues with audio not playing, please ensure you have the "Use Web Proxy" option enabled in your System Settings.

Renewing a Let's Encrypt Certificate

The web service will automatically renew your LetsEncrypt certificates. If you provide an e-mail in the initial setup process, that e-mail will be used to send you reminders of upcoming expiration in the event that automatic renewal fails.

Always Use HTTPS (HTTP Strict Transport Security)

From the "System Settings" panel inside AzuraCast, you can enable the "Always Use HTTPS" setting in the "Security & Privacy" tab.

Once this setting is enabled, not only will all users be redirected to the secure version of AzuraCast when visiting, but HTTP Strict Transport Security (HSTS) will be enabled, which requires a valid SSL certificate to function. This significantly improves the security of your connection to AzuraCast and should be enabled whenever possible.

Disabling HSTS via CLI



On this page >

```
cd /var/azuracast
bash docker.sh cli azuracast:settings:set always_use_ssl 0
```

This will disable the setting and enable you to reconnect to your installation.

Using a Custom Certificate

If you have a custom SSL certificate on your host, you should create a `docker-compose.override.yml` file in your `/var/azuracast` directory on the host server with the contents below, modified to reflect your domain name and the path to your SSL certificate and key:

```
services:
  web:
    volumes:
      - /path/on/host/to/ssl.crt:/var/azuracast/acme/ssl.crt:ro
      - /path/on/host/to/ssl.key:/var/azuracast/acme/ssl.key:ro
```

Tip

Please note that Icecast expects an RSA private key as well as a certificate file with the complete certificate chain. For custom certificates in the `.pem` format generated by something like `Certbot` you will need to convert them like this:

```
openssl rsa -in privkey.pem -out example.com.key
```

```
openssl crl2pkcs7 -nocrl -certfile fullchain.pem | openssl pkcs7 -print_certs -out
example.com.crt
```

Finally you need to restart AzuraCast in order to apply the changes:



On this page >

 Edit page

← Previous
Roles & Permissions

Next →
Storage Locations

Live Demo

Want to see AzuraCast for yourself?

We have a demo site with a small AzuraCast installation and a single radio station, where you can see the basics of the AzuraCast web interface in action.

AzuraCast Demo



- **Username:** `demo@azuracast.com`
- **Password:** `demo`

Refreshing? Check back soon!

The AzuraCast demo instance automatically resets at the top of every hour to a fresh copy. If you try to access the demo site at exactly the top of the hour, you may be automatically redirected back to our website; just try again in a minute or two.

 Edit page

Previous
← About AzuraCast

Next
Install AzuraCast →

Docker

Docker installations come with four files by default:

- `docker.sh`, the Docker Utility Script;
- `.env`, which contains environment variables used by Docker Compose itself;
- `azuracast.env`, which contains customizable environment variables sent to AzuraCast and related services; and
- `docker-compose.yml`, a large file that defines all of the services used by AzuraCast and how they interact.

Docker Utility Script

If you're using the Docker installation method to run AzuraCast, we have created a helpful utility script to perform common functions without having to type long command names.

Download the Utility Script

If you've recently followed the [Docker installation instructions](#), you already have the Docker Utility Script installed. The file is named `docker.sh`.

If you have an older installation, you can use the Docker Utility Script by running these commands inside your AzuraCast directory on your host computer:

```
curl -L https://raw.githubusercontent.com/AzuraCast/AzuraCast/main/docker.sh > docker.sh
chmod a+x docker.sh
```




On this page



Overview

```
./docker.sh ci1 [command_name]
```

Runs any command exposed by the command line interface tools.

Available Commands

Install AzuraCast

```
./docker.sh install
```

Pulls the latest version of all Docker images and sets up the AzuraCast database. When complete, your AzuraCast instance should be up and running.

Update AzuraCast

```
./docker.sh update-self  
./docker.sh update
```

Note: You should always run update-self first to update the updater script.

Automatically pulls down any updated Docker images and applies any database and configuration changes since your last AzuraCast update.

Uninstall AzuraCast

```
./docker.sh uninstall
```



On this page >

Overview

This command will fully remove any station media, statistics and metrics, and the entire database associated with your AzuraCast instance.

Back Up Files and Settings

```
./docker.sh backup [/path/to/backup.tar.gz]
```

Creates a .tar.gz backup copy of the media, statistics and metrics of every station, along with a copy of the full AzuraCast database. You can later restore from this same file in the event of data loss or corruption.

Restore Files and Settings

```
./docker.sh restore /path/to/backup.tar.gz
```

Extracts a .tar.gz file previously created by this same script's backup command, copying media, statistics and metrics for each station into AzuraCast and importing the version of the database contained in the backup.

⚠ This command replaces your current installation!

Restoring from a backup will remove any existing AzuraCast database or media that exists inside the Docker volumes.

Customizing Docker Installations



On this page > Overview

- When updating (using the `docker.sh` utility script), it is recommended to run `./docker.sh update-self` before running `./docker.sh update`, to ensure the Docker Utility Script itself is up to date before it updates your Docker installation.
- Environment variables set in `.env` are only used by Docker Compose itself, and aren't passed directly into the AzuraCast containers. You should only modify this file to change the HTTP and HTTPS port mappings used by Nginx (see the "Use Non-Standard Ports" section above).
- The `azuracast.env` file is specific to your environment and can be customized however you like. It will not be replaced during any updates. Once your database has been created, however, changing the password listed in this file will cause the system to fail. If you want to destructively wipe your existing database and other files and set up a new one with the updated password, add the `-v` flag to the end of `docker-compose down` to remove all existing volumes, including your database.
- If possible, you should not directly modify `docker-compose.yml`, as some updates may modify how it is defined to resolve bugs or add new features. When updating, you will always be asked if you want to update this file; if you have not modified it, you should always do so.
- Instead of modifying `docker-compose.yml`, you can create a file named `docker-compose.override.yml` with your customizations. The structure of this file is the same as the main `docker-compose.yml` file, and is automatically parsed by Docker Compose to override any definitions in the main file. Updates will not replace this file.

Modifying Docker

The AzuraCast Docker installation is built to serve the needs of the vast majority of installations out of the box, but there may be times when you need to customize how Docker serves your installation, while still retaining the benefits of compatibility and easy updating that Docker offers.

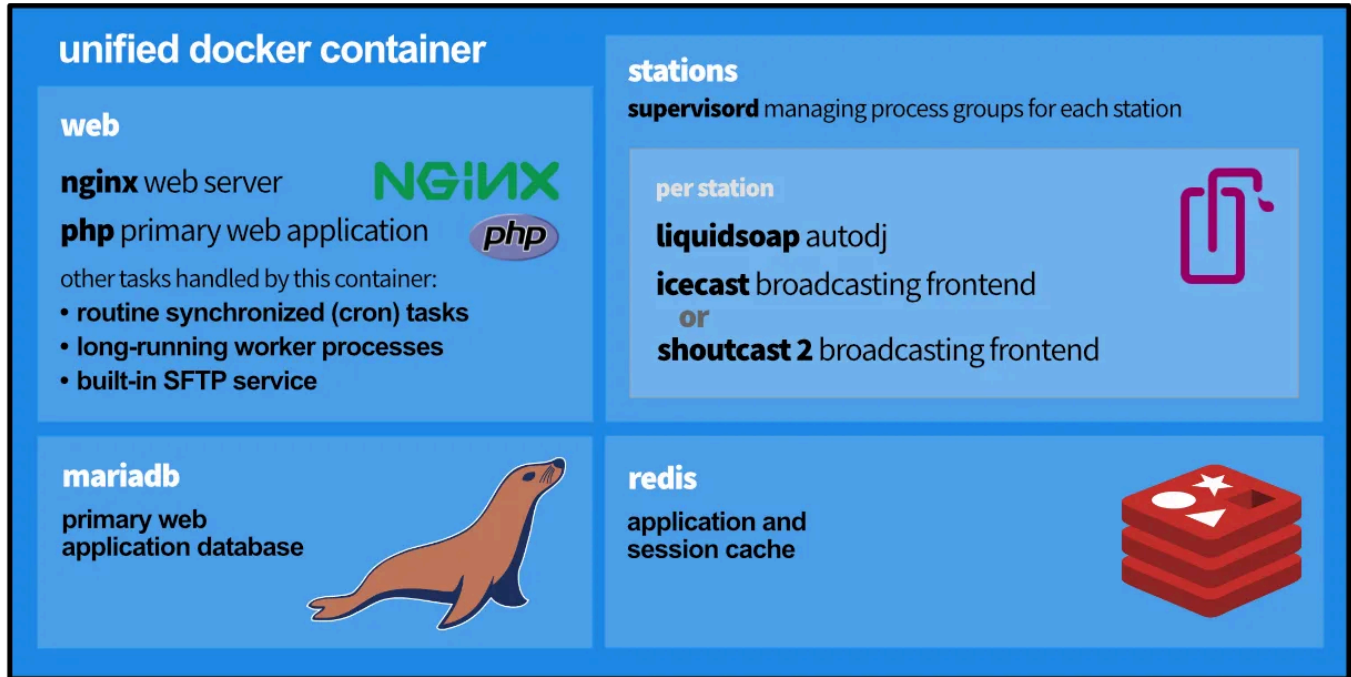
The Overall Infrastructure



On this page



Overview



Docker Compose

In order to make our Docker installation simple to maintain, we rely on a secondary piece of software called [Docker Compose](#). Rather than using lengthy command-line commands, Docker Compose allows you to write simple YAML files to configure the way our Docker containers launch, which ports they forward to the host, mounted disk volumes, etc.

If you follow our installation instructions (or use one of our prebuilt images), your Docker installation is located on your host computer at `/var/azuracast`. Within this folder, you'll see four files by default:

- `docker-compose.yml`, the primary Docker Compose configuration file. AzuraCast's updater automatically prompts you to keep this file updated, which is strongly recommended;
- `docker.sh`, the Docker Utility Script that provides easy aliases for common Docker tasks, such as updating, executing commands inside containers, and more;



On this page



Overview

thus used for higher-level configuration changes like port mappings.

Overriding Docker Compose

Rather than modifying `docker-compose.yml` directly, it is strongly recommended to instead create a new file called `docker-compose.override.yml` in the same folder. Docker Compose will automatically look for and include this file if it exists, and apply its configuration on top of the base file's instructions.

There are some considerations when creating your own `docker-compose.override.yml` :

- The file must be valid YAML using the standard [Docker Compose format](#).
- When modifying ports or other lists, you can only add new items, not remove existing ones.

Once you've modified your Docker Compose configuration, you should apply these changes by running:

```
docker-compose down
docker-compose up -d
```

Note that this will temporarily shut down your AzuraCast installation and will briefly disconnect your listeners.

Using Non-standard Ports

**Tip**

In newer versions, you no longer need to manually do this; you can instead use the installer by answering "Yes" when prompted to customize settings.



On this page



Overview

```
cd /var/azuracast
./docker.sh update-self
./docker.sh change-ports
```

To override more complex functionality in your Docker installation, see the “Customizing Docker” section below.

Expanding the Station Port Range

**Tip**

Installations running version 0.16.0 or higher can now use our installation script to customize ports via the terminal which will automatically write the changes into the `docker-compose.yml` file for you.

For performance reasons, by default Docker installations only open radio ports from port 8000 to 8500. This allows for 50 unique stations to operate.

Depending on your hardware, it may be possible to run more than 50 stations on one AzuraCast instance, but if you want to directly access the additional radio ports, you can follow this simple process.

In the same folder where your Docker installation is (if using recommended instructions, this is `/var/azuracast`), create a new file named `docker-compose.override.yml`.

In this file, paste the following contents:

```
services:
  web:
    ports:
```



On this page



Overview

When creating a new station, AzuraCast will attempt to automatically assign it an available port from the available port range. If you change this port range, you should let AzuraCast know by adding or updating the following values in your `azuracast.env` file:

```
AUTO_ASSIGN_PORT_MIN=8500
AUTO_ASSIGN_PORT_MAX=8999
```

You will need to restart your Docker containers using `docker-compose down`, then `docker-compose up -d` to apply any changes made to these files.

Mounting a Directory Into a Station

You may want to add music to a station from a directory on your host machine without copying the data into AzuraCast. You can mount the directory into your stations and web container to make them available to AzuraCast by creating a `docker-compose.override.yml`.

In the same folder where your Docker installation is (if using recommended instructions, this is `/var/azuracast`), create a new file named `docker-compose.override.yml`.

In this file, paste the following contents:

```
services:
  web:
    volumes:
      - /path/on/host/computer:/var/azuracast/stations/<STATION_NAME>/media/
```

Replace the `<STATION_NAME>` with the name of the station directory found under the "Administration" section of the station's profile settings and modify the `/path/on/host/computer` with the path to the directory that you want to mount.

You will need to restart your Docker containers using `docker-compose down`, then `docker-compose up -d` to apply any changes made to this file.



On this page



Overview

In the same folder where your Docker installation is (if using recommended instructions, this is `/var/azuracast`), create a new file named `docker-compose.override.yml`.

In this file, paste the following contents:

```
services:
  web:
    volumes:
      - /path/on/host/computer:/var/azuracast/stations
```

Modify the `/path/on/host/computer` with the path to the directory that you want to mount.

You will need to restart your Docker containers using `docker-compose down`, then `docker-compose up -d` to apply any changes made to this file.

Custom Default Track

**Tip**

In newer versions of AzuraCast, you no longer need to upload a custom default track this way; you can upload a unique "Custom Fallback Track" per-station using the web interface.

When nothing is playing on your station you'll hear the default `error.mp3` file of AzuraCast playing. You can replace this file by mounting your own `.mp3` file via a `docker-compose.override.yml`.

In the same folder where your Docker installation is (if using recommended instructions, this is `/var/azuracast`), create a new file named `docker-compose.override.yml`.

You can place your `.mp3` file anywhere on your host machine. You just have to specify the path to it by replacing this part: `/path/to/your/file.mp3`



On this page > Overview

In this file, paste the following contents:

```
services:
  web:
    volumes:
      - /path/to/your/file.mp3:/usr/local/share/icecast/web/error.mp3
```

You will need to restart your Docker containers using `docker-compose down`, then `docker-compose up -d` to apply any changes made to this file.

Stream Intro File



Tip

In newer versions of AzuraCast, you can now directly upload an intro track on the Mount Point edit page.

You can add a music file to play when someone initially connects to your stream. Remember when creating intro files that they **must match the exact same format, bitrate and sample rate as your mount point to work properly.**

First, tell the Docker filesystem where to find your intro file. Inside the AzuraCast directory on your host (by default, `/var/azuracast`), create a file named `docker-compose.override.yml` with the following contents:

```
services:
  stations:
    volumes:
      - /path/to/your/file.mp3:/usr/local/share/icecast/web/intro.mp3
```

Now restart AzuraCast via `docker-compose down && docker-compose up -d`.



On this page > Overview

```
<intro>/intro.mp3</intro>
```

For more information, see the [Icecast documentation](#).

Custom Favicon



Tip

In newer versions of AzuraCast, you can manage this via the web UI.

To override the favicon and other browser icons used by AzuraCast, you should follow these steps:

1. Upload your icon to [favicon-generator.org](#). Be sure to leave "Generate icons for Web, Android, Microsoft, and iOS (iPhone and iPad) Apps" checked.
2. Once your icon set is downloaded, it will be in a .zip file format; extract that file and copy it to the host server where AzuraCast is hosted.
3. Inside the AzuraCast directory on your host (by default, /var/azuracast), create a file named docker-compose.override.yml with the following contents:

```
services:
  web:
    volumes:
      - /path/to/your/favicon/folder:/var/azuracast/www/static/icons/production
```

4. Restart AzuraCast via `docker-compose down && docker-compose up -d`.

Increase PHP Allowed Memory

When you encounter an error like the following one you will need to increase the amount of memory that PHP is allowed to use:



On this page



Overview

There you can find the following entry:

```
# PHP's maximum memory limit.  
# PHP_MEMORY_LIMIT=128M
```

Change this to look like this:

```
# PHP's maximum memory limit.  
PHP_MEMORY_LIMIT=512M
```

You should increase the value until the error dissappears.

The `M` after the value stands for `Megabytes`.

Refer to [PHP's documentation](#) for more information about the allowed values for this setting.

After you have changed this setting you need to restart AzuraCast via `docker-compose down && docker-compose up -d`.

Adding custom NGINX configurations

If you need to add some custom configurations to NGINX you can do this by your `custom.conf` file into the `web` container via the `docker-compose.override.yml` like this:

```
services:  
  web:  
    volumes:  
      - ./custom.conf:/etc/nginx/azuracast.conf.d/custom.conf
```

All config files placed in the `/etc/nginx/azuracast.conf.d/` are automatically included in the main AzuraCast NGINX configuration.

[On this page](#)[Overview](#)A small, light blue pencil icon. [Edit page](#)

Previous
← Using Custom JavaScript

Next
Multi-Site Docker →
Installation

Backup & Restore

AzuraCast includes a built-in utility for backing up your installation, including your database, all stations and their settings, and optionally the local media tied with the station(s).

Ways to Backup

Via the Web Interface

The “System Administration” section of your AzuraCast web interface has a dedicated “Backup” page, where you can configure periodic automated backups, run a one-time backup and download existing backup files.

Backups run through the web panel do not interrupt your broadcasts or disconnect your listeners.

Via the Command Line

If you want more control over where your backup files are going, or want to take advantage of pre-existing tools (like cron tasks) to handle the backup, you can use the Docker Utility Tool to generate backups as well.

The most basic version of this command is:

```
cd /var/azuracast
./docker.sh backup path-to-backup.zip
```

You can also pass the `—exclude-media` flag to back up just the database and statistics, but not the media itself, which significantly reduces the backup file size (but you should make sure to back your



On this page >

```
./docker.sh backup path-to-backup.zip --exclude-media
```

Both .zip and .tar.gz formats are supported for backups. The correct format will automatically be determined by the extension of the filename you specify for the backup file.

Restoring a Backup

These are the general steps for restoring a backup of AzuraCast:

This may take a while...

Large backups (100GB+) can take a considerably long time to fully restore onto the server and *may use* large amounts of CPU and or Memory.

1. Install AzuraCast ([see our guide](#))
 - If you have an older backup file and encounter issues with restoring, install AzuraCast on `stable` by answering `Y` to the question "Prefer stable release versions of AzuraCast?"
2. After the installation is finished, run the restore command:
 - `./docker.sh restore path-to-backup.zip`
3. If you have used the `stable` installation due to an older backup file you should now switch back to the `rolling-release` via the following command and answer `N` to the question if you want to use the stable version:
 - `./docker.sh setup-release`
4. Finally run the following command to update the installation to the current rolling-release version:
 - `./docker.sh update`

 Edit page



On this page >

SFTP Server



Tip

The Built-in SFTP server is only available on the Docker installation.

All AzuraCast Docker installations come with a built-in SFTP server that allows you to easily connect to your media directory, upload and move files using software of your choice.

Creating SFTP Users



Tip

You will only be able to connect to the SFTP server to manage your media if your station's storage location is a "Local" one to the server. If you're connecting to S3, Dropbox, etc. then you can instead use clients that can connect directly to the same storage location your station is using.

Under the **Media** sidebar menu for a given station, you will see an entry labeled **SFTP Users**. Click this to view existing users and the connection details for connecting to your server via an SFTP client of your choice.

Default Port

By default, the SFTP service listens externally on port 2022. If there is something else using this port or you want to change it for any other reason, you can edit the **.env** file on your host machine and add or modify this line:



On this page >

Note, this is a file named `.env`, with no filename, and not the `azuracast.env` file

Connecting to SFTP

SFTP clients

Here's a small list of FTP clients you can use that are known to work with AzuraCast's SFTP system.

- [Filezilla](#) - Windows, MacOS and Linux 32 bit & 64 bit
- [WinSCP](#) - Windows only
- [Cyberduck](#) - Windows and MacOS

A more exhaustive list can be found [here](#).

Mounting as a Directory on Your Computer

There are also numerous tools that let you create a virtual filesystem based on an SFTP connection, which then lets you drag and drop files to and from the filesystem using your OS's standard file explorer:

- [Mountain Duck](#) - Windows and MacOS
- [ExpanDrive](#) - Windows, MacOS and Linux
- [SSHFS for Linux/MacOS](#)
- [SSHFS for Windows](#)

For more information on configuring SSHFS, see [this thread](#).

Edit page



On this page >

Media Management

Media management is at the core of AzuraCast's built-in AutoDJ. By uploading media and assigning it to a playlist, the AutoDJ will automatically compose a live stream broadcast 24 hours a day, 7 days a week.

**Tip**

For your AutoDJ to detect any files in your Media Manager, they must be added to at least one playlist. You can have media files in your station's media directory that aren't associated with any playlist, but they won't be played automatically.

Uploading Media

Web Uploader

You can upload media directly via your web browser, by either selecting files or clicking and dragging them into a section at the top of the Media Manager.

SFTP (Docker Only)

For Docker installations, AzuraCast offers a built-in SFTP server which allows you to easily manage your media in bulk. Visit the [SFTP Server](#) guide for more information.

S3 Compatible

Stations can utilize S3 compatible services (Backblaze, Digital Ocean Spaces, Linode Storage and S3) to manage it's media remotely, view our [S3 configuration](#) guide for more details.

[On this page](#) >

Once assigned to a playlist, any files added to the folder will automatically be added to those same playlists. The system won't remove any additional playlists that the media is in, but it will add the missing ones from the folder.

Required Permissions

To manage media, users must be in a role that has the "Manage Station Media" permission for the given station.

On-Demand Media

Music from playlists with on-demand streaming enabled in playlist settings (under "Include in On-Demand Player") will be available to stream via the On-Demand Media public page. For this option to become available, "Enable On-Demand Streaming" under [Profile](#) -> [Edit Profile](#) -> [On-Demand Streaming](#) must be enabled.

[✎ Edit page](#)

← Previous
Troubleshooting

Next →
Mount Points

Command-Line Interface

AzuraCast features a powerful Command-Line Interface (CLI) tool that allows you to perform maintenance and troubleshooting tasks “under the hood”.

In order to use the AzuraCast CLI, you must have shell terminal access to the server running AzuraCast (to the host machine, if you’re running Docker).

Some of these commands are only intended for developers to debug or to resolve issues, use caution when executing these commands and only do them when told to, or you know what you’re doing.

Invoking the CLI

Using the Docker Utility Script:

If you’re using the Docker installation method, it’s recommended to download the Docker Utility Script, which will allow you to use this shorter syntax:

...

```
./docker.sh cli [command]
```

From a Docker Installation:

...

```
docker-compose run --rm cli azuracast_cli [command]
```

Ansible Installation:

[On this page >](#)

Account Related CLI Commands

This section will cover all account related CLI commands and the response the CLI will provide.

List all accounts

```
(cli_command) azuracast:account:list
```

AzuraCast User Accounts

=====

E-mail Address	Name	Roles	Created

name@domain.com		Super Administrator	YYYY-MM-DD HH:MM

Create a unique login recovery URL for the specified account

```
(cli_command) azuracast:account:login-token [example]
```

Generate Account Login Recovery URL

=====

The account recovery URL is:

`https://domain/recover/hash:hash`

Log in using this temporary URL and set a new password using the web interface.



On this page >

```
(cli_command) azuracast:account:reset-password [example]
```

Reset Account Password

=====

The account password has been reset. The new temporary password is:

[example]

Log in using this temporary password and set a new password using the web interface.

Set the account specified as a global administrator

```
(cli_command) azuracast:account:set-administrator [example]
```

Set Administrator

=====

The account associated with e-mail address "[example]" has been set as an administrator

API Related CLI Commands

This section will cover our API related command(s), this'll be related to your AzuraCast's API.

Regenerate AzuraCast's API documentation

```
(cli_command) azuracast:api:docs
```



On this page >

Internal CLI Commands

This section will cover our internal processes, such as Liquidsoap, SSL and SFTP authentication.

Get the external IP address for this instance

```
(cli_command) azuracast:internal:ip  
  
[example]
```

Running this command will return the IP of this machine.

Handle Liquidsoap API calls

```
(cli_command) azuracast:internal:liquidsoap [example] [example]
```

This command can run various Liquidsoap API calls, this is primarily used for debugging or development purposes.

Reload broadcast frontends when an SSL certificate changes

```
(cli_command) azuracast:internal:on-ssl-renewal
```

Attempt SFTP Authentication



On this page >

Send upcoming song feedback from the AutoDJ back to AzuraCast



```
(cli_command) azuracast:internal:sftp-event
```

AzuraCast Settings CLI Commands

This section will cover our internal settings ranging from base URL to smtp. This can be used should you be locked out of the settings or unable to access AzuraCast directly from the browser.

List all settings for AzuraCast



```
(cli_command) azuracast:settings:list
```

This command will return all the settings you can modify via the Command Line Interface, which will be shown in the next field.

Modify settings for AzuraCast



```
(cli_command) azuracast:settings:set [example] [example]
```

The `setting-key` parameter can be collected from the `settings:list` CLI command, this command will allow you to modify almost all portions of AzuraCast's settings, this is useful should you be having issues with misconfigured settings.



On this page >

```
(cli_command) azuracast:setup [ --update ]
```

Runs any necessary database updates to bring your AzuraCast installation to the latest version. This is normally run automatically as part of the installation and update processes, but can be run manually for troubleshooting or local development.

⚠ This command disconnects listeners!

Running this command will disconnect all current active listeners to your radio stations.

Clear All Caches

```
(cli_command) cache:clear
```

Clears all caches used internally by AzuraCast. This can be used as a troubleshooting step if you are encountering issues with out-of-date information appearing on dashboard pages. Note that some pages may take slightly longer to load after all caches are cleared.

Run Synchronization Tasks

```
(cli_command) sync:run [nowplaying|short|medium|long]
```

Manually invoke the synchronized tasks ("cron jobs") that normally run automatically behind the scenes to keep AzuraCast updated.

- **nowplaying** corresponds to the every-15-second check of all stations' currently playing song and listener metrics
- **short** corresponds to the every-minute sync task
- **medium** corresponds to the every-5-minutes sync task



On this page >

Manually Reprocess All Media



```
(cli_command) azuracast:media:reprocess
```

Iterates through all stations' media directories and manually reloads the metadata information stored inside AzuraCast with the latest data on the files themselves. This is useful for troubleshooting songs that are stuck in "Processing" status, or if you have recently uploaded multiple songs via SFTP.

Restart All Radio Stations



```
(cli_command) azuracast:radio:restart
```

Shuts down both the frontends (Icecast, SHOUTcast, etc) and backends (Liquidsoap) of all radio stations, rewrites their configuration files, then relaunches them. This is identical to the "Restart Broadcasting" command inside the web interface.

This command disconnects listeners!

Running this command will disconnect all current active listeners to your radio stations.

These tasks can also be invoked directly from the web interface via the Administration homepage.

Other Commands

The AzuraCast CLI interface also exposes a number of other advanced commands. These commands are intended for developers to use when building the application, and often should not be run by station owners on production installations.



On this page >

- [Doctrine Command Line Reference](#)
- [Doctrine Migrations](#)

 Edit page

← Previous
Cloudflare

Next →
Using Custom CSS

Streaming Software

⚠ Caution

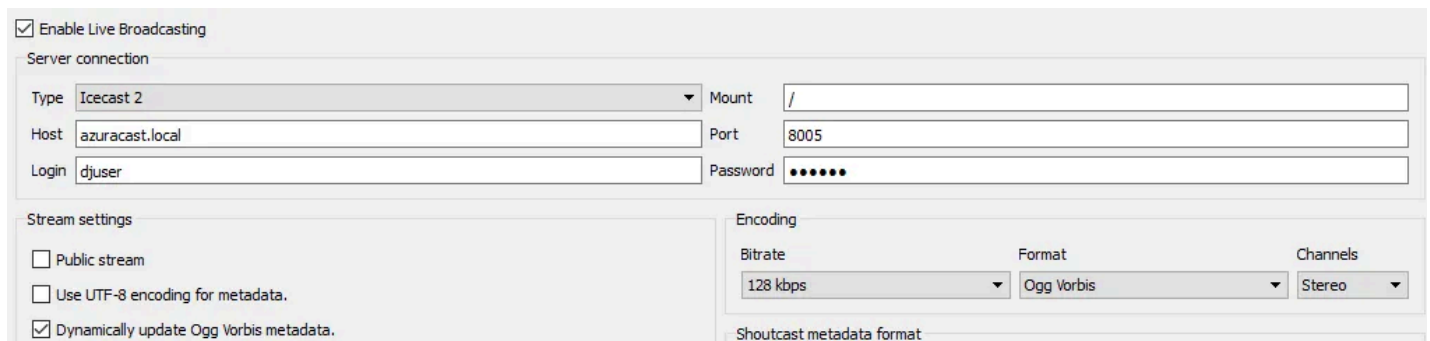
If you are using a reverse proxy service (like CloudFlare) in front of your installation, be aware that these services often prevent direct connections to the ports that broadcasters connect to. You should instruct your streamers to use the direct IP address of your server instead.

Mixxx

Linux, MacOS, Windows; Free and Open-Source

[Mixxx](#) is free and open-source DJ software that also supports live broadcasting via Icecast and ShoutCast servers.

Live broadcasting settings can be found in the preferences menu ([Options](#) -> [Preferences](#)). It is recommended to use "Icecast 2" mode when broadcasting to AzuraCast. Out of the box, Mixxx does not include the LAME MP3 encoder (for licensing reasons), but it is not needed when connecting to AzuraCast; just set your broadcast format to 'Ogg Vorbis' and LiquidSoap will transcode the broadcast into the correct output format.



The screenshot shows the 'Live Broadcasting' settings in Mixxx. The 'Enable Live Broadcasting' checkbox is checked. Under 'Server connection', 'Type' is set to 'Icecast 2', 'Mount' is '/', 'Host' is 'azuracast.local', 'Port' is '8005', 'Login' is 'djuser', and 'Password' is masked with dots. Under 'Stream settings', 'Public stream' and 'Use UTF-8 encoding for metadata' are unchecked, while 'Dynamically update Ogg Vorbis metadata' is checked. Under 'Encoding', 'Bitrate' is '128 kbps', 'Format' is 'Ogg Vorbis', and 'Channels' is 'Stereo'. A 'Shoutcast metadata format' section is partially visible at the bottom.



On this page



Overview

Cross-platform and available for free at <https://sourceforge.net/projects/butt/>. Streaming via both Icecast mode and ShoutCast mode are available.

Icecast Mode

Use the exact information specified on the Streamer/DJ page's right sidebar. "Address" is "Server" and "Icecast mountpoint" is "Mount Name".

The screenshot shows the AzuraCast interface. On the left, the 'Streamer/DJ Accounts' table has columns for ACTIONS, USERNAME, PASSWORD, and NOTES. It contains one entry with USERNAME 'djuser' and PASSWORD 'djpass'. An 'Edit Server' dialog box is open in the center, showing fields for Name (AzuraCast Local), Type (IceCast selected), Address (azuracast.local), Port (8005), Password (djpass), IceCast mountpoint (/), and IceCast user (djuser). On the right, the 'Connection Information' sidebar shows 'IceCast Clients' and 'ShoutCast v1 Clients' with their respective details.

ACTIONS	USERNAME	PASSWORD	NOTES
EDIT DELETE	djuser	djpass	

Edit Server

Name:

Type: ☐ ShoutCast ☒ IceCast

Address: Port:

Password: [Hide Password](#)

IceCast mountpoint: IceCast user:

[Cancel](#) [Save](#)

Connection Information

IceCast Clients

Server: azuracast.local

Port: 8005

Mount Name: /

ShoutCast v1 Clients

Server: azuracast.local

Port: 8006

Password: dj_username:dj_password
(DJ username and password separated by a colon)

ShoutCast Mode

Use the given port specified in the "ShoutCast v1 Clients" section. Since ShoutCast v1 does not let you specify a username, specify your password as `dj_username:dj_password` (the username and password for the DJ separated by a colon).



On this page



Overview

SAM Cast

Windows; Proprietary, Commercial

[SAM Cast](#) is commercial software sold by Spatial. It is compatible with AzuraCast by using the MP3 encoder and the "Icecast 2" server connection settings.

Broadcasting Steps

From the main SAM Cast window, click "Encoders" at the bottom left.



On this page



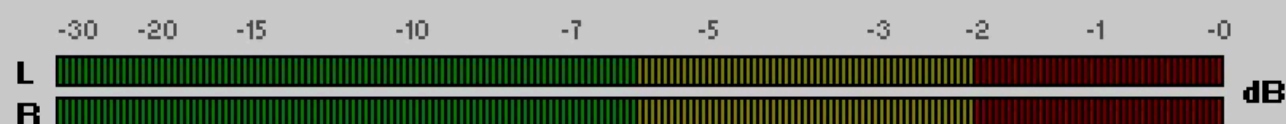
Overview

Stream Time: 0d, 00:00:00

Current Listeners: 0

Current Time: 1:10:56 PM

Peak Listeners: 0

**START**

Event log

Clear Log

2019-06-09 12:57:24> SAM Cast configuration loaded.
2019-06-09 12:57:24> Initialization completed.

Encoders

Statistics

Sound Processing

In the Encoders window that pops up, click the plus sign button on the toolbar to add a new encoder.



On this page >

Overview

The interface shows a menu bar with a 'Menu' dropdown and several icons: a plus sign (being clicked by a mouse cursor), a crossed hammer and wrench, a minus sign, a play button, a stop button, and a document icon. Below the menu bar is a table with the following columns: #, Format, Status, and Description. The table is currently empty. At the bottom of the interface is a volume meter for L and R channels, with a scale from -30 to -0 dB. The meter shows green bars for the L channel and red bars for the R channel.

You will be asked to select an encoder. It is recommended to select `MP3 (LAME_ENC)`, because this allows for the easiest constant bit rate (CBR) streaming, which causes the fewest issues when broadcasting.

Click "OK" at the bottom to continue to encoder details.



On this page



Overview

- ☐ aacPlus v2
- ☒ MP3 (LAME_ENC)
- ☐ WMA (v9 advanced) [ROBUST]
- ☐ WMA (v9 advanced)
- ☐ Ogg (Vorbis)
- ☐ CheapestStream.com Encoder
- ☐ mp3PRO

No details available

OK

Cancel

On the first tab of the encoder details, change your bitrate from the default 64kbps to your desired bitrate. This should likely match the highest bitrate that you broadcast out to your listeners. By default, it should at least be 128kbps.



On this page



Overview

Bitrate kbpsMode Sampling rate Hz☐ Auto start encoder after seconds☒ Auto reconnect encoder after seconds☐ Allow scripts in stream

OK

Cancel

Click the "Server Details" tab. On the "Server Type" selector, choose "Icecast", then choose "Icecast 2" in the secondary selector.

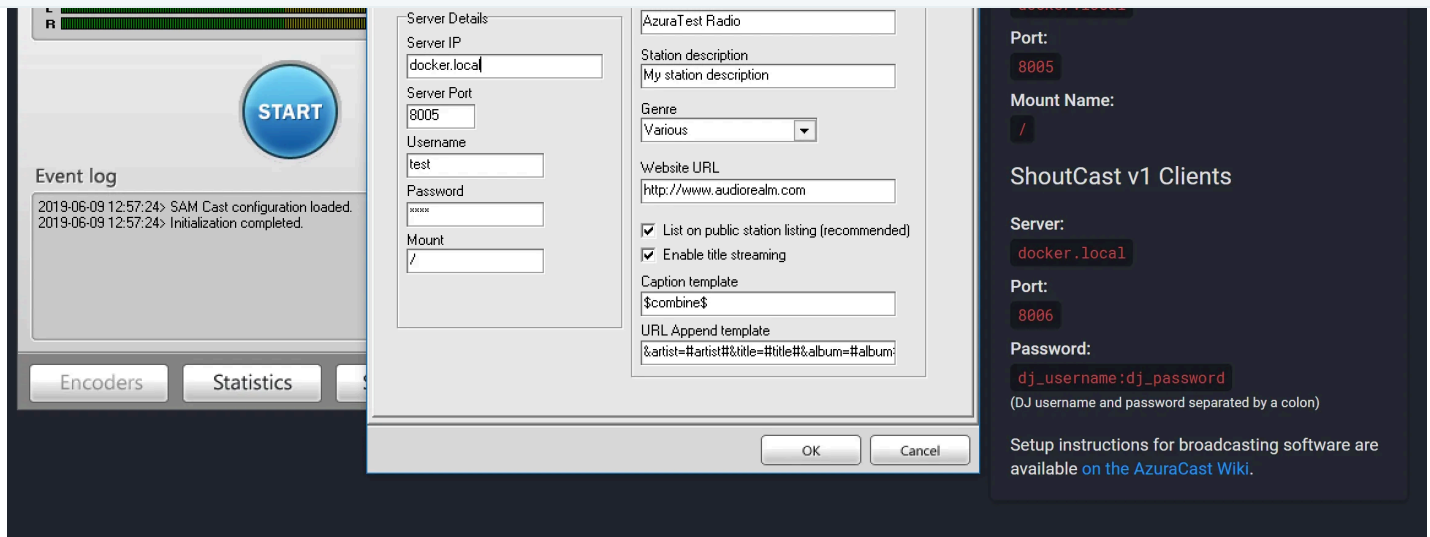
You can now populate the "Server Details" section with the connection information supplied to you via AzuraCast's "Streamers/DJs" page. You will want to use the connection information for "Icecast Clients", the first section on the page.



On this page



Overview



Click OK to return to the main Encoders pane, where you will now see a new encoder with “Idle” status.

Select the encoder, then click the play-button icon in the toolbar to begin broadcasting.



On this page >

Overview

#	Format	Status	Description
1	MP3: 128kb/s (CBR), Auto...	Idle	Encoder not started

RadioBOSS

Windows; Proprietary, Commercial

[RadioBOSS](#) is a commercial software sold by IP Kuzmitski D.V. at DJSoft.net.

Streaming via both Icecast mode and ShoutCast mode are available.

The Live broadcasting feature is possible in their Standard and Advanced edition (not in the Express edition).

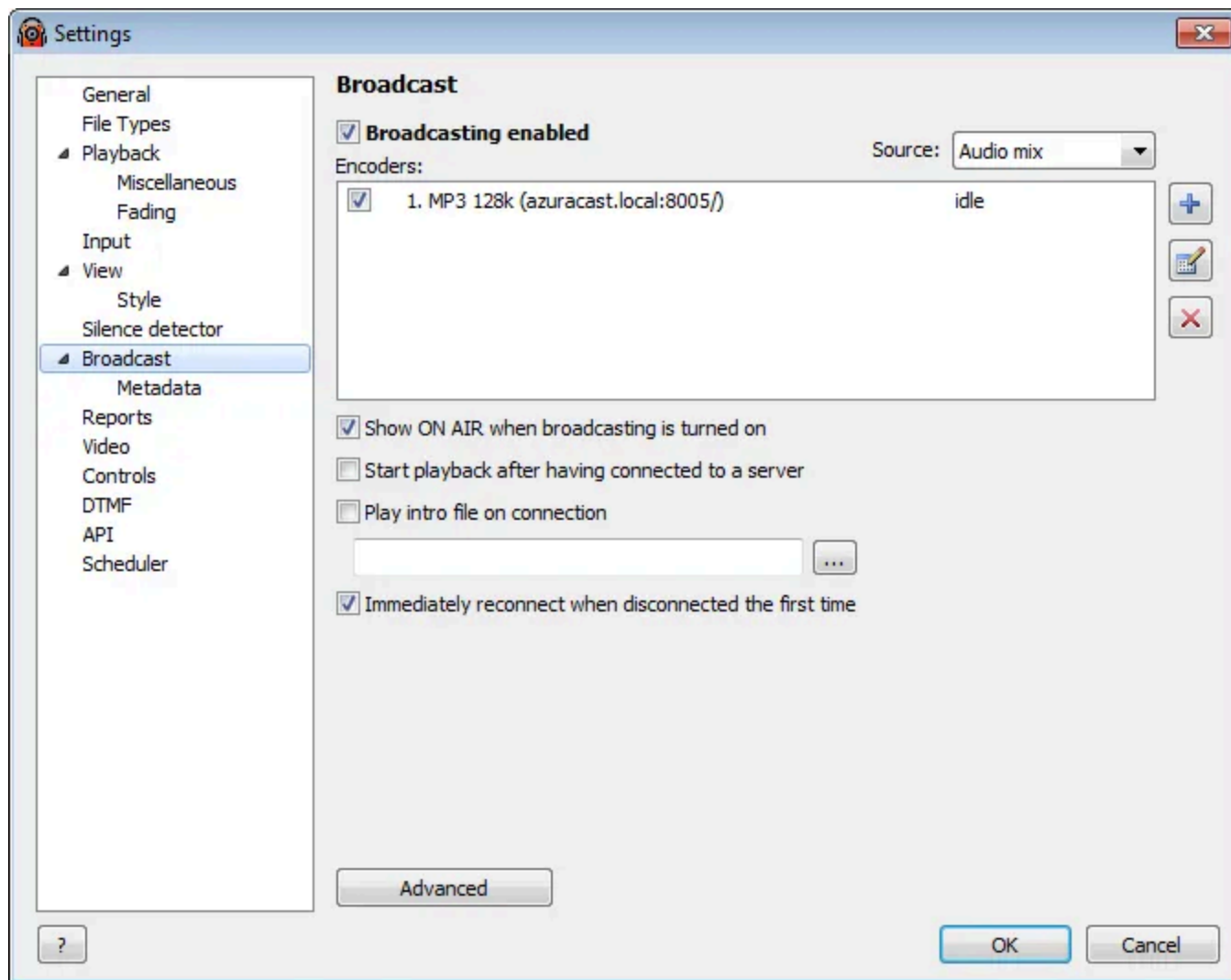
Note that for ease of use we use the [Configuration Wizard](#), you can also enter all connection details manually.



On this page > Overview

2. In the **Settings** window, select **Broadcast** on the left

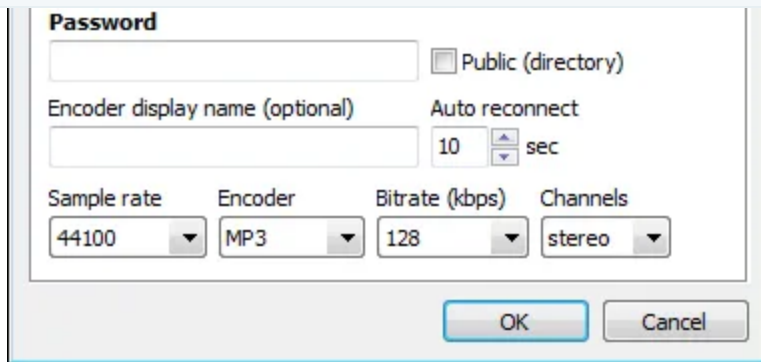
3. To add a new encoder, click the **+** button



4. The Encoder-window will open, next click the button called **Configuration Wizard**.



On this page > Overview

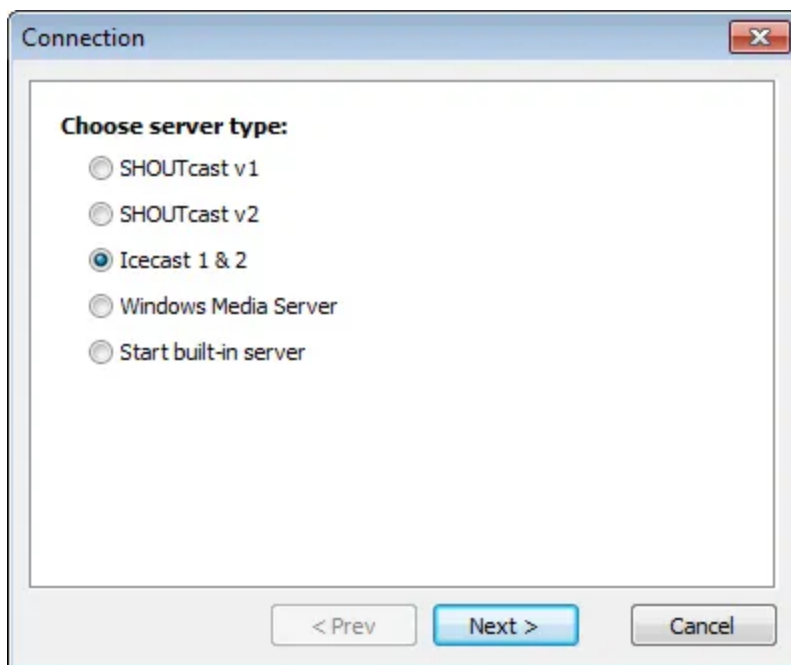


A configuration dialog box for AzuraCast. It has a title bar with 'Password' and a close button. The dialog contains several fields and controls: a password input field, a checkbox for 'Public (directory)', an 'Encoder display name (optional)' input field, an 'Auto reconnect' section with a numeric input set to '10' and a 'sec' unit, and a section for audio settings with four dropdown menus: 'Sample rate' (44100), 'Encoder' (MP3), 'Bitrate (kbps)' (128), and 'Channels' (stereo). At the bottom are 'OK' and 'Cancel' buttons.

Follow the instructions for the server type you have set up your Azuracast station with:

Icecast

- Select **Icecast 1 & 2** and click **Next >**



A 'Connection' dialog box with a title bar and a close button. It contains a section titled 'Choose server type:' with five radio button options: 'SHOUTcast v1', 'SHOUTcast v2', 'Icecast 1 & 2' (which is selected), 'Windows Media Server', and 'Start built-in server'. At the bottom are three buttons: '< Prev', 'Next >', and 'Cancel'.

- Take over the corresponding credentials you can find in Azuracast:



On this page > Overview

8005

Username (optional)

djuser

Password

••••••

Mount point

/

< Prev Finish Cancel

- Click **Finish**

SHOUTcast

The following settings are recommended for both ShoutCast v1 as v2 (DNAS 2)

Note 1: Use the given port specified in the "ShoutCast v1 Clients" section.

Note 2: Be sure to select **v1**, as v2 will append the Stream ID (SID), which won't work.

- Select **SHOUTcast v1** and click **Next >**

Connection

Choose server type:

☒ SHOUTcast v1

☐ SHOUTcast v2

☐ Icecast 1 & 2

☐ Windows Media Server

☐ Start built-in server

< Prev Next > Cancel



On this page > Overview

A screenshot of the AzuraCast configuration window. It has a light gray border and a white background. At the top, there's a text input field containing 'azuracast.local'. Below it, the word 'Port' is in bold, followed by a spin box showing '8005'. Then, 'Username (optional)' is in bold, followed by a text input field containing 'djuser'. Below that, 'Password' is in bold, followed by a password input field with seven dots. At the bottom, there are three buttons: '< Prev' (disabled), 'Finish' (active/highlighted), and 'Cancel' (disabled).

- Click **Finish**

6. Finish by changing the rest of the settings by preference:

- Connection
 - Sample Rate
 - Encoder
 - Bitrate
 - Channels
 - Station info

If needed, here you can override the info that can be found in the main **Settings** (From step 1), under **Broadcasting** -> **Metadata**

1. Then finally click **OK** to save the Encoder.

2. In the Settings window, check the checkboxes for all encoders that you want to use. Remember to turn on the option **Broadcasting enabled** on top.



On this page > Overview

If the logs show “[E40] Cannot start broadcast” in red, go over the above steps.

More information can be found in the RadioBOSS Help System. Press **Ctrl+F1** inside the RadioBOSS application, or in the top menu bar go to **Help** -> **Contents**.

The following articles in there are helpful:

- **Operation** -> **Broadcasting Internet radio**
- **Options** -> **Broadcast**
- **Options** -> **Broadcast** -> **Metadata**

PlayIt Live

Windows; Proprietary, Free

[PlayIt Live](#) is a free live-assist and automated radio playout system from PlayIt Software.

It allows streaming via **Icecast**, **SHOUTcast v1** and **SHOUTcast v2** via a plugin, that you can download for free [here](#), after creating an account on their website. This plugin also allows you to record the stream, in MP3.

The installation steps for the plugin are relatively simple, so they’re not covered in this guide.

Note: *This software is only capable of streaming via MP3, due to plugin limitations.*

Instructions

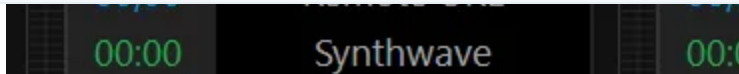
These instructions are valid and required for both streaming modes (Icecast and SHOUTcast), so be sure to follow this part before proceeding any further.

1. After installing the Internet Broadcast plugin, select the menu **Plugins** menu, on the top menu bar and select the **Plugin Manager...** option.



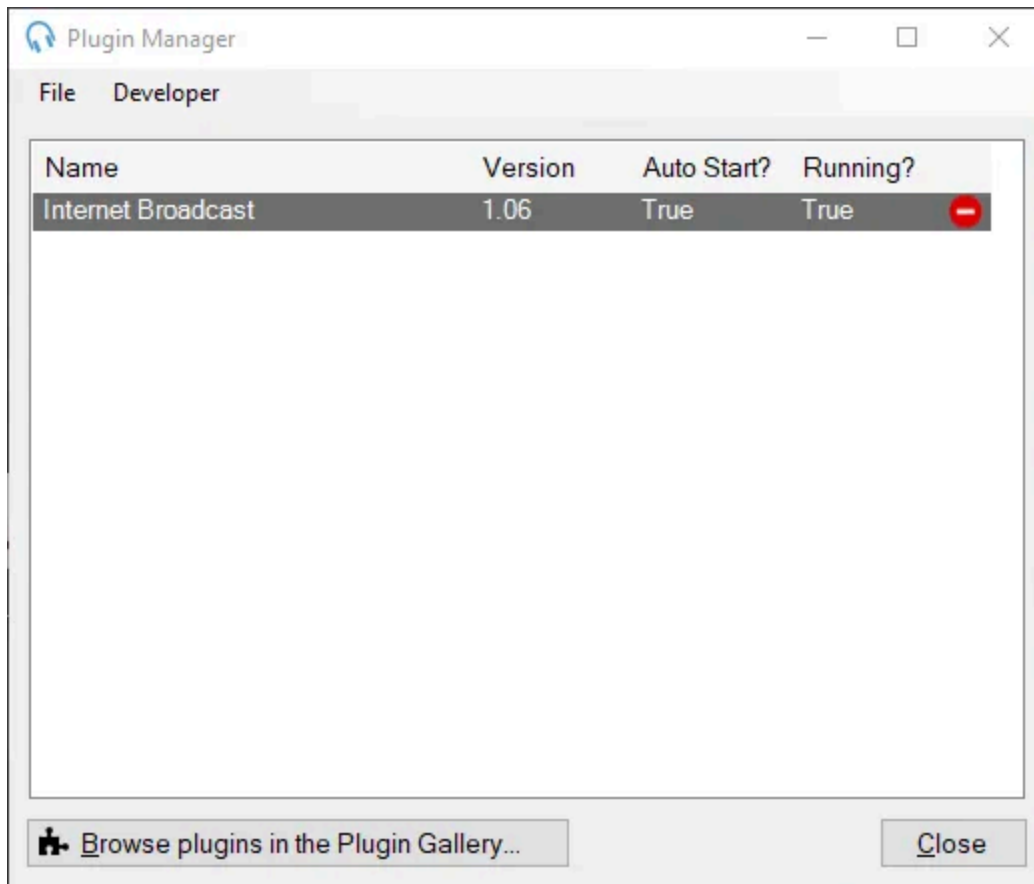
On this page >

Overview



Select the *Plugins* menu, then the *Plugin Manager* option

2. A new window should appear, and within it the Internet Broadcast plugin.



The *Plugin Manager* window

3. Open the plugin settings, by double clicking on it's name. This window should appear.



On this page >

Overview

Streams

+ Add Edit Delete

Encoder	Status	Description
---------	--------	-------------

Now Playing: ☐ Override

☐ Auto start streams

Plugin Settings

There are a few settings that you can customize here:

- In **Audio source to broadcast** you can define which input is going to be transmitted to the server;
- **Auto start streams** let's you start streaming after opening the software.

4. Under **Streams**, click **Add** to add a new streaming server.

Now follow the specific instructions for your streaming server: **Icecast** or **SHOUTcast**.

Icecast

1. Using the connection details, available in your AzuraCast instance, under the **Streamers/DJ Accounts** page,



On this page >

Overview

Server Address:

Server Port:

Username: if unsure, leave blank

Password:

Mount: e.g. /live

SID: if unsure, use 1

Audio

Format:

Bit rate (Quality):

Station

Station Name:

Description:

Genre:

URL:

Audio Archive

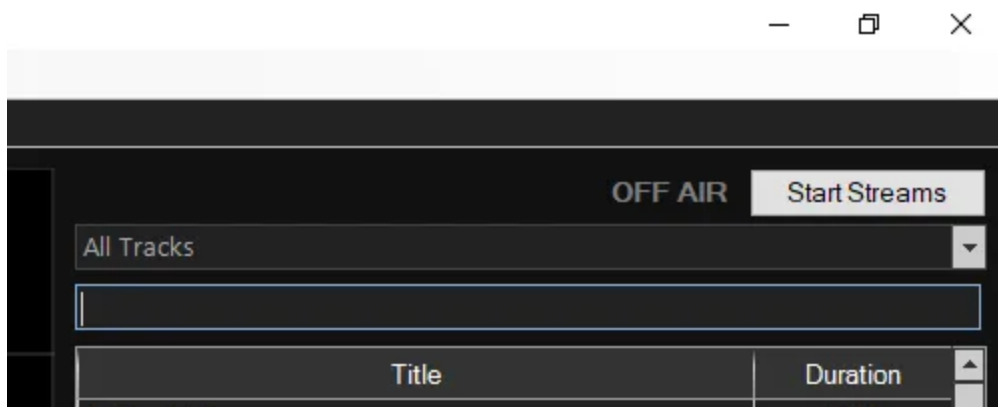
☐ Save audio to file:

☒ Enable title streaming

☐ Disabled

Connection Information for Icecast

... fill this form in the PlayIt Live software.





On this page >

Overview

NOTES.

- The **Save audio to file** field can be left unchecked. Check it if you want to record the stream into a MP3 file.
- Under **Audio**, select the audio bitrate and the format (stereo or mono) that matches your station.

2. After filling the form fields, click **OK** to save the server details.

3. To start broadcasting, click on Start Streams here on in the main interface of the software.

ShoutCast v1 Clients

Server:
demo.azuracast.com

Port:
8005

Password:
dj_username:dj_password
(DJ username and password separated by a colon)

If the server is configured correctly, the **OFF AIR** text will change to **ON AIR**. If not, re-check your configuration.

SHOUTcast

1. Using the connection details, available in your AzuraCast instance, under the **Streamers/DJ Accounts** page, ...



On this page >

Overview

PORT:

8005

Password:

dj_username:dj_password

(DJ username and password separated by a colon)

... fill this form in the PlayIt Live software.

Configure Encoder Plugin

Server

Server Type: SHOUTcast V1

Server Address: demo.azuracast.com

Server Port: 8005

Username: if unsure, leave blank

Password: *****

Mount: e.g. /live

SID: 1 if unsure, use 1

Audio

Format: Mp3

Bit rate (Quality): 128 kbps Stereo

Station

Station Name: Demo Station

Description: Some station...

Genre: Music

URL: https://azuracast.com

Audio Archive

☐ Save audio to file: Browse...

☒ Enable title streaming

☐ Disabled

OK Cancel

Using the details above to fill the form



On this page >

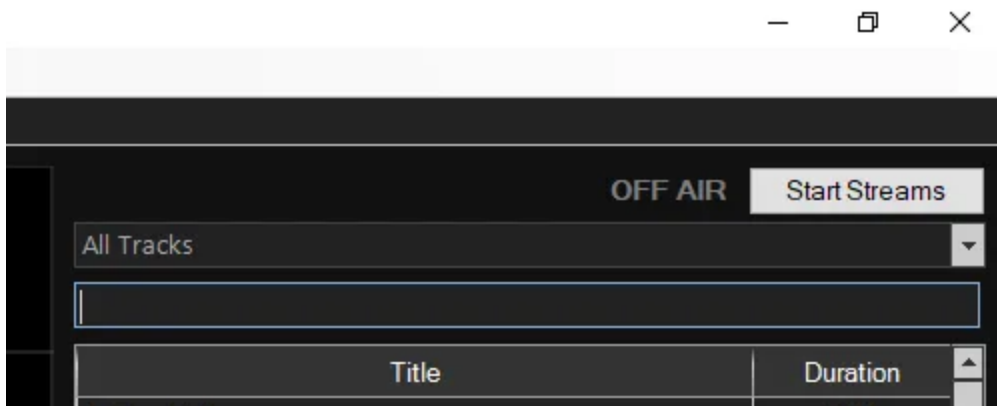
Overview

• In **Server type**, select Shoutcast v1.

- The **Password** field must be in this format: `dj_username:dj_password`
- The **Save audio to file** field can be left unchecked. Check it if you want to record the stream into a MP3 file.
- Under **Audio**, select the audio bitrate and the format (stereo or mono) that matches your station.

2. After filling the form fields, click **OK** to save the server details.

3. To start broadcasting, click on Start Streams here on in the main interface of the software.



If the server is configured correctly, the **OFF AIR** text will change to **ON AIR**. If not, re-check your configuration.

Edit page

← Previous
Streamers & DJs

Next →
Advanced Features

Streaming Software

Caution

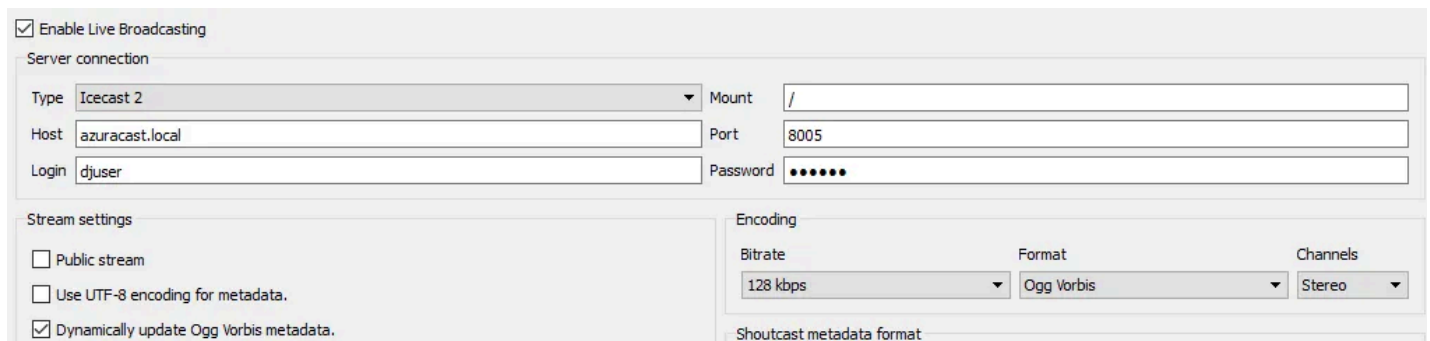
If you are using a reverse proxy service (like CloudFlare) in front of your installation, be aware that these services often prevent direct connections to the ports that broadcasters connect to. You should instruct your streamers to use the direct IP address of your server instead.

Mixxx

Linux, MacOS, Windows; Free and Open-Source

[Mixxx](#) is free and open-source DJ software that also supports live broadcasting via Icecast and ShoutCast servers.

Live broadcasting settings can be found in the preferences menu ([Options](#) -> [Preferences](#)). It is recommended to use "Icecast 2" mode when broadcasting to AzuraCast. Out of the box, Mixxx does not include the LAME MP3 encoder (for licensing reasons), but it is not needed when connecting to AzuraCast; just set your broadcast format to 'Ogg Vorbis' and LiquidSoap will transcode the broadcast into the correct output format.



The screenshot shows the 'Preferences' dialog box in Mixxx, specifically the 'Live Broadcasting' section. The 'Enable Live Broadcasting' checkbox is checked. Under 'Server connection', the 'Type' is set to 'Icecast 2', 'Mount' is '/', 'Host' is 'azuracast.local', 'Port' is '8005', 'Login' is 'djuser', and 'Password' is masked with dots. Under 'Stream settings', 'Public stream' and 'Use UTF-8 encoding for metadata' are unchecked, while 'Dynamically update Ogg Vorbis metadata' is checked. Under 'Encoding', 'Bitrate' is '128 kbps', 'Format' is 'Ogg Vorbis', and 'Channels' is 'Stereo'. There is also a 'Shoutcast metadata format' field.



On this page



Overview

Cross-platform and available for free at <https://sourceforge.net/projects/butt/>. Streaming via both Icecast mode and ShoutCast mode are available.

Icecast Mode

Use the exact information specified on the Streamer/DJ page's right sidebar. "Address" is "Server" and "Icecast mountpoint" is "Mount Name".

The screenshot shows the AzuraCast interface. On the left, the 'Streamer/DJ Accounts' table has columns for ACTIONS, USERNAME, PASSWORD, and NOTES. It contains one entry with USERNAME 'djuser' and PASSWORD 'djpass'. An 'Edit Server' dialog box is open in the center, with fields for Name (AzuraCast Local), Type (radio buttons for ShoutCast and IceCast, with IceCast selected), Address (azuracast.local), Port (8005), Password (djpass), IceCast mountpoint (/), and IceCast user (djuser). On the right, the 'Connection Information' sidebar shows 'IceCast Clients' with Server: azuracast.local, Port: 8005, and Mount Name: /. Below that, 'ShoutCast v1 Clients' shows Server: azuracast.local, Port: 8006, and Password: dj_username:dj_password (with a note that DJ username and password are separated by a colon).

ShoutCast Mode

Use the given port specified in the "ShoutCast v1 Clients" section. Since ShoutCast v1 does not let you specify a username, specify your password as `dj_username:dj_password` (the username and password for the DJ separated by a colon).



On this page



Overview

SAM Cast

Windows; Proprietary, Commercial

[SAM Cast](#) is commercial software sold by Spacial. It is compatible with AzuraCast by using the MP3 encoder and the "Icecast 2" server connection settings.

Broadcasting Steps

From the main SAM Cast window, click "Encoders" at the bottom left.



On this page



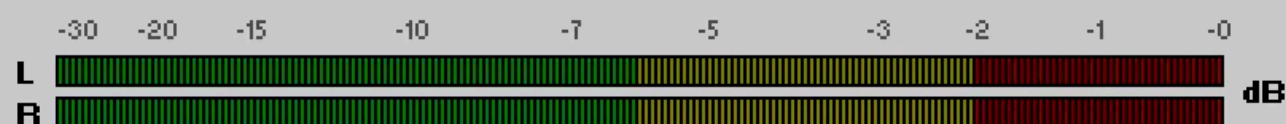
Overview

Stream Time: 0d, 00:00:00

Current Listeners: 0

Current Time: 1:10:56 PM

Peak Listeners: 0

**START**

Event log

Clear Log

2019-06-09 12:57:24> SAM Cast configuration loaded.
2019-06-09 12:57:24> Initialization completed.

Encoders

Statistics

Sound Processing

In the Encoders window that pops up, click the plus sign button on the toolbar to add a new encoder.



On this page >

Overview

The interface shows a menu bar with a 'Menu' dropdown and several icons: a plus sign (+), a crossed hammer and wrench, a minus sign (-), a play button, a stop button, and a document icon. A hand cursor is clicking the plus sign icon. Below the menu bar is a table with the following columns: #, Format, Status, and Description. The table is currently empty. Below the table is a volume meter for L and R channels, with a scale from -30 to -0 dB. The meter shows green bars for the L channel and yellow bars for the R channel.

You will be asked to select an encoder. It is recommended to select `MP3 (LAME_ENC)`, because this allows for the easiest constant bit rate (CBR) streaming, which causes the fewest issues when broadcasting.

Click "OK" at the bottom to continue to encoder details.



On this page



Overview

- ☐ aacPlus v2
- ☒ MP3 (LAME_ENC)
- ☐ WMA (v9 advanced) [ROBUST]
- ☐ WMA (v9 advanced)
- ☐ Ogg (Vorbis)
- ☐ CheapestStream.com Encoder
- ☐ mp3PRO

No details available

OK

Cancel

On the first tab of the encoder details, change your bitrate from the default 64kbps to your desired bitrate. This should likely match the highest bitrate that you broadcast out to your listeners. By default, it should at least be 128kbps.



On this page



Overview

Bitrate	128	kbps	☐ Auto start encoder after
Mode	Stereo		5 seconds
Sampling rate	Auto	Hz	☒ Auto reconnect encoder after
			15 seconds
			☐ Allow scripts in stream

OK Cancel

Click the "Server Details" tab. On the "Server Type" selector, choose "Icecast", then choose "Icecast 2" in the secondary selector.

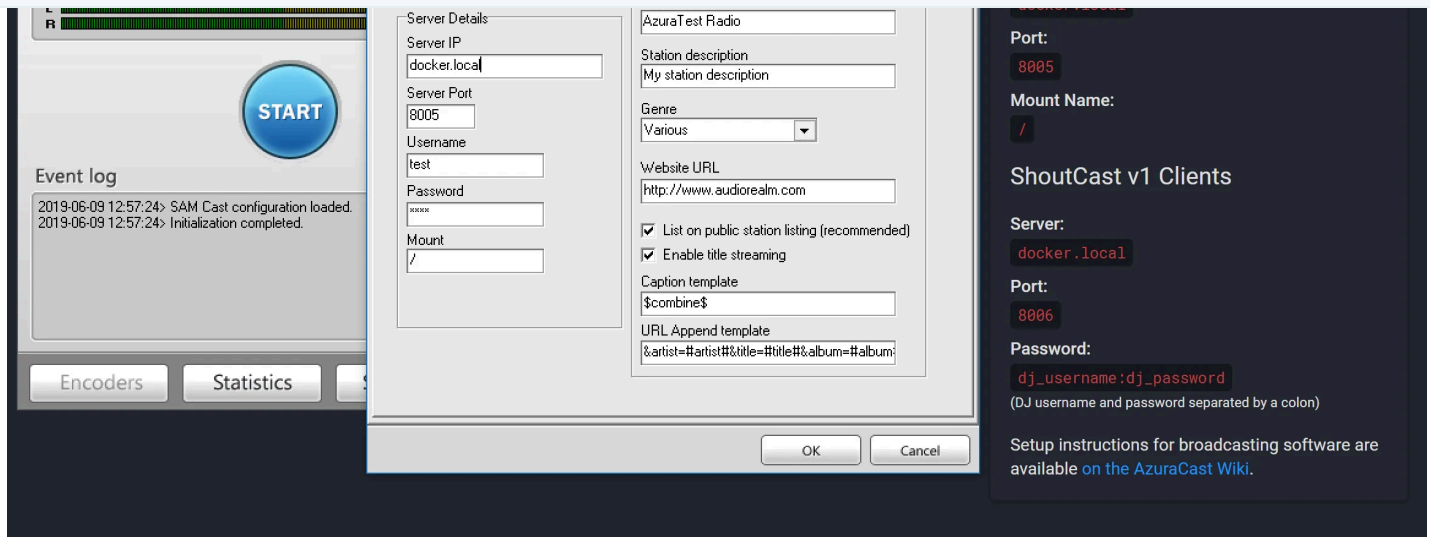
You can now populate the "Server Details" section with the connection information supplied to you via AzuraCast's "Streamers/DJs" page. You will want to use the connection information for "Icecast Clients", the first section on the page.



On this page



Overview



Click OK to return to the main Encoders pane, where you will now see a new encoder with “Idle” status.

Select the encoder, then click the play-button icon in the toolbar to begin broadcasting.



On this page >

Overview

#	Format	Status	Description
1	MP3: 128kb/s (CBR), Auto...	Idle	Encoder not started

Level meter (dB): -30, -20, -15, -10, -7, -5, -3, -2, -1, -0. L and R channels are shown with green and red bars.

RadioBOSS

Windows; Proprietary, Commercial

[RadioBOSS](#) is a commercial software sold by IP Kuzmitski D.V. at DJSoft.net.

Streaming via both Icecast mode and ShoutCast mode are available.

The Live broadcasting feature is possible in their Standard and Advanced edition (not in the Express edition).

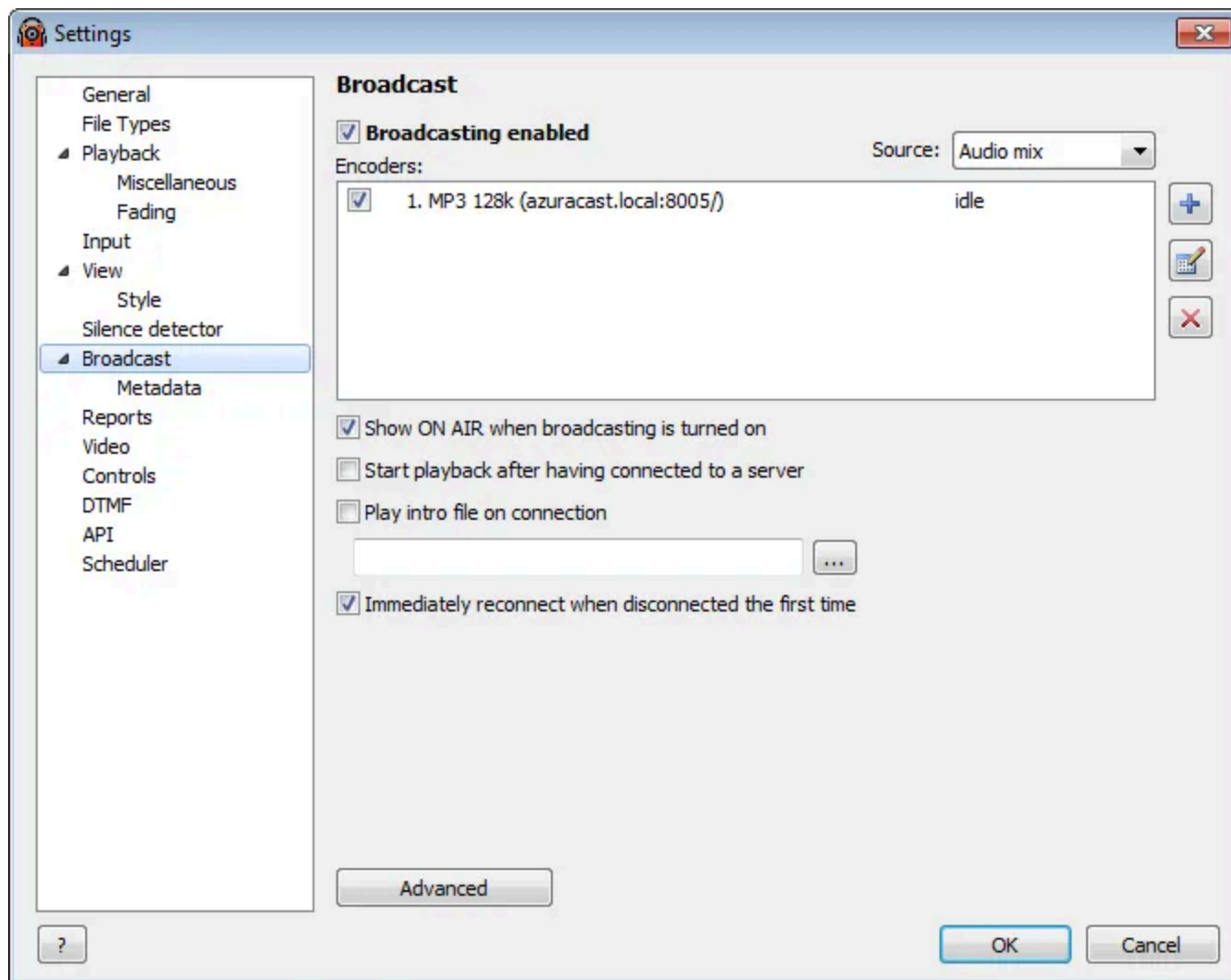
Note that for ease of use we use the [Configuration Wizard](#), you can also enter all connection details manually.



On this page > Overview

2. In the **Settings** window, select **Broadcast** on the left

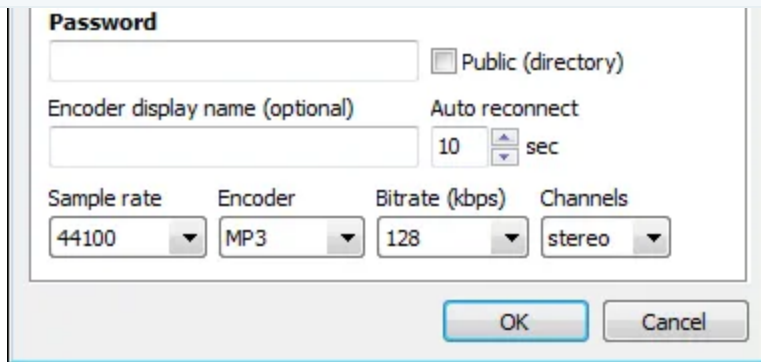
3. To add a new encoder, click the **+** button



4. The Encoder-window will open, next click the button called **Configuration Wizard**.



On this page > Overview

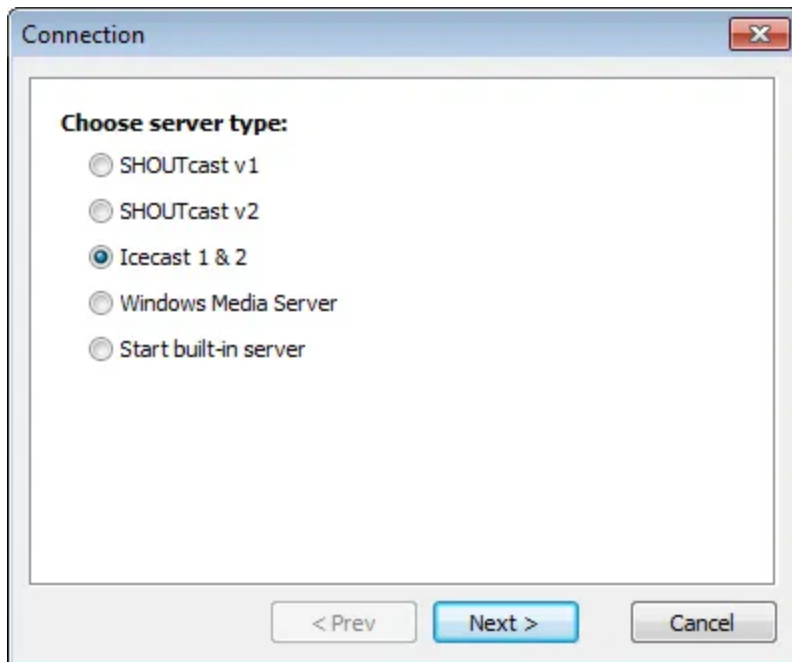


A configuration dialog box for AzuraCast. It has a title bar with a close button. The content includes a 'Password' field, a 'Public (directory)' checkbox, an 'Encoder display name (optional)' field, an 'Auto reconnect' spinner set to 10 seconds, and a section for audio settings with dropdowns for Sample rate (44100), Encoder (MP3), Bitrate (128), and Channels (stereo). At the bottom are 'OK' and 'Cancel' buttons.

Follow the instructions for the server type you have set up your Azuracast station with:

Icecast

- Select **Icecast 1 & 2** and click **Next >**



A 'Connection' dialog box with a title bar and a close button. It contains a section titled 'Choose server type:' with five radio button options: SHOUTcast v1, SHOUTcast v2, Icecast 1 & 2 (which is selected), Windows Media Server, and Start built-in server. At the bottom are '< Prev', 'Next >', and 'Cancel' buttons.

- Take over the corresponding credentials you can find in Azuracast:



On this page >

Overview

8005

Username (optional)
djuser

Password
••••••

Mount point
/

< Prev Finish Cancel

- Click Finish

SHOUTcast

The following settings are recommended for both ShoutCast v1 as v2 (DNAS 2)

Note 1: Use the given port specified in the "ShoutCast v1 Clients" section.

Note 2: Be sure to select v1, as v2 will append the Stream ID (SID), which won't work.

- Select SHOUTcast v1 and click Next >

Connection

Choose server type:

☒ SHOUTcast v1

☐ SHOUTcast v2

☐ Icecast 1 & 2

☐ Windows Media Server

☐ Start built-in server

< Prev Next > Cancel



On this page > Overview

A screenshot of the AzuraCast configuration window. It has a light gray border and a white background. At the top, there's a text input field containing 'azuracast.local'. Below it, the word 'Port' is in bold, followed by a spin box showing '8005'. Then, 'Username (optional)' is in bold, followed by a text input field containing 'djuser'. Below that, 'Password' is in bold, followed by a password input field with seven dots. At the bottom, there are three buttons: '< Prev' (disabled), 'Finish' (highlighted in blue), and 'Cancel' (disabled).

- Click **Finish**

6. Finish by changing the rest of the settings by preference:

- Connection
 - Sample Rate
 - Encoder
 - Bitrate
 - Channels
 - Station info

If needed, here you can override the info that can be found in the main **Settings** (From step 1), under **Broadcasting** -> **Metadata**

1. Then finally click **OK** to save the Encoder.

2. In the Settings window, check the checkboxes for all encoders that you want to use. Remember to turn on the option **Broadcasting enabled** on top.



On this page > Overview

If the logs show “[E40] Cannot start broadcast” in red, go over the above steps.

More information can be found in the RadioBOSS Help System. Press **Ctrl+F1** inside the RadioBOSS application, or in the top menu bar go to **Help** -> **Contents**.

The following articles in there are helpful:

- **Operation** -> **Broadcasting Internet radio**
- **Options** -> **Broadcast**
- **Options** -> **Broadcast** -> **Metadata**

PlayIt Live

Windows; Proprietary, Free

[PlayIt Live](#) is a free live-assist and automated radio playout system from PlayIt Software.

It allows streaming via **Icecast**, **SHOUTcast v1** and **SHOUTcast v2** via a plugin, that you can download for free [here](#), after creating an account on their website. This plugin also allows you to record the stream, in MP3.

The installation steps for the plugin are relatively simple, so they’re not covered in this guide.

Note: *This software is only capable of streaming via MP3, due to plugin limitations.*

Instructions

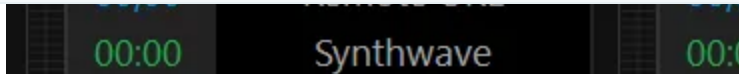
These instructions are valid and required for both streaming modes (Icecast and SHOUTcast), so be sure to follow this part before proceeding any further.

1. After installing the Internet Broadcast plugin, select the menu **Plugins** menu, on the top menu bar and select the **Plugin Manager...** option.



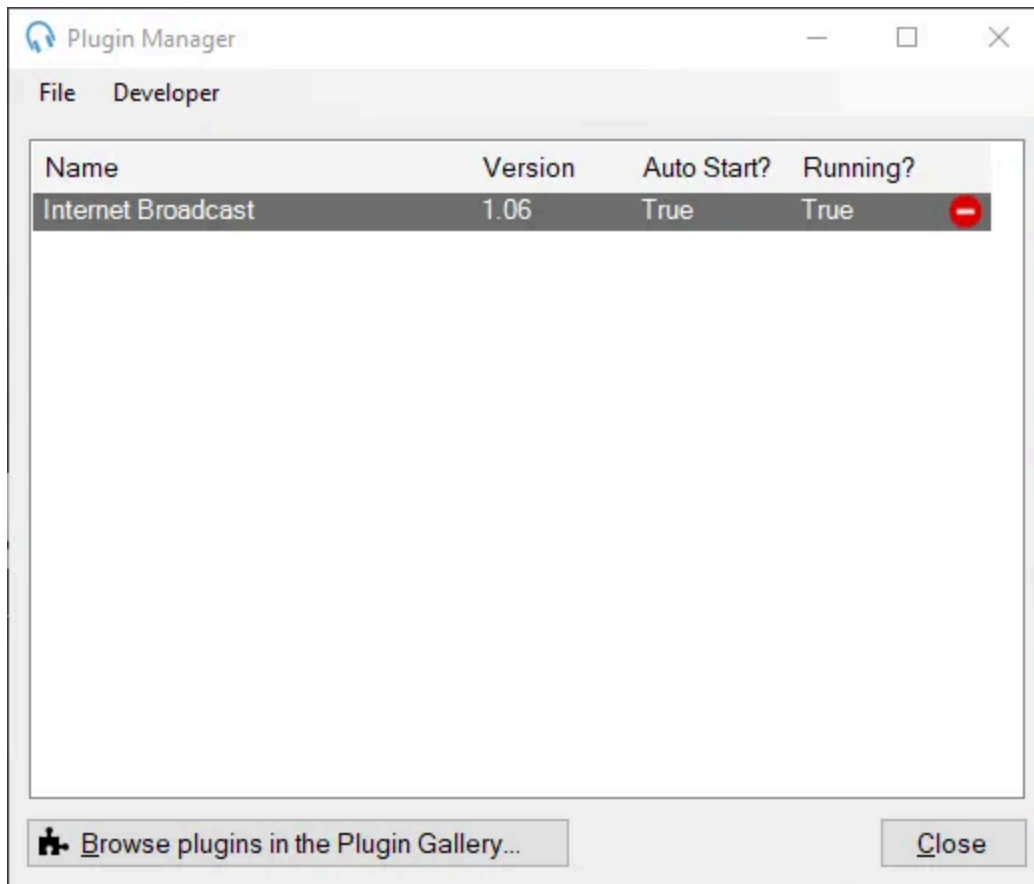
On this page >

Overview



Select the *Plugins* menu, then the *Plugin Manager* option

2. A new window should appear, and within it the Internet Broadcast plugin.



The *Plugin Manager* window

3. Open the plugin settings, by double clicking on it's name. This window should appear.



On this page >

Overview

Plugin Settings

There are a few settings that you can customize here:

- In **Audio source to broadcast** you can define which input is going to be transmitted to the server;
- **Auto start streams** let's you start streaming after opening the software.

4. Under **Streams**, click **Add** to add a new streaming server.

Now follow the specific instructions for your streaming server: **Icecast** or **SHOUTcast**.

Icecast

1. Using the connection details, available in your AzuraCast instance, under the **Streamers/DJ Accounts** page,



On this page >

Overview

Server Address:

Server Port:

Username: if unsure, leave blank

Password:

Mount: e.g. /live

SID: if unsure, use 1

Audio

Format:

Bit rate (Quality):

Station

Station Name:

Description:

Genre:

URL:

Audio Archive

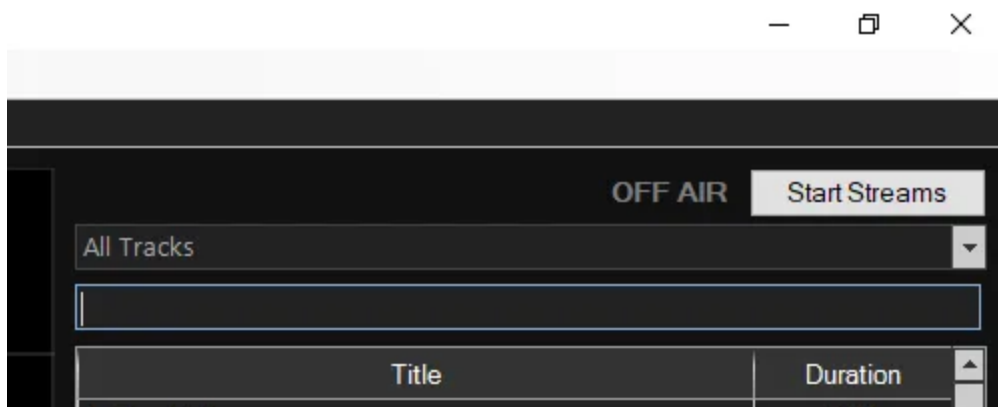
☐ Save audio to file:

☒ Enable title streaming

☐ Disabled

Connection Information for Icecast

... fill this form in the PlayIt Live software.





On this page



Overview

NOTES.

- The **Save audio to file** field can be left unchecked. Check it if you want to record the stream into a MP3 file.
- Under **Audio**, select the audio bitrate and the format (stereo or mono) that matches your station.

2. After filling the form fields, click **OK** to save the server details.

3. To start broadcasting, click on Start Streams here on in the main interface of the software.

The screenshot shows a configuration window titled "ShoutCast v1 Clients". It contains three input fields: "Server:" with the value "demo.azuracast.com", "Port:" with the value "8005", and "Password:" with the value "dj_username:dj_password". Below the password field, there is a note in parentheses: "(DJ username and password separated by a colon)".

If the server is configured correctly, the **OFF AIR** text will change to **ON AIR**. If not, re-check your configuration.

SHOUTcast

1. Using the connection details, available in your AzuraCast instance, under the **Streamers/DJ Accounts** page, ...



On this page >

Overview

PORT:

8005

Password:

dj_username:dj_password

(DJ username and password separated by a colon)

... fill this form in the PlayIt Live software.

Configure Encoder Plugin

Server

Server Type: SHOUTcast V1

Server Address: demo.azuracast.com

Server Port: 8005

Username: if unsure, leave blank

Password: *****

Mount: e.g. /live

SID: 1 if unsure, use 1

Audio

Format: Mp3

Bit rate (Quality): 128 kbps Stereo

Station

Station Name: Demo Station

Description: Some station...

Genre: Music

URL: https://azuracast.com

Audio Archive

☐ Save audio to file: Browse...

☒ Enable title streaming

☐ Disabled

OK Cancel

Using the details above to fill the form



On this page >

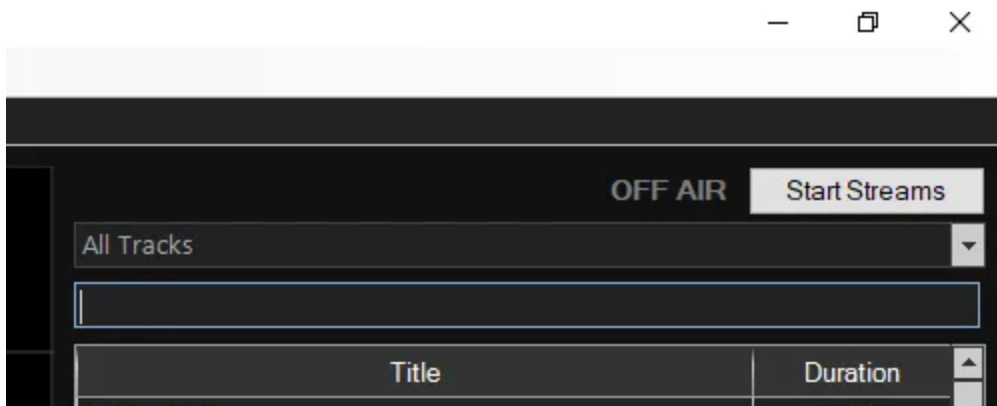
Overview

• In **Server type**, select Shoutcast v1.

- The **Password** field must be in this format: `dj_username:dj_password`
- The **Save audio to file** field can be left unchecked. Check it if you want to record the stream into a MP3 file.
- Under **Audio**, select the audio bitrate and the format (stereo or mono) that matches your station.

2. After filling the form fields, click **OK** to save the server details.

3. To start broadcasting, click on Start Streams here on in the main interface of the software.



If the server is configured correctly, the **OFF AIR** text will change to **ON AIR**. If not, re-check your configuration.

Edit page

← Previous
Streamers & DJs

Next →
Advanced Features

SFTP Server



Tip

The Built-in SFTP server is only available on the Docker installation.

All AzuraCast Docker installations come with a built-in SFTP server that allows you to easily connect to your media directory, upload and move files using software of your choice.

Creating SFTP Users



Tip

You will only be able to connect to the SFTP server to manage your media if your station's storage location is a "Local" one to the server. If you're connecting to S3, Dropbox, etc. then you can instead use clients that can connect directly to the same storage location your station is using.

Under the **Media** sidebar menu for a given station, you will see an entry labeled **SFTP Users**. Click this to view existing users and the connection details for connecting to your server via an SFTP client of your choice.

Default Port

By default, the SFTP service listens externally on port 2022. If there is something else using this port or you want to change it for any other reason, you can edit the **.env** file on your host machine and add or modify this line:



On this page >

Note, this is a file named `.env`, with no filename, and not the `azuracast.env` file

Connecting to SFTP

SFTP clients

Here's a small list of FTP clients you can use that are known to work with AzuraCast's SFTP system.

- [Filezilla](#) - Windows, MacOS and Linux 32 bit & 64 bit
- [WinSCP](#) - Windows only
- [Cyberduck](#) - Windows and MacOS

A more exhaustive list can be found [here](#).

Mounting as a Directory on Your Computer

There are also numerous tools that let you create a virtual filesystem based on an SFTP connection, which then lets you drag and drop files to and from the filesystem using your OS's standard file explorer:

- [Mountain Duck](#) - Windows and MacOS
- [ExpanDrive](#) - Windows, MacOS and Linux
- [SSHFS for Linux/MacOS](#)
- [SSHFS for Windows](#)

For more information on configuring SSHFS, see [this thread](#).

Edit page



On this page >

Synchronized Tasks

AzuraCast has some background tasks that are regularly triggered to run internal processing jobs that are necessary for the operation of your stations.

On this page you will find an overview of all the `Sync Tasks` that AzuraCast depends on.

Increasing Execution Timeout

If you are regularly running into timeouts while a sync task is processing you can manually increase the time that the sync tasks are allowed to run via the `azuracast.env` in `/var/azuracast/`.

In the `azuracast.env` look for the following entries and increase the specified time in seconds to your needs.

```
# The maximum execution time (and lock timeout) for the 15-second, 1-minute and 5-minute s
# SYNC_SHORT_EXECUTION_TIME=600
```

```
# The maximum execution time (and lock timeout) for the 1-hour synchronization task.
# SYNC_LONG_EXECUTION_TIME=1800
```

This is how these lines should look like after changing (the values are just for demonstration):

```
# The maximum execution time (and lock timeout) for the 15-second, 1-minute and 5-minute s
SYNC_SHORT_EXECUTION_TIME=900
```

```
# The maximum execution time (and lock timeout) for the 1-hour synchronization task.
SYNC_LONG_EXECUTION_TIME=2200
```




On this page >

[docker-compose down](#)
[docker-compose up -d](#)

Synchronized Tasks

ReactivateStreamer

When manually disconnecting a live DJ you can let AzuraCast disable their account for a set amount of time. This task is responsible for re-activating the DJ accounts after the time has passed.

CheckRequests

This task is responsible for sending user requested songs to the Liquidsoap AutoDJ to play.

RunBackup

This task is responsible for checking if a scheduled backup needs to be run.

CleanupRelays

This task is responsible for the automatic cleanup of relay data.

CheckMedia

This task is responsible for scanning the media storage location for newly added or modified media files that AzuraCast needs to process.

CheckFolderPlaylists

This task is responsible for automatically assigning the media of a folder to the playlist that the folder is assigned to.

[On this page >](#)

RunAnalytics

This task is responsible for updating daily listener data and analytics that can take time to calculate.

CleanupHistory

This task is responsible for cleaning up old listener and song history data according to the system settings.

CleanupStorage

This task is responsible for clearing the temporary data directory as well as checking the media storage locations for files that need to be removed.

RotateLogs

This task is responsible for rotating logs and backup of the AzuraCast installation.

UpdateGeoLite

This task is responsible for automatically updating your GeoLite database for IP-address geolocation if you have supplied an API key.

[✎ Edit page](#)

← Previous
Storage Locations

Next →
Users

Promoting Your Station

There are many directories around the Web where users can find new radio stations to listen to. This page documents how to integrate your AzuraCast station with these directories.

Webhook-Based Directories

These directories support any broadcast technology and receive “Now Playing” metadata updates via a custom API, which we implement in AzuraCast via our Web Hooks feature.

TuneIn AIR

AzuraCast supports sending automatic metadata updates to TuneIn’s [AIR broadcasting API](#) via a Web Hook.

You will first need a station ID from TuneIn. Contact them directly for more information about signing up for an API account.

Once you have a station ID, you can create a new web hook via the **Web Hooks** station sidebar menu inside AzuraCast.

Radio.de

AzuraCast supports sending automatic metadata updates to [Radio.de’s Metadata Push API](#) as of Version 0.19.1.

You should first contact the Radio.de team directly to gain access to the API. Once you have the necessary credentials, you can create a new web hook via the **Web Hooks** station sidebar inside AzuraCast.



On this page >

Contact the GetMixerAudio team first to gain access to their API. Once you have the necessary credentials, you can create a new web hook via the **Web Hooks** station sidebar inside AzuraCast.

YellowPages Directories

These directories use Icecast's built-in YellowPages (YP) discovery mechanism to submit routine updates. They require using Icecast to broadcast rather than Shoutcast or HLS.



Tip

Adding YellowPages directories requires that the "Enable Advanced Settings" system setting is enabled in AzuraCast. An installation administrator can enable this setting at any time from the web interface.



Tip

By default, all mount points on a station will be advertised to any Yellow Pages directory. To limit which mount points are visible, you can edit a mount point and uncheck the **Publish to "Yellow Pages" Directories** setting.

Xiph Icecast Directory

Xiph, the maintainers of Icecast, also maintain a public directory of stations at dir.xiph.org.

To add your station to the directory, edit your station's profile, then under the "Broadcasting" tab, you will find "Custom Configuration" at the bottom of the page.

You can add the following entry to broadcast to the Xiph directory:

```
<directory>
  <yp-url-timeout>15</yp-url-timeout>
  <yp-url>http://dir.xiph.org/cgi-bin/yp-cgi</yp-url>
```



On this page >

Tip

Listing on this directory requires that your station description and other profile fields be 255 characters or fewer.

The station directory at internet-radio.com supports and recommends using Icecast's YellowPages directory to manage your station listing on their web site. Their forum includes [specific instructions for Icecast](#).

To integrate the custom directory code into AzuraCast, edit your station's profile, then under the "Broadcasting" tab, you will find "Custom Configuration" at the bottom of the page.

You can add the following entry:

```
<directory>
  <yp-url-timeout>15</yp-url-timeout>
  <yp-url>http://icecast-yp.internet-radio.com</yp-url>
</directory>
```

Icecast Header Directories

These directories receive metadata updates via custom headers provided directly in your Icecast stream. They require using Icecast to broadcast rather than Shoutcast or HLS.

Tip

Adding custom Icecast headers requires that the "Enable Advanced Settings" system setting is enabled in AzuraCast. An installation administrator can enable this setting at any time from the web interface.

radio-browser.info

[On this page](#) >

mount point:

```
<http-headers>
  <header name="icy-index-metadata" value="1" />
  <header name="icy-logo" value="https://example.com/besticon.png" />
  <header name="icy-country-code" value="at" />
  <header name="icy-country-subdivision-code" value="at-1" />
  <header name="icy-language-codes" value="en,de" />
  <header name="icy-main-stream-url" value="http://example.com:8000/teststream" />
  <header name="icy-geo-lat-long" value="12.345,-23.456" />
</http-headers>
```

⚠ Power users only!

HTTP-headers must be added to each individual mount point. Adding them to the station-wide Custom Configuration (under [Profile](#) -> [Edit Profile](#) -> [Broadcasting](#)) will not work.

[✎ Edit page](#)

← Previous
Playlists

Next
Remote Relays →

Update AzuraCast

Updating AzuraCast will update both the web app itself and all of its dependencies, so you will be on the latest version of all of the supporting software.

During the update process, your stations will be briefly offline to listeners, so you should set aside a time to update and notify listeners if necessary.

Always back up first!

The AzuraCast team is not responsible for any downtime or loss of data of your production installations of AzuraCast if you run into any errors while updating. Especially if you are updating a production installation with a lot of stations on a single server.

For Docker Installations

Before Updating: Update Docker & Docker Compose

If it's been quite some time since you last updated, your version of Docker and Docker Compose may be out of date, and updates may trigger errors as a result.

You can update Docker and Docker Compose using the utility script by running:

```
cd /var/azuracast
./docker.sh update-self
./docker.sh install-docker
./docker.sh install-docker-compose
```



On this page >

```
cd /var/azuracast
./docker.sh update-self
./docker.sh update
```

Automated Updates

By default, the updater will prompt you to update your `docker-compose.yml` file. If you aren't making any changes to this file and want to automate the update process, you can use the command below to automatically answer "yes" to this question:

```
cd /var/azuracast
./docker.sh update-self
yes "" | ./docker.sh update
```

For Ansible Installations

AzuraCast also includes a handy updater script for Ansible installations that pulls down the latest copy of the codebase from Git, flushes the site caches and makes any necessary database updates. Run these commands as any user with `sudo` permissions:

```
cd /var/azuracast/www

sudo chmod a+x update.sh
sudo ./update.sh
```

Force a Full Update



On this page >



```
./update.sh --full
```

Switching Update Release Channels

AzuraCast ships two different release channels ("Stable" and "Rolling Release"), which you can switch between per installation. For more information, see our [Release Channels](#) page.

 Edit page

← Previous
Install AzuraCast

Next
Logs →

Playlists

Required Permissions

To manage playlists, users must be in a role that has the “Manage Station Media” permission for the given station.

Playlists are collections of media that are played by the AutoDJ according to certain rules. For example, you can have one or more playlists that are considered “General Rotation” and will play throughout the day, or playlists that only play every few songs, or every few minutes.

Any playlists types can also be scheduled to play in one or more specific time blocks, or to play between a certain date range.

Playlist Priorities

In newer versions of AzuraCast (released after September 1, 2024), you can specify the priority of individual playlists. If two playlists are both eligible to be played at the same time, the higher priority playlist will always be selected. This allows you to “stack” playlists in the same schedule slot, letting the priority system determine the actual playback order. You can also specify the priority of incoming listener requests (if they’re enabled) to prevent them from always taking priority over general playlists.

Default Priorities

If you don’t manually specify the priority of playlists, or if you’re on an earlier version of AzuraCast, playlists follow this priority system by default:

[On this page >](#)

Once Per X Hour Playlists, Scheduled	7
Once Per X Hour Playlists, Unscheduled	6
Once Per X Songs Playlists, Scheduled	5
Once Per X Songs Playlists, Unscheduled	4
Once Per X Minutes Playlists, Scheduled	3
Once Per X Minutes Playlists, Unscheduled	2
General Rotation Playlists	0

Advanced Playlists

What are Advanced Playlists?

If you create a new playlist in AzuraCast, you will notice one possible option for the playlist type called “Advanced Playlist”. This article will explain what an advanced playlist is, and what benefits it can offer to power users.

To summarize, **an advanced playlist is a feature intended for “power users” who want to write the entire configuration for their playlist on Liquidsoap by hand, but still take advantage of AzuraCast for managing the contents of the playlist.**

If you use an advanced playlist, AzuraCast will still manage what songs are on the playlist, and let you add/remove songs via the web interface, but it won’t try to automatically insert the corresponding configuration into Liquidsoap for you, like it would with any of the other playlist types.

This allows you to use far more complex custom rules for when a playlist plays than AzuraCast’s web interface would otherwise allow. The example below demonstrates one use case for this feature (and is, in fact, the exact reason it exists in the software).



On this page >

to force that songs or music clips from two playlists always play right next to each other. This is possible, however, with more advanced Liquidsoap configuration. Thus, the “Advanced Playlist” was born.

Let’s say you create two playlists named “Commercials” and “Bumpers”. AzuraCast will automatically insert variables for these playlists inside the station’s `liquidsoap.liq` file, with the names being prefixed by “playlist_”, with all characters converted to lower case and spaces replaced with underscores. For example, the corresponding variable names in this case are “playlist_commercials” and “playlist_bumpers”, respectively.

When you’re using Liquidsoap as your AutoDJ software, on a station’s primary management page under “Broadcasting” in the sidebar you will see a section named “Edit Liquidsoap Configuration”. This page allows you to input raw code into the `liquidsoap.liq` file that will be used when the AutoDJ runs your station.

In this case, the following rule was added to the custom configuration to produce the side-by-side playlist result:

```
radio = rotate(weights=[1,1,5], [playlist_commercials, playlist_bumpers, radio])
```

The variable `radio` is already defined in the Liquidsoap configuration, and represents the entire composed radio stream as it will broadcast out to listeners. The playlist names were defined using the rules described above. The `rotate` function is internal to Liquidsoap and follows a specific pattern of playback order, which is exactly what we want in this case.

Note: By default, playlists are considered “fallible” in Liquidsoap terminology; that is, they can contain invalid files or otherwise aren’t guaranteed to supply reliable output. More information about fallible sources is available on the [Liquidsoap documentation](https://www.liquidsoap.info/doc1/sources/). Liquidsoap always requires an “infallible” source of audio to fall back to if the fallible sources don’t produce a valid output. In the example above, the infallible source is the existing `radio` variable, but if you are completely overriding the output signal, you should make sure to have at least one “safe” playlist; that is, one playlist wrapped with the `mkSAFE` function, like:

[On this page >](#)

adding your own advanced configuration options. Feel free to experiment with advanced configuration options on a test instance of AzuraCast, because if something doesn't work quite right, you can always just remove the customization, restart the station and be back up and running.

[✎ Edit page](#)

← Previous
Mount Points

Next →
Promoting Your Station

Logs

Before submitting any GitHub issues, you should take a look at the terminal logs that AzuraCast outputs. They can often provide additional information about the error, or include very useful information that should be included in any GitHub issue you create.

Users with the appropriate permissions can also view many logs directly through AzuraCast itself. The Log Viewer feature is available under “Logs” in each station’s management page.

System Logs

If you are administrating your own AzuraCast installation you can check most logs through the

Administration -> System Logs .

In addition to the Icecast , SHOUTcast and Liquidsoap logs you can find the AzuraCast Application Log here which contains useful information about what the AzuraCast application itself is doing. Email message logs are also sent here.

Log Viewer

The Logs link in your station sidebar menu gives you access to the Logs from your station’s

Icecast , SHOUTcast and Liquidsoap processes.

In addition to the logs, you can also view the Icecast, SHOUTcast and Liquidsoap configuration scripts that AzuraCast automatically generates for you.

Log Types



On this page >

file.

Liquidsoap Log

This log file contains the log output of the Liquidsoap instance for your station. Liquidsoap is responsible for the audio transcoding in AzuraCast and interfaces with the AzuraCast application to provide the AutoDJ for your station.

In this log you can find information on how the streams are generated, what information the AutoDJ sent to Liquidsoap, errors, warnings and more.

If you encounter any problems with your stream generation this log should contain useful information in diagnosing the issue.

Icecast Access Log

This log file contains the raw access log data of the Icecast server.

For most issues this log does not contain any useful information but can be used for analytical purposes.

Icecast Error Log

This log file contains the error output of the Icecast server.

If you encounter any problems with accessing the streams or your mount points this is where you should start looking into.

Docker Container Logs

Some system logs can only be accessed from a shell session on the host computer. To view the logs for your AzuraCast installation, run:

[On this page](#) >

For Ansible Installations

Since the Ansible installation interacts directly with your host server, its logs are in various locations across the system.

- AzuraCast: `/var/azuracast/www_tmp/app.log`
- Nginx Access: `/var/azuracast/www_tmp/access.log`
- Nginx Errors: `/var/azuracast/www_tmp/error.log`
- PHP: `/var/azuracast/www_tmp/php_errors.log`
- Supervisord: `/var/azuracast/www_tmp/supervisord.log`
- Redis: `/var/log/redis/redis-server.log`
- MariaDB: `/var/log/mysql`

For each station, logs for radio software will be inside

`/var/azuracast/stations/{station_short_name}/config` , with the following filenames:

- Liquidsoap: `liquidsoap.log`
- Icecast: `icecast.log`
- SHOUTcast: `sc_serv.log`

[✎ Edit page](#)

← Previous
Update AzuraCast

Next
Optimizing →

Station Management

Required Permissions

To view a station's administration section, users must be in a role that has the "View Station Page" permission for the specified station.

Each "Station" in AzuraCast is a separate entity with its own media, playlists, DJs, mount points, remote relays and statistics.

You can create any number of stations in a single AzuraCast installation, limited only by the total resources available on the server; for example, by default, only the ports for the first 50 stations are exposed.

Each station has a main "Profile" page that offers an overview of that station's currently playing status, the status of its AutoDJ and main broadcasting software, and a list of all of its available mount points and remote relays.

If a user has the appropriate permissions, they will also see additional management options along the left sidebar of the Station Management page.

AutoDJ (Backend) vs Broadcasting Software (Frontend)

Every station has two essential components that are always running and which generate a broadcasted signal for the station:

- The **AutoDJ** (also occasionally known as the "Broadcasting Backend") is a piece of software (in AzuraCast's case, Liquidsoap) that compiles media from various sources, different playlists and

[On this page >](#)

- The **Broadcasting Software** (or the "Broadcasting Frontend") is the lightweight piece of software (either Icecast or SHOUTcast) whose primary job is to take the signal created by the AutoDJ and broadcast it out to potentially hundreds or thousands of listeners. This software is specifically optimized to support very many concurrent incoming connections with very little impact on system performance.

Part of AzuraCast's function is to ensure both of these processes are properly configured, automatically started and kept running behind the scenes. When running an AzuraCast installation, you don't have to directly start either of these services for any station, and (with the exception of some specific changes, which require a brief restart) don't need to modify them directly for configuration changes in AzuraCast to take effect.

SHOUTcast

Adding a License Key

- Visit the [Station Profile](#)
- Click [Edit Profile](#)
- Switch to the [Broadcasting](#) tab
- Put the following in the [Custom Configuration](#) section:

```
{ "licenceid": "YOUR_LICENSE_ID", "userid": "YOUR_USER_ID" }
```

[✎ Edit page](#)[← Previous
SFTP Server](#)[Next →
Streamers & DJs](#)



On this page >

AzuraRelay

AzuraRelay is “relay in a box” containing a lightweight web application and Icecast servers that can connect to and relay an AzuraRelay parent instance.

If you want to offer your streams to users around the world or accept a large volume of incoming traffic, AzuraRelay is the perfect companion to AzuraCast.

Parent Installation Requirements

Before installing AzuraRelay, make sure your “parent” AzuraCast installation:

- is updated to the latest version,
- has the direct radio ports (i.e. 8000, 8010) exposed (i.e. isn't behind a CloudFlare proxy)

System Requirements

- A 64-bit x86 (x86_64) CPU (ARM devices, like the Raspberry Pi 3/4, are not supported.)
- 512MB or greater of RAM
- 20GB or greater of hard drive space

For Linux hosts, the `sudo`, `curl` and `git` packages should be installed before installing . Most Linux distributions include these packages already.

Installing

AzuraRelay is powered by Docker and uses pre-built images that contain every component of the software. Don't worry if you aren't very familiar with Docker; our easy installer tools will handle

[On this page >](#)

Pick a base directory on your host computer that AzuraRelay can use. If you're on Linux, you can follow the steps below to use the recommended directory:

```
mkdir -p /var/azurarelay  
cd /var/azurarelay
```

Use these commands to download our Docker Utility Script, set it as executable and then run the Docker installation process:

```
curl -L https://raw.githubusercontent.com/AzuraCast/AzuraRelay/main/docker.sh > docker.sh  
chmod a+x docker.sh  
./docker.sh install
```

On-screen prompts will show you how the installation is progressing.

Updating

Using the included Docker utility script, updating is as simple as running:

```
./docker.sh update-self  
./docker.sh update
```

[✎ Edit page](#)



On this page >

Getting Started

Development on the main AzuraCast application should always be applicable to a broad community of radio station operators and not specific features to one station or group of stations.

If you would like to build a set of features specific to one station or group of stations, you should take advantage of AzuraCast's plugin architecture. The plugin system takes advantage of event listeners that are built into AzuraCast itself. Check out the [example plugin](#) for more details on what is possible via plugins.

Contributing Code Changes

AzuraCast is open-source software, and as part of this dedication to openness and transparency, we fully support contributions from members of the community who are skilled in the languages that we use to build our applications.

A majority of our repositories come with an `.editorconfig` file in the root, which will set many standards for indentation, punctuation and other style items for you automatically. You may need to enable EditorConfig support in your IDE of choice.

If your IDE does not support EditorConfig, the most important standard to remember that we follow is the PHP Framework Interoperability Group's [PSR-12 Extended Coding Style](#) standard.

Accessibility, security, and modern best practices are very important in AzuraCast's development. Any newly contributed code can, and should, take advantage of the full suite of new features made available in PHP 8.0 and newer.

Contributions are also welcome in the supporting technologies used to make AzuraCast possible, such as:

[On this page >](#)

Setting Up a Local Environment

Regardless of your host operating system, it's highly recommended to use **Docker** when developing locally, as it offers portability, convenience, and a very close approximation of how AzuraCast runs on production environments.

For the steps below, we're assuming you've created a folder where you will store all of your AzuraCast-related projects, like `/home/myuser/azuracast/`.

You will need Git and Docker installed locally. If you're on Windows or Mac, the best way to use Docker is via the native [Docker Desktop](#) applications for those platforms.

For Windows, an installer tool like [Scoop](#) is highly recommended for dependencies like Git and your terminal of choice. A third-party shell client like Cmder is also often helpful.

Clone the Core Repository

Using Git, clone the AzuraCast core repository into a subfolder of your working directory.

Note for Windows developers: Before cloning the repositories, you should ensure your Git is locally configured to not automatically convert line endings from Linux style (LF) to Windows style (CRLF), which will break AzuraCast. You can set this globally by running:

```
git config --global core.autocrlf input
```

To do this, your initial command should look like:

```
git clone https://github.com/AzuraCast/AzuraCast.git
```

Setup via Docker Utility Script



On this page >

From the previous command, you can run this script by running:

```
cd AzuraCast
bash docker.sh install-dev
# Alternatively, you can use `make install` if `make` is installed on your system
```

Modify the Environment File

During the developer setup process, if you haven't done so already, you will be prompted to customize your `azuracast.env` file to make sure the proper values are input into this file.

Many of the values left blank by default are what are called "fixtures", which will automatically populate parts of the database upon initial installation, saving you the trouble of setting those values up every time you reset your database or code.

If you want to populate those values, you can modify them in the `azuracast.env` file, where they will look like the fields below:

```
INIT_BASE_URL=http://azuracast.local
INIT_INSTANCE_NAME="local development"
INIT_ADMIN_EMAIL=
INIT_ADMIN_PASSWORD=
... etc.
```

When initializing a development instance, AzuraCast will create both a standard demo account and system admin user profile, using the values set in `azuracast.env` if provided. `INIT_DEMO_API_KEY` and `INIT_ADMIN_API_KEY` are associated with the demo and admin users respectively, and should be a valid split-token key. The default login credentials for the demo user are `demo@azuracast.com` with password `demo`. Additionally, AzuraCast will create a demo station AzuraTest Radio (`azuratest_radio`) that will use the defined `INIT_MUSIC_PATH` and `INIT_PODCASTS_PATH`.



On this page >

image and run the containers. After the script finishes (as indicated by `✓ Container azuracast Started`), you may need to wait a few more minutes for the initialization process to finish running. You can check the progress by running `docker logs -f azuracast`.

Common Tasks

Many common tasks are made simpler by the addition of a `Makefile` to our core repository. It's strongly recommended to install `make` on your host OS or inside WSL.

Rebuilding the Docker Images

By default, developer installations bind mount the current code from your local computer into the Docker container, so changes to the code will be visible immediately in the running web application.

If you need to make changes to the containers themselves, though, or your containers get out of date with the ones in our repositories, you can rebuild them by running:

```
docker-compose build
# OR
make build
```

You should then restart your installation by running:

```
docker-compose down
docker-compose up -d
# OR
make restart
```

If you want to fully rebuild, restart, and re-run the setup process after a Docker image change, you can run all of those as one command:



On this page >

Building Static Assets

AzuraCast automatically builds static assets as they're changed in the dev environment. Modifications inside the `/frontend` folder are automatically watched and rebuilt when developing, then compiled down into static files as part of our Docker image build process.

Accessing `bash` Inside the Container

It is common to need to access the bash shell inside the running `web` container in order to see or modify the current state of the application while it's running.

There is a helper command you can run using the Docker Utility Script to easily accomplish this:

```
bash docker.sh bash
# OR
make bash
```

Customize Local SSL

Out of the box, AzuraCast comes with a self-signed SSL certificate that you can use for local development. Most browsers will complain about this, but you can add an exception to allow you to proceed anyway.

If you want to use a valid SSL certificate on your local installation, we highly recommend a tool called [mkcert](#).

We commonly use this tool to register a local SSL-enabled domain at `azuracast.local`. You will need to edit your `/etc/hosts` file to point this domain to your own computer as a one-time change.

The process for installing this certificate into the AzuraCast ecosystem looks like this command, run from the main repository's project root:



On this page >

To make your new certificate take effect, you will need to restart the Docker containers using:

```
docker-compose down
docker-compose up -d
# OR
make restart
```

Translations (Locales)

Locales are managed by the application in two places: the backend PHP code and the frontend (primarily Vue.js) JavaScript code. When new locales are added or translations are changed, they should be processed in both locations.

There are two steps to the translation process:

Generating New Translations

When new strings have been added to the application, they should be added to the .POT (localization template) file, so that our CrowdIn translation service recognizes it as a translatable string.

```
bash docker.sh cli locale:generate
# OR
make generate-locales
```

Import New Translated Strings



On this page >

```
bash docker.sh cli locale:import  
# OR  
make import-locales
```

 Edit page

← Previous
Translation

Next →
AzuraCast APIs

Installing AzuraCast on Windows

Installing AzuraCast on newer editions of Windows 10 (and Windows 11) is quite simple, thanks to the recent addition of the Windows Subsystem for Linux (WSL) version 2.

Installing with WSL2

**Tip**

Version 2 of the Windows Subsystem for Linux (WSL) will work on any edition of Windows, including Home editions.

Install Docker Desktop for Windows

The first step is to install WSL2 on your computer. One of the easiest ways to do this is to [download Docker Desktop for Windows](#).

When running the installer, you will be prompted to automatically set up your Windows installation to work with WSL2. Make sure this option remains checked, and it will direct you to the necessary steps to get WSL2 running for your installation.

Set WSL2 as the Default Version

Once WSL2 is installed and configured, and Docker Desktop is up and running, open a terminal window and type the following command:

```
wsl --set-default-version 2
```



On this page >

Install a Linux Distribution from the Windows Store

Now you should install a Linux distribution that will serve as the host of your AzuraCast instance. Per our [requirements](#), you can select from several operating systems as your host OS, but the one with the best compatibility and support is the latest LTS build of Ubuntu, which you can install from the link below:

- [Ubuntu on the Microsoft Store](#)

Select That Distribution as Default

For convenience, you can set the newly installed distribution as your "default" WSL2 distribution, which means it automatically gets integration with Docker Desktop for Windows.

You can list the names of all installed WSL2 distributions by running:

```
wsl --list
```

To set one as your default, enter its name in the command below. If you followed the recommendations here and used Ubuntu, you can run this command:

```
wsl --set-default Ubuntu
```

Open a Linux Shell

Your newly installed Linux distribution should be listed on the Start menu. Clicking the name of the distribution will open a terminal window with access to that distribution. If you're using Windows Terminal, you should also see a new option when creating a new tab to open a shell that connects to your installed Linux distribution.



On this page >

- [Docker Installation](#)

Installation Complete

Once your installation is complete, by default, you will be able to access AzuraCast from your host Windows operating system by visiting `http://localhost`.

 [Edit page](#)

Translation

Do you speak both English and another language? You can help us in a big way by helping translate AzuraCast!

AzuraCast is used around the world, and we want our web application to be accessible to users who aren't familiar with English, our primary language.

Thanks to the help of our friends at CrowdIn, translating the strings used in our application is easy! Just visit our translation page and create an account, and you can start submitting translations. CrowdIn also provides suggested translations in case you are unsure of certain words or phrases.

Submit Translations with CrowdIn



We do our best to incorporate translation changes as frequently as possible, but sometimes new updates are delayed. If you have completed a significant translation project, please feel free to give us a gentle reminder by creating a Github issue. Once the translations have been updated, we will close the issue to let you know.

 [Edit page](#)

← Previous
Suggest a New Feature

Next →
Getting Started

Reporting a Bug



Tip

You must follow [troubleshooting steps](#) prior to filing an issue on GitHub.

In our issue template, we ask for [logs](#) and other details that are part of the troubleshooting steps, and supplying these greatly helps us resolve issues.

Support for AzuraCast is only provided for the current [Stable](#) and up to date [Rolling-Release](#) versions. Make sure to update your installation before reporting an issue. We regularly publish updates that resolve issues and add new features.

If your issue isn't in our known issues or common solutions, check out our [GitHub Issues](#) board. We use GitHub to track the status of errors, bug reports and general questions.

If there's already an open GitHub issue for the specific issue you're having, you can follow that issue for updates. If not, you can create a new issue from the same page.

When creating a new issue, you'll be presented with a template of questions to answer to help our team provide quick and accurate support.

Please make sure to answer these questions and provide as much detail as possible!

Please include the following:

- Any steps that you have already attempted
- Any errors you may have encountered
- A full step-by-step guide on how to reproduce the issue
- Screenshots if applicable
- Relevant [logs \(click here for instructions on how to get them\)](#).



On this page >

← Previous
Optimizing

Next →
Troubleshooting

Using Custom JavaScript

Custom JavaScript for Public Pages

AzuraCast provides you with the possibility to add custom JS via the [Custom Branding](#) page at `/admin/branding`.

There you can add your own JavaScripts to the public pages with the editor for [Custom JS for Public Pages](#).

You can attach to events of the public player like this:

```
$(document).on('now-playing', function(np_new) {  
    // custom code with np_new  
});
```

Example: Video Playback for Public Page

You can use [Custom JS for Public Pages](#) to add a video element to the page and style via the [Custom CSS for Public Pages](#) using our example below.

Custom CSS for Public Pages

```
[data-theme] body.page-minimal .background-video {  
    width: 100vw;  
    height: 100vh;  
    object-fit: cover;  
    position: fixed;  
    left: 0;  
    right: 0;  
    top: 0;
```

[On this page >](#)

Custom JS for Public Pages

```
let videoBackgroundElement = document.createElement('video');
videoBackgroundElement.autoplay = true;
videoBackgroundElement.loop = true;
videoBackgroundElement.muted = true;
videoBackgroundElement.poster = 'Enter.URL.Here';
videoBackgroundElement.className = 'background-video';

let videoBackgroundSource = document.createElement('source');
videoBackgroundSource.src = 'Enter.URL.Here';
videoBackgroundSource.type = 'video/mp4';

videoBackgroundElement.appendChild(videoBackgroundSource);

document.body.append(videoBackgroundElement);
```

[✎ Edit page](#)

← Previous
Using Custom CSS

Next →
Docker

Remote Relays

Required Permissions

To manage remote relays, users must be in a role that has the “Manage Station Media” permission for the given station.

When you want to broadcast the radio signal created on your AzuraCast installation to a remote source, you should create a remote relay.

There are two primary types of remote relay:

- **Statistics Only:** You can add a relay, but not instruct the AutoDJ to connect to it; this is useful if the remote relay already connects to your main station and relays the broadcast itself. The relay's statistics will be included with your other station statistics automatically.
- **Full Relay:** In this type of relay, your AzuraCast installation's AutoDJ broadcasts its signal to the remote relay directly, and AzuraCast then includes the remote relay's statistics along with your other station statistics.

AzuraRelay

If you're serving radio traffic to a large volume of listeners and want to add redundant relays in the easiest way possible, we offer a separate project called [AzuraRelay](#) that is the perfect complement to AzuraCast itself.

Install AzuraRelay on a VPS of your choice, configure it to refer to the base AzuraCast installation, and it will automatically pull a list of available streams and relay them, while sending listener statistics back to the main installation.



On this page >

Take a look at our [documentation of the AzuraRelay](#) for installation instructions.

 Edit page

← Previous
Promoting Your Station

Next
SFTP Server →

Release Channels

For Docker installations, AzuraCast offers two different release channels:

- A “**Stable**” channel that only updates every few weeks or months and consists of changes that have been running on many installations without any issue for quite some time. These releases are tagged with version numbers (i.e. 0.10.3, 0.10.4).
- A “**Rolling Release**” channel that updates every time changes are made to the AzuraCast codebase, which can sometimes be as frequent as multiple times in a single day. This channel is useful if you’re testing new features, running a non-production environment, or need a bug fix that’s only recently been made available.

By default, our one-click installers for web hosts are set to prefer stable release builds. When installing yourself, you’ll be prompted with the question of whether to prefer stable releases during the initial installation process.

Switching Release Channels

Take a backup first!

Regardless of which direction you are switching, it is strongly recommended when switching channels to take a backup of your station’s data beforehand to more easily recover from any issues that may arise.

If you want to switch release channels on an existing installation, note the rules below before proceeding:

Switching from “Stable” to “Rolling Release”

[On this page >](#)

Switching from Rolling Release to Stable

You should only switch from "Rolling Release" to "Stable" when your rolling-release version is older than or the exact same version as the current stable version. In other words:

- If your rolling release version is dated October 2, 2020 and the stable version was released on October 5, you can switch channels and update without any issue, BUT
- If your rolling release version is dated October 7, 2020 and the stable version was released on October 5, you **should not** switch channels and should instead wait until the next stable release.

The reason this can be a problem is because of database migrations: new features implemented in the "Rolling Release" channel can include corresponding database changes, and an update to a "Stable" release that's older than your current one would require that you roll back database migrations; however, to the "Stable" version those migrations don't exist (since it is frozen in time from before they were created), causing your database to fall into an incorrect state.

As an important note, though, if you have mistakenly switched to the "Stable" channel and it has caused your database to go out-of-sync, it is highly likely that just switching back to the "Rolling Release" channel will resolve your issues and return your installation to normal. You can then wait until the next tagged "Stable" release to switch channels again.

Setting AzuraCast to Use a Different Channel

For Docker Installations

You can switch release channels by logging in to your host computer (the server running AzuraCast) via SSH and executing:

```
cd /var/azuracast
./docker.sh update-self
```



On this page >

You can then run the update script (`./docker.sh update`) and your installation will switch its active release channel.

For Ansible Installations

Ansible installations are directly tied to the Git repository, so switching channels is a matter of switching the branch that's checked out.

Switching to Rolling Release

```
git reset --hard
git checkout main
git pull
```

Switching to Stable

```
git reset --hard
git checkout stable
git pull
```

Edit page

Requirements

AzuraCast is powered by Docker and uses pre-built images that contain every component of the software. Don't worry if you aren't very familiar with Docker; our easy installer tools will handle installing Docker and Docker Compose for you, and updates are very simple.

A general disclaimer...

AzuraCast does not ensure, and is not liable for, music licenses and royalties paid to creators. Copyright and royalty laws vary widely from country to country around the world, and you will need to follow the appropriate laws for your location. If in doubt, consult with an attorney.



System Requirements

Minimum Requirements

- A 64-bit x86 (x86_64/amd64) or ARM64 CPU
- at least 2GB of RAM
- 20GB or greater of hard drive space
- A computer/server capable of running Docker

For Linux hosts, the `sudo`, `curl` and `git` packages should be installed before installing AzuraCast. Most Linux distributions include these packages already. This is the barest of minimum to run AzuraCast, there's no guarantees it will run perfectly.

Recommended Requirements

- 4 CPU cores



On this page >

Recommended Requirements should allow you to run a few stations with ease, this is aimed for hobby usage (5 to 10 stations). [If you wish to increase the memory even further, consult this guide on Swap Space](#)

Operating System



Tip

Regardless of which OS you choose, to avoid any compatibility or permissions issues you should install AzuraCast on a fresh minimal installation without Docker or Docker Compose installed.

We strongly recommend one of the following distributions and versions for your AzuraCast installation:

- Ubuntu 22.04 LTS (Docker only)
- Ubuntu 20.04 LTS (Ansible / Docker)
- Ubuntu 18.04 LTS (Docker only)
- Debian 10 "Buster" (Ansible / Docker)
- Debian 9 "Stretch" (Ansible / Docker)
- MacOS - Using Docker Desktop
- Windows 10 - Using Docker Desktop + WSL2
- Windows 11 - Using Docker Desktop + WSL2



Known Incompatibilities



On this page >

should consider using a different server for AzuraCast, or you can use the unsupported Ansible installation method.

- **CentOS:**

- Recent versions of CentOS are moving away from supporting Docker and instead supporting Podman, a different container-based solution. AzuraCast does not currently support being deployed on Podman. If you are selecting a Linux OS for your server, the LTS distributions of Ubuntu and Debian are strongly recommended.

- **MacOS incompatibilities with M1 CPUs:**

- Apple's flagship chip, the M1, is not fully compatible with AzuraCast and the Docker Desktop due to incompatibilities with processor architecture. We'll review this in the future to see if there's any workarounds.



Resource Usage

On the Administration page you can find a basic **Server Status** section that shows you the **CPU Load**, **Memory** and **Disk Space** usage of your server.

Icecast

For Icecast you can find some benchmarks for the following topics at <https://icecast.org/loadtest/>

- [Testing the maximum numbers of connected listeners](#)
- [Testing the maximum numbers of connected sources](#)
- [Icecast/Shoutcast comparison](#)

Edit page



On this page >

Using Custom CSS

An important change in newer versions...

In version 0.19.0, we switched from an old style system to a newer version of Bootstrap, and in the process, the prefix on the commands below changed from `[data-theme]` to `[data-bs-theme]`. You should update any custom CSS you're using accordingly.

AzuraCast provides you with the possibility to customize the default CSS via the [Custom Branding](#) page at `/admin/branding`.

There you can alter the styling of the internal and public pages via your own CSS with the editors for [Custom CSS for Public Pages](#) and [Custom CSS for Internal Pages](#).

This page will give you a simple overview of the CSS classes and ids used in the HTML of AzuraCast that you can then use in your own CSS to customize those elements.

If you have no prior experience with CSS take a look at the [MDN Web Docs](#) about CSS.

Note about Automatic Theme Detection

AzuraCast now has a built-in feature where it will automatically detect the user's preferred color scheme from their browser via a CSS media query. It's recommended that if you intend to customize your public pages, you disable this by visiting the [Custom Branding](#) page and setting a specific theme for public pages that isn't "Prefer System Default".

The instructions below assume that this change has been made already.

Custom CSS for Public Pages

[On this page >](#)

define CSS styles that should only apply to a single station by prefixing your CSS selectors like this:

```
# Targeting either dark or light themed pages
[data-bs-theme] body.station-my_radio {
    ...
}

# Targeting only light themed pages
[data-bs-theme="light"] body.station-my_radio {
    ...
}

# Targeting only dark themed pages
[data-bs-theme="dark"] body.station-my_radio {
    ...
}
```

Changing the background image

The CSS for the background image on public pages is defined like this:

```
# See above for theme-specific prefixes
[data-bs-theme] body.page-minimal {
    background: #edecec url(../img/hexbg.png);
    background-size: cover;
    background-attachment: fixed;
}
```

You can change the background image by using the same CSS selector and then just overwrite the `background` attribute like this:

```
[data-bs-theme] body.page-minimal {
    background: url(https://example.com/my_custom_background_image.png);
}
```




On this page >

You can change the background color of the radio player like this:

```
[data-bs-theme] body.page-minimal .public-page .card {  
    background-color: #5e0f16;  
}
```

The CSS selector for the title of the player is as follows:

```
[data-bs-theme] body.page-minimal .public-page .card .card-body .card-title {  
    ...  
}
```

The buttons at the bottom of the player can be styled with the following CSS selector:

```
[data-bs-theme] body.page-minimal .public-page .card .card-actions .btn {  
    ...  
}
```

The elements of the player itself are found in the `radio-player-widget` class and are separated further under `now-playing-details` and `radio-controls`.

As an example on how to customize those elements this page provides you with a short snippet for one element of each part of the player widget. For more detailed information it is recommended to directly look at the HTML of the page via your browsers HTML inspector.

This example shows you how to change the text color of the title and artist:

```
[data-bs-theme] body.page-minimal .public-page .card .now-playing-title {  
    color: #fff;  
}  
  
[data-bs-theme] body.page-minimal .public-page .card .now-playing-artist {  
    color: #fefefe;  
}
```



On this page >

```
[data-bs-theme] body.page-minimal .public-page .card .radio-control-play-button a {  
    color: #000;  
}
```

Styling the modals for Song History & Song Requests

To change the background color of the modals you can use the following CSS selector:

```
[data-bs-theme] body.page-minimal .modal .modal-content {  
    background-color: #5e0f16;  
}
```

To change the text color of the title use the following CSS selector:

```
[data-bs-theme] body.page-minimal .modal .modal-content .modal-title {  
    color: #fff;  
}
```

To change the background color of the pagination use the following CSS selector:

```
[data-bs-theme] body.page-minimal .modal .modal-content .pagination {  
    background: #3d3d3d;  
}
```

The text color of the Song Requests table header and rows can be changed like this:

```
[data-bs-theme] body.page-minimal .modal .modal-content .table thead {  
    color: #fff;  
}  
  
[data-bs-theme] body.page-minimal .modal .modal-content .table tbody {  
    color: #fff;  
}
```



On this page >

Custom CSS for internal pages

The internal pages can also be conditionally styled for the dark and light theme like it is done for the public pages:

```
[data-bs-theme="light"] body {  
    ...  
}  
  
[data-bs-theme="dark"] body {  
    ...  
}
```

Changing the color of the top navigation

If you want to change the color of the top navigation you can use the following CSS selector to use your own color and remove the background image:

```
[data-bs-theme] header.navbar {  
    background-image: none;  
    background: #3d3d3d;  
}
```

Changing the card header color

Most cards and some other elements in the internal pages have a blue background color. In order to change this color you can use the following CSS selector:

```
[data-bs-theme] .bg-primary-dark {  
    background-color: #5e0f16 !important;  
}
```



On this page >



Previous

Command-Line Interface

Next



Using Custom JavaScript

Install AzuraCast

Did you know?

AzuraCast is completely free for you to install and use, even for commercial purposes. We're a small team, and we depend on donations to support our work.

If you enjoy it, please consider [donating](#) to support us. We are a small team building a very big application, so your support is very important to us.

AzuraCast is server-based software that is installed on a VPS, dedicated server or other computer and serves both your station broadcasts and its own web interface through the server's network connection.

You can also install AzuraCast yourself on most server hardware and many computers running Linux.

One-Click Installers

The easiest way to get started with AzuraCast is to create a new VPS with one of our supported partners via the options below.

[Linode \(Akamai\)](#)[Vultr](#)[DigitalOcean](#)

Deploy a new Linode instance with the AzuraCast one-click installer in the Linode Marketplace.

[Linode AzuraCast One-Click Image](#)

Self-Hosted Installation

[On this page >](#)

Docker (Recommended)

Docker installation now supports ARM64 devices as well (i.e. newer Raspberry Pi devices, and ARM64-based cloud hosts), so you can follow these same installation steps.

For a majority of users, our Docker installation method is the preferred way of installing and using AzuraCast. Click the link below for Docker installation instructions.

[Install AzuraCast with Docker](#)

Docker App Installers

AzuraCast is also available via Docker-based app installer tools. These tools let you install AzuraCast alongside other Docker images in a simple visual manner.

[DockSTARTer](#)[CapRover](#)

Windows 10/11 (WSL2)

If you're looking to install AzuraCast on your Windows installation, this is now possible thanks to the Windows Subsystem for Linux (WSL2). Follow our Windows-specific guide for tips on how to get AzuraCast up and running easily on a Windows computer.

[Installing on Windows with WSL](#)

Synology NAS



On this page >

ARM processors are not officially supported.

Installing on a Synology NAS



 Edit page

Previous
← **Live Demo**

Next
Update AzuraCast →

Advanced Features

Power users only!

The primary reason we hide these features by default is because new users stumble upon them without realizing that they can have unintended consequences. These features give you enough power to break your station or installation, so they should be used with caution.

If you modify an advanced feature and it breaks something, your best recourse is to revert the changes you made. If you make custom configuration to Liquidsoap or Icecast and those changes don't work, you should use the Log Viewer utility to view the logs of those respective tools; the issue may be with either your code or a bug in those pieces of software.

Enabling Advanced Features

In new installations of AzuraCast, we turn off some of the more "dangerous" features which can have unintended consequences if misused by users who are unfamiliar with them.

Advanced features hidden by default include:

- Station custom port assignments (which are not cleared through the firewall by default in new Docker installations),
- Providing a custom media base directory (because only certain paths are mounted into the Docker filesystem),
- Customizing Icecast or Liquidsoap configuration (as errors caused by custom configuration are not caused by AzuraCast itself), and
- Advanced playlist settings (because they bypass AzuraCast's AutoDJ).



On this page >

advanced features of AzuraCast.

Broadcasting

Custom Configuration

A common use case of the station-wide Icecast config is to publish your station's mount points to [YellowPage directories](#).

Additionally, you can overwrite some values such as `<location>` and the `<admin>` email address.

 Edit page

← Previous
Streaming Software

Next
AzuraRelay →